

Proiectarea algoritmilor. O abordare practică

Traian Rebedea

Mihai Dascălu

Ștefan Trăușan-Matu

Prefață

Cartea de față este destinată studenților și pasionaților de informatică care doresc să afle detalii referitoare la proiectarea eficientă a algoritmilor. Focusul materialelor prezentate constă în aprofundarea elementelor necesare rezolvării fiecărei clase de probleme analizate, prin introducerea mai multor alternative de rezolvare și prin evidențierea algoritmilor optimi.

Metodele prezentate în capitolele următoare sunt de folos, în primul rând, studenților care urmează cursul de Proiectarea Algoritmilor predat la Facultatea de Automatică și Calculatoare din cadrul Universității Politehnica din București (ca un supliment pentru laborator, al cursului, unde sunt prezentate mai multe elemente teoretice, inclusiv formalizate), dar nu numai. Cartea poate fi utilă oricărui student sau practician care are nevoie de o trecere în revistă a principalelor tehnici de programare și a algoritmilor de bază necesari în cariera unui programator sau a unui inginer software.

Nivelul problemelor prezentate, precum și al discuțiilor din cadrul capitolelor următoare, este unul introductiv, însă sunt abordate și teme mai puțin cunoscute chiar și de către publicul mai experimentat în algoritmi, precum algoritmi pentru jocuri, căutări în spațiul stărilor, propagarea constrângerilor etc. Deși materialele prezentate sunt introductive, există câteva cerințe minimale pentru înțelegerea materialelor, precum noțiuni de bază de calcul al complexității algoritmilor (notații de complexitate și recurențe) sau de programare (pentru înțelegerea pseudocodului prezentat) și structuri de date (arbori binari de căutare, heap-uri, hashing etc.).

În capitolele următoare sunt prezentate mai multe șabloane de rezolvare a problemelor, corelate cu modalitățile de construire a soluțiilor aferente, iar algoritmi prezentați adresează următoarele subiecte:

- Divide et Impera, Greedy și Programare Dinamică;
- Backtracking și optimizări (prospective și retroactive);
- Algoritmi pentru jocuri (minimax);
- Parcurgeri pe grafuri (BFS, DFS) și sortare topologică;
- Căutări în spațiul stărilor (best-first, A*);

- Alte aplicații ale DFS (componente tare conexe, puncte de articulație și punți);
- Drumuri minime și arbori minimi de acoperire;
- Flux maxim;
- Algoritmi aleatori.

Suplimentar, anexa I prezintă principiile de bază de scriere a programelor (*coding style*) pentru asigurarea conciziei și clarității în implementarea problemelor. Fiecare capitol prezentat în această carte este însoțit de aplicații practice care pot fi rezolvate de către persoanele interesate cu scopul de a-și verifica cunoștințele acumulate.

Sperăm că această carte să vă ușureze familiarizarea cu algoritmii de bază utilizați în proiectarea algoritmilor și să vă ajute în dezvoltarea de soluții cât mai eficiente, complexe și elaborate.

Materialul prezentat a fost rafinat în multiple iterații pentru laboratorul materiei Proiectarea Algoritmilor de o multitudine de persoane printre care amintim pe: Costin Chiru, Vlad Posea, Clementin Cercel, Ștefan Rușeți, Claudia Cârdei, Valentin Stanciu, Claudiu Mihail, Filip Buruiană, Radu Iacob, Andrei Poenaru și alții.

Cuprins

Prefață	5
Cuprins	7
I Divide et Impera	11
I.1 Importanță și aplicații practice	11
I.2 Prezentarea generală a problemei	11
I.3 Concluzii și observații	15
I.4 Probleme propuse	15
I.5 Indicații de rezolvare	16
Referințe	18
II Greedy	19
II.1 Importanță și aplicații practice	19
II.2 Prezentarea generală a problemei	19
II.3 Concluzii și observații	22
II.4 Probleme propuse	23
II.5 Indicații de rezolvare	23
Referințe	24
III Programare Dinamică	26
III.1 Importanță și aplicații practice	26
III.2 Prezentarea generală a problemei	26
III.3 Concluzii și observații	30
III.4 Probleme propuse	30
III.5 Indicații de rezolvare	32
Referințe	33

IV	Backtracking și Optimizări	35
IV.1	Importanță și aplicații practice	35
IV.2	Descrierea problemei și a rezolvărilor	35
IV.3	Concluzii și observații	43
IV.4	Probleme propuse	43
IV.5	Indicații de rezolvare	44
	Referințe	44
V	Algoritmul Minimax pentru jocuri	46
V.1	Importanță și aplicații practice	46
V.2	Descrierea problemei și a rezolvărilor	46
V.3	Concluzii și observații	54
V.4	Probleme propuse	55
V.5	Indicații de rezolvare	55
	Referințe	56
VI	Parcurgerea Grafurilor. Sortarea Topologică	57
VI.1	Importanță și aplicații practice	57
VI.2	Descrierea problemei și a rezolvărilor	57
VI.3	Concluzii și observații	63
VI.4	Probleme propuse	64
VI.5	Indicații de rezolvare	65
	Referințe	65
VII	Aplicații DFS	66
VII.1	Importanță și aplicații practice	66
VII.2	Noțiuni teoretice	66
VII.3	Componente tare conexe	67
VII.4	Puncte de articulație	70
VII.5	Punți	71
VII.6	Componente biconexe	72

Cuprins	9
VII.7 Concluzii și observații	73
VII.8 Probleme propuse	73
VII.9 Indicații de rezolvare	73
Referințe	74
VIII Drumuri minime	75
VIII.1 Importanță și aplicații practice	75
VIII.2 Concepte	75
VIII.3 Drumuri minime de sursă unică	76
VIII.4 Drumuri minime între oricare două noduri	80
VIII.5 Cazuri speciale	82
VIII.6 Concluzii și observații	83
VIII.7 Probleme propuse	84
VIII.8 Indicații de rezolvare	84
Referințe	85
IX Arbori minimi de acoperire	86
IX.1 Importanță și aplicații practice	86
IX.2 Descrierea problemei și a rezolvărilor	86
IX.3 Concluzii și observații	91
IX.4 Probleme propuse	91
IX.5 Indicații de rezolvare	91
Referințe	92
X Flux Maxim	93
X.1 Importanță și aplicații practice	93
X.2 Descrierea problemei și a rezolvărilor	93
X.3 Variații ale problemei clasice	98
X.4 Concluzii și observații	99
X.5 Probleme propuse	100
X.6 Indicații de rezolvare	100

Referințe	101
XI Algoritmi euristici. Algoritmul A*	102
XI.1 Importanță și aplicații practice	102
XI.2 Descrierea problemei și a rezolvărilor	102
XI.3 Concluzii și observații	112
XI.4 Probleme propuse	112
XI.5 Indicații de rezolvare	113
Referințe	113
XII Algoritmi aleatori	115
XII.1 Aplicații practice	115
XII.2 Descrierea problemei și a rezolvărilor	115
XII.3 Concluzii și observații	119
XII.4 Probleme propuse	120
XII.5 Indicații de rezolvare	121
Referințe	121
Anexa 1 – Convenții de programare (Coding style)	122
Principii	122
Referințe	124

I Divide et Impera

I.1 Importanță și aplicații practice

Paradigma Divide et Impera (D&I, Dezbină și Stăpânește, Divide and Conquer) este o schemă utilizabilă în multe domenii, ea stând și la baza construirii de algoritmi eficienți pentru probleme variate. Ea presupune împărțirea problemei inițiale în sub-probleme, de obicei disjuncte, care pot fi rezolvate independent, iar apoi soluțiile lor sunt combinate pentru a obține soluția problemei inițiale.

Printre aplicațiile practice principale ale D&I enumerăm: sortări (de exemplu, algoritmi merge sort - sortarea prin interclasare [1], quicksort – sortarea rapidă [2]), înmulțirea numerelor mari (de exemplu, algoritmul lui Karatsuba [3]), analiza sintactică (de exemplu, analizoare sintactice de tip top-down [4]) sau calcularea transformatei Fourier discretă (de exemplu, algoritmul Fast Fourier Transform – FFT – pentru înmulțirea rapidă a polinoamelor [5]). Un alt domeniu de utilizare a tehnicii divide et impera este programarea paralelă, datorită faptului că sub-problemele fiind independente, ele pot fi rezolvate pe mașini/procesoare diferite.

I.2 Prezentarea generală a problemei

Proiectarea unui algoritm de tip Divide et Impera (D&I) presupune trei pași fundamentali [6]:

- **Divide:** împarte problema în una sau mai multe *probleme similare* cu problema inițială, de *dimensiuni mai mici*;
- **Impera (stăpânește):** rezolvă subprobleme recursiv; dacă dimensiunea sub-problemelor este mică, acestea sunt rezolvate iterativ;
- **Combină:** combină soluțiile sub-problemelor, de obicei disjuncte, în vederea generării soluției problemei inițiale.

Complexitatea algoritmilor de tip D&I poate fi exprimată folosind următoarea relație:

$$T(n) = D(n) + S(n) + C(n),$$

unde $D(n)$, $S(n)$ și $C(n)$ reprezintă complexitățile celor 3 pași descriși mai sus, respectiv Divide, Stăpânește și Combină.

Căutarea binară

Pornind de la un **vector sortat crescător** ($v[1..n]$) care conține valori reale distincte și o valoare x , se dorește identificarea poziției la care se regăsește elementul x în vectorul dat. Pentru rezolvarea acestei probleme folosim un algoritm D&I:

- **Divide:** vectorul curent este împărțit în doi sub-vectori de dimensiune $n/2$.
- **Stăpânește:** aplicăm algoritmul de căutare binară pe sub-vectorul care conține valoarea căutată.
- **Combină:** soluția sub-problemei devine soluția problemei inițiale, motiv pentru care nu mai este nevoie de etapa de combinare.

Pseudocod:

```

BinarySearch(v, p, q, x)
    // x nu se află în v, condiție de oprire
    if (p > q) return;
    // etapa divide
    m = (p + q) / 2;
    // etapa stăpânește
    if (v[m] == x) return m
    if (v[m] > x) return BinarySearch(v, p, m-1, x);
    if (v[m] < x) return BinarySearch(v, m+1, q, x);

```

Complexitatea algoritmului este dată de relația $T(n) = T(n/2) + O(1)$, ceea ce implică, de exemplu, aplicând teorema Master [6, 7]: $T(n) = O(\log n)$.

Sortarea prin interclasare

Sortarea prin interclasare (*merge sort*) [1] este un algoritm de sortare a unui vector ce folosește paradigma D&I astfel:

- **Divide:** vectorul inițial, de dimensiune n , este împărțit în doi sub-vectori de dimensiune egală $n/2$ (dacă dimensiunea vectorului inițial – n – nu este pară vor rezulta doi sub-vectori de dimensiune aproximativ egală nefiind important care este mai mare cu un element);

- **Stăpânește:** cei doi sub-vectori sunt sortați recursiv folosind sortarea prin interclasare; recursivitatea se oprește când dimensiunea unui sub-vector este 1, vector implicit sortat;
- **Combină:** cei doi sub-vectori sortați sunt interclasați pentru a recompu vectorul inițial sortat.

În Figura 1.1 sunt reprezentați pașii efectuați pentru sortarea unui vector cu 8 elemente, scoțând în evidență atât apelurile recursive (etapele divide și stăpânește), cât și interclasarea soluțiilor subproblemelor (etapa combină) la revenirea din recursivitate.

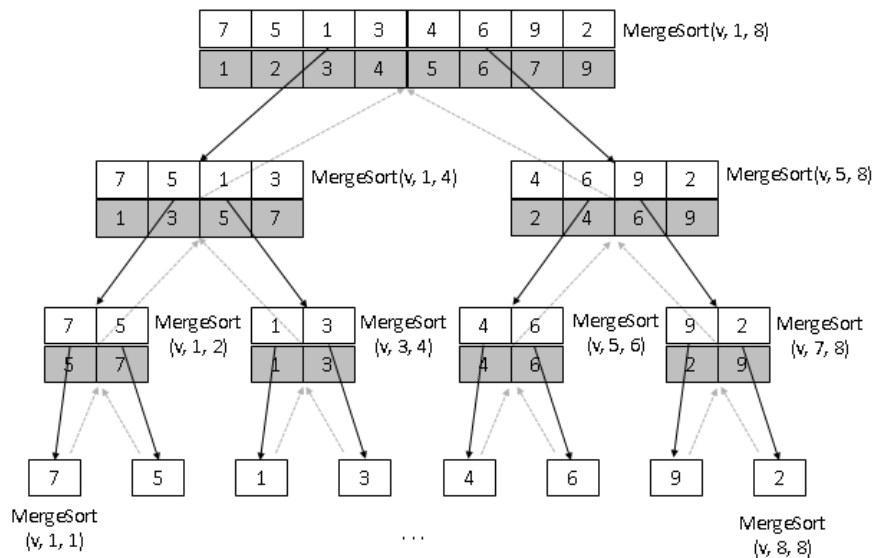


Figura 1.1 - Exemplu de rulare a algoritmului de sortare prin interclasare. Cu săgeți pline sunt evidențiate apelurile recursive, iar cu săgeți punctate revenirea din recursivitate și combinarea soluțiilor subproblemelor

Pseudocod:

```

MergeSort(v, p, q) // v - vector, p - limită inferioară,
                  // q - limită superioară
    if (p == q) return; // condiția de oprire
    m = (p + q) / 2;    // etapa divide
    MergeSort(v, p, m); // etapa stăpânește
    MergeSort(v, m+1, q);
    Merge(v, p, q);    // etapa combină

```

```

Merge(v, p, q)    // interclasare sub-vectori
    m = (p + q) / 2;
    i = p;
    j = m + 1;
    k = 1;
    while (i <= m && j <= q)
        if (v[i] <= v[j]) u[k++] = v[i++];
        else u[k++] = v[j++];
    while (i <= m)
        u[k++] = v[i++];
    while (j <= q)
        u[k++] = v[j++];
    copy(v[p..q], u[1..k-1]);

```

Complexitatea algoritmului este dată de formula: $T(n) = D(n) + S(n) + C(n)$, unde $D(n) = O(1)$, $S(n) = 2 * T(n/2)$ și $C(n) = O(n)$. Astfel vom avea următoarea relație de recurență $T(n) = 2 * T(n/2) + O(n)$. Aplicând teorema Master [6, 7], rezultă că complexitatea algoritmului este $T(n) = O(n * \log n)$.

Turnurile din Hanoi

Se consideră trei tije A, B, C și n discuri de dimensiuni distincte (1, 2, ... n în ordinea crescătoare a dimensiunilor), situate inițial toate pe tija A în ordinea 1, 2, ... n (de la vârf către bază). Singura operație care se poate efectua este de a selecta un disc ce se află în vârful unei tije și plasarea lui în vârful altei tije astfel încât acesta să fie așezat deasupra unui disc de dimensiune mai mare decât a sa. În acest context, trebuie implementat un algoritm care mută toate discurile de pe tija A pe tija B (problema turnurilor din Hanoi), putând folosi tija C ca suport intermediar. Pentru rezolvarea problemei folosim următoarea strategie [7]:

- Mutăm primele $n-1$ discuri de pe tija A pe tija C folosind tija B;
- Mutăm discul n pe tija B;
- Mutăm apoi cele $n-1$ discuri de pe tija C pe tija B folosindu-ne de tija A.

Pseudocod [7]:

```

Hanoi(n, A, B, C)
// mută n discuri de pe tija A pe tija B folosind tija C
// ca tija intermediară
    if (n >= 1)
        Hanoi(n-1, A, C, B);
        Muta_disc(A, B);
        Hanoi(n-1, C, B, A);

```

Complexitate: $T(n) = 2 * T(n-1) + O(1)$, recurență ce conduce la soluția $T(n) = O(2^n)$.

Obs: Acest algoritm poate fi considerat tot de tip D&I dar, spre deosebire de cei descriși anterior, vedem că are o complexitate exponențială, deoarece subproblemele în care se face împărțirea nu sunt decât cu o unitate mai mici. Situațiile acestea particulare, în care dimensiunea sub-problemelor scade cu un termen (complexitate uzuală $T(n) = a * T(n-b) + f(n)$) și nu se reduce cu un factor (complexitate uzuală $T(n) = a * T(n/b) + f(n)$), se mai numesc și rezolvări de tipul descrește și stăpânește (*decrease and conquer*, în loc de divide și stăpânește).

I.3 Concluzii și observații

Divide et impera este o tehnică eficientă de rezolvare a problemelor bazată trei etape principale: *divide*, *stăpânește* și *combină*. Pe web se pot găsi mai multe exemple de algoritmi care folosesc tehnica divide et impera [8] pentru o gamă variată de probleme.

I.4 Probleme propuse

1. Înmulțirea matricelor folosind algoritmul lui Strassen

Să se scrie un algoritm care calculează produsul a două matrice ($C = A * B$), folosind metoda divide-et-impera. Se va folosi următoarea abordare: o matrice $n \times n$ se poate scrie ca o matrice de dimensiune 2×2 ce conține ca elemente submatrice de dimensiuni $n/2 \times n/2$. Pentru a înmulți două matrice 2×2 se folosesc doar 7 înmulțiri de submatrice, după cum este definit în continuare.

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} * \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

Definim următoarele rezultate intermediare obținute prin înmulțirea submatricelor:

$$\begin{aligned} M_1 &= (A_{11} + A_{22}) * (B_{11} + B_{22}) \\ M_2 &= (A_{21} + A_{22}) * B_{11} \\ M_3 &= A_{11} * (B_{12} - B_{22}) \\ M_4 &= A_{22} * (B_{21} - B_{11}) \end{aligned}$$

$$\begin{aligned} M5 &= (A11 + A12) * B22 \\ M6 &= (A21 - A11) * (B11 + B12) \\ M7 &= (A12 - A22) * (B21 + B22) \end{aligned}$$

Folosind rezultatele intermediare putem obține submatricele care alcătuiesc rezultatul:

$$\begin{aligned} C11 &= M1 + M4 - M5 + M7 \\ C12 &= M3 + M5 \\ C21 &= M2 + M4 \\ C22 &= M1 - M2 + M3 + M6 \end{aligned}$$

2. Elementul median dintr-un vector

Fie o mulțime de numere reale cu n elemente reprezentată cu un vector v nesortat. Scrieți un algoritm de complexitate $O(n)$ care să determine elementul median al mulțimii (echivalentul elementului de pe poziția $n/2$ din vectorul sortat, sau al k -lea ($k = n/2$) cel mai mic element din vector).

3. Subsecvența de sumă maximă

Fie un șir $S = (s_1, s_2, \dots, s_N)$ de lungime n cu numere reale. O subsecvență a șirului este de forma: $(s_i, s_{i+1}, \dots, s_j)$ cu $1 \leq i \leq j \leq n$, iar suma subsecvenței este definită ca $S_{ij} = s_i + s_{i+1} + \dots + s_j$. Se cere să se determine subsecvența de sumă maximă folosind un algoritm divide-et-impera.

Exemplu:

Pentru $n = 7$ și secvența: (5 -6 3 4 -2 3 -3), subsecvența de sumă maximă este: (3 4 -2 3) a cărei sumă este 8.

Obs: Varianta cerută folosind o abordare de tip Divide et Impera are o complexitate $O(n * \log n)$, mai mare față de alternativa Greedy care este $O(n)$.

I.5 Indicații de rezolvare

1. Înmulțirea matricelor folosind algoritmul lui Strassen

Inițial se vor adăuga linii și coloane cu elemente nule, până când cele două matrice vor fi pătrate, de dimensiune putere a lui 2, $n = 2^k$.

Se partiționează matricele în blocuri de dimensiuni egale, ca în desenul de mai sus. Pentru a calcula cele 7 matrice intermediare M_{1-7} , se apelează recursiv procedura de înmulțire (fiecare matrice M este produsul a altor două matrice de dimensiune mai mică $n/2 = 2^k/2 = 2^{k-1}$).

Complexitate:

Relația de recurență va fi $T(n) = 7 * T(n/2) + n^2$

Aplicând teorema Master, vom obține complexitatea algoritmului lui Strassen: $T(N) = O(n^{\log_2 7}) \approx O(n^{2.8})$

2. Elementul median dintr-un vector

Ne propunem să rezolvăm generalizarea problemei – identificarea celui de-al k -lea cel mai mic element din vectorul inițial, urmând ca apelul final să folosească $k = n/2$). Această problemă se mai numește și problema selecției celui de-al k -lea element sau statistici de ordine.

Astfel, se pornește de la funcția de partiționare a algoritmului Quicksort. Fie q indexul returnat de această funcție. Dacă $k = q$ atunci înseamnă că s-a găsit elementul căutat ($v[k]$). Dacă $q > k$, se va apela recursiv funcția de selecție pentru $v[p \dots q-1]$, în caz contrar, se apelează pentru $v[q+1 \dots r]$, unde p și r reprezintă capetele șirului inițial.

Complexitate: $O(n)$, dacă se folosește partiționare aleatoare.

3. Subsecvența de sumă maximă

Se divide șirul inițial în două subșiruri de lungime egală. Se calculează subsecvența de sumă maximă pentru fiecare din cele două subșiruri. Pentru a combina soluțiile rezultate, trebuie să aflăm și subsecvența de sumă maximă ce are capetele în fiecare din cele două subșiruri. Astfel, se va alipi sufixul de sumă maximă din primul subșir cu prefixul de sumă maximă din cel de-al doilea subșir. Această operație de combinare a subsoluțiilor are complexitatea $O(n)$ în cel mai defavorabil caz.

Complexitate: $O(n * \log n)$

Referințe

- [1] Algoritmul MergeSort, disponibil la <http://www.sorting-algorithms.com/merge-sort>
- [2] Algoritmul QuickSort, disponibil la <http://www.sorting-algorithms.com/quick-sort>
- [3] Karatsuba, A. and Ofman, Yu. (1962). Multiplication of Many-Digital Numbers by Automatic Computers. Proceedings of the USSR Academy of Sciences 145: 293–294. Translation în the academic journal Physics-Doklady, 7 (1963), pp. 595–596
- [4] Frost, R., Hafiz, R. and Callaghan, P. (2007) Modular and Efficient Top-Down Parsing for Ambiguous Left-Recursive Grammars. În 10th International Workshop on Parsing Technologies (IWPT), ACL-SIGPARSE, 109 - 120, June 2007, Prague.
- [5] Brigham, E. O. (2002). The Fast Fourier Transform. New York: Prentice-Hall.
- [6] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). Introduction to Algorithms (3rd ed.). Cambridge, MA: MIT Press.
- [7] Giumale, C. (2004) Introducere în Analiza Algoritmilor. Bucuresti: Polirom
- [8] Divide-and-conquer algorithms, Chapter 2, Berkeley University, disponibil la <http://www.cs.berkeley.edu/~vazirani/algorithms/chap2.pdf>

II Greedy

II.1 Importanță și aplicații practice

În general, tehnicile Greedy [1-3] și Programare Dinamică (prezentată în capitolul următor) sunt folosite pentru rezolvarea problemelor de optimizare. Aceste tehnici pot adresa probleme în sine sau pot fi subprobleme dintr-un algoritm mai complex. De exemplu, algoritmul Dijkstra pentru determinarea drumului minim într-un graf selectează un nod nou la fiecare pas folosind o alegere lacomă (*greedy*) ce ține cont doar de optimul pentru pasul curent. Schema Greedy de algoritmi mai este folosită și în codificarea Huffman sau în găsirea unui arbore minim de acoperire a unui graf.

Exista însă probleme care ne pot induce în eroare. Astfel, există probleme în care urmărind criteriul Greedy de optim local nu ajungem la soluția optimă global. Este foarte important să identificăm cazurile când se poate aplica tehnica Greedy și cele când este necesară o altă abordare.

Totuși, această soluție neoptimă întoarsă de un algoritm greedy incorect poate reprezenta o aproximare suficientă în anumite cazuri specifice, pentru probleme foarte dificile. De exemplu, problemele NP-complete necesită un timp de execuție de cele mai multe ori inacceptabil pentru a găsi optimul absolut întrucât, cel puțin în acest moment, nu se cunosc soluții de complexitate polinomială pentru ele. Pentru a rezolva astfel de probleme într-un timp rezonabil, putem găsi o soluție apropiată de cea optimă folosind algoritmi de tip Greedy.

II.2 Prezentarea generală a problemei

Termenul „greedy” se traduce prin „lacom”. Astfel, algoritmii de tip greedy vor să construiască într-un mod cât mai rapid soluția unei probleme. Ei se caracterizează prin luarea unor decizii rapide care duc la găsirea unei soluții potențiale a problemei. Însă, asemenea decizii rapide nu duc întotdeauna la o soluție optimă (sau corectă). De aceea, ne vom concentra pe identificarea anumitor tipuri specifice de probleme pentru care se pot obține soluții optime folosind alegeri „lacome” („greedy”). În general, aplicarea acestei metode, în cazul unei probleme de care nu se știe sigur că poate fi rezolvată în manieră „greedy”, trebuie făcută doar după demonstrarea matematică a faptului că

aplicarea ei duce la găsirea soluției corecte și nu blocarea într-o soluție ne-optimală.

Algoritmii Greedy se numără printre cei mai direcți algoritmi posibili. Ideea de bază este simplă: având o problema de optimizare, de calcul al unui cost minim sau maxim, se va alege la fiecare pas decizia cea mai favorabilă, fără a evalua global eficiența soluției. Această cea mai favorabilă alegere la un anumit pas se numește alegere greedy sau optimă local. În general, există mai multe soluții posibile ale unei probleme de optimizare. Dintre acestea, idealul este să se selecteze doar soluții optime, conform unor anumite criterii sau, dacă nu este posibil, o soluție cât mai apropiată, conform criteriului optimal impus.

Descrierea formală a unui algoritm greedy poate fi reprezentată, în mod simplist, folosind următorul pseudocod:

greedy (C)

```
// C este mulțimea candidaților; în S construim soluția
// parțială a problemei
S = ∅
while (!soluție(S) and C≠∅)
    x = un element din C care minimizează/maximizează
select(x)
    C = C \ {x}
    if (fezabil(S ∪ {x}))
        S = S ∪ {x}
// întoarcem soluția finală
if (soluție(S))
    return S
print "nu am găsit soluție finală folosind greedy"
```

Abordarea de tip "greedy" devine evidentă: la fiecare pas se alege cel mai bun candidat de la momentul respectiv, fără a studia impactul viitor al alternativelor disponibile în moment respectiv. Astfel, se renunță la analiza impactului ulterior al unei decizii sau la viabilitatea unei selecții în timp. Dacă un candidat este inclus în soluție, alegerea este univocă și acesta rămâne parte din soluția finală fără a putea fi modificat, iar dacă este exclus din soluție, nu va mai putea fi selectat niciodată drept un potențial candidat.

Subșirul de sumă maximă

Fie un șir de n numere pentru care se cere determinarea unui subșir de numere cu suma maximă. Un subșir al unui șir este format din elemente (nu neapărat consecutive) ale șirului respectiv, în ordinea în care acestea apar în șir. Pentru numerele 1 -5 6 2 -2 4 răspunsul este 1 6 2 4 (suma 13).

Soluție: Pentru această problemă tot ce avem de făcut este să verificăm dacă fiecare număr este pozitiv sau nu. În cazul pozitiv, acesta este introdus în subșirul soluției.

Complexitate: $O(n)$

Problema cuielor

Fie n scânduri de lemn, descrise sub formă de intervale închise cu capete reale. Găsiți o mulțime minimă de cuie (de poziții - numere) cu care să fie prinse una sau mai multe scânduri, astfel încât fiecare scândură să fie bătută de cel puțin un cui. Formal, trebuie găsită o mulțime de puncte de cardinal minim, M , astfel încât pentru orice interval $[a_i, b_i]$ din cele n intervale trebuie să existe cel puțin un punct $x \in M$ care să aparțină intervalului $[a_i, b_i]$.

Complexitate: $O(n \cdot \log n)$

Exemplu:

- intrare: $n = 5$, intervalele: $[0, 2]$, $[1, 7]$, $[2, 6]$, $[5, 14]$, $[8, 16]$
- ieșire: $M = \{2, 14\}$; Explicație: punctul 2 se află în primele 3 intervale, iar punctul 14 în ultimele 2.

Soluție: Se observă că dacă x este un punct din M care nu este capăt dreapta al nici unui interval, o translație a lui x la dreapta care îl duce în capătul dreapta cel mai apropiat nu va schimba intervalele care conțin punctul. Prin urmare, există o mulțime de cardinal minim M pentru care toate punctele x sunt capete dreapta.

Astfel, vom crea mulțimea M folosind numai capete dreapta în felul următor:

- cât timp au mai rămas intervale nemarcate:
 - selectăm cel mai mic capăt dreapta al unui interval nemarcat, $B_{\min}$; acesta trebuie să fie în M , deoarece este singurul punct care se află în interiorul intervalului care se termină în $B_{\min}$;
 - marcăm toate intervalele nemarcate care conțin $B_{\min}$;
 - adăugăm $B_{\min}$ la M .

Pentru a obține o complexitate redusă, sortăm inițial toate cele $2n$ capete și le parcurgem de la stânga la dreapta. Pentru fiecare punct distingem cazurile:

- dacă este capăt stânga, introducem intervalul în lista de „intervale în procesare” și trecem mai departe;
- dacă este capăt dreapta și intervalul respectiv nu conține nici un punct din M , atunci am găsit cel mai mic capăt dreapta al unui interval nemarcat, introducem capătul în M și marcăm toate intervalele din lista de intervale în procesare;
- dacă este capăt dreapta și intervalul din care face parte este deja marcat, trecem mai departe.

Complexitate:

- sortare: $O(n * \log n)$;
- parcurgerea capetelor: $O(n)$;
- adăugarea și ștergerea unui interval din lista de intervale în procesare: $O(1)$;
- total: $O(n \log n)$

II.3 Concluzii și observații

Tehnica Greedy reprezintă o abordare rapidă, de complexitate scăzută, care conduce, în general, la un optim local al unei probleme de optimizare. Folosirea criteriului alegerii locale greedy nu garantează atingerea unui optim global. Astfel, trebuie înțeles faptul că rezultatul obținut este optim doar dacă alegerea locală făcută de algoritm conduce la optimul global. În cazul în care deciziile de la pasul curent influențează lista de decizii de la pasul următor (sau pașii următori), este posibilă obținerea unei valori sub-optimale. În astfel de cazuri, pentru găsirea unui optim absolut se poate ajunge inclusiv la soluții supra-polinomiale. De aceea, dacă se optează pentru o soluție greedy, algoritmul trebuie însoțit de o demonstrație a corectitudinii.

Totuși, există o gamă largă de probleme pentru care algoritmi de tip greedy pot garanta optimalitatea soluției găsite. În cadrul cursului de Proiectarea Algoritmilor sunt prezentate mai multe detalii, incluzând legătura între matroizi și soluțiile greedy.

II.4 Probleme propuse

1. Problema rucsacului (varianta continuă)

Un camion poate transporta T tone de material. Există n tipuri de materiale disponibile, fiecare caracterizat de greutatea $G[i]$ disponibilă și de valoarea $V[i]$ adusă de transportarea sa ($1 \leq i \leq n$). Să se decidă ce cantități din fiecare material vor fi transportate pentru a maximiza valoarea totală a produselor transportate de către camion.

2. Planificarea temelor de către studenți

Pe parcursul unui semestru, un student are de rezolvat n teme. Se cunoaște enunțul tuturor celor n teme de la începutul semestrului. Timpul de rezolvare pentru oricare dintre teme este de o săptămână și nu se poate lucra la mai multe teme în același timp. Pentru fiecare temă, se cunoaște și un termen limită $d[i]$ (exprimat în săptămâni) și un punctaj $w[i]$. Nicio fracțiune din punctaj nu se mai poate obține după expirarea termenului limită al unei teme. Să se definească o planificare de realizare a temelor, în așa fel încât punctajul obținut la final să fie maxim.

3. Numere de tip palindrom

Se dorește construirea a două numere de tip palindrom (prin inversare – citirea de la dreapta la stânga – numărul rămâne la fel), primul de lungime $L > 0$, al doilea de lungime $l > 0$ astfel încât suma lor să fie minimă. Cele două numere de tip palindrom nu pot să înceapă cu cifra 0. Pentru construirea lor, se dau numărul L și numărul de apariții al fiecărei cifre în cele două numere (adică $c[i] = \text{nr. de apariții al cifrei } i \text{ în primul număr} + \text{nr. de apariții al cifrei } i \text{ în al doilea număr}$, $0 \leq i \leq 9$).

II.5 Indicații de rezolvare

1. Problema rucsacului (varianta continuă)

Se calculează profitul obținut per unitatea de material transportată $V[i]/G[i]$ ($1 \leq i \leq n$) și se încarcă material în ordinea acestor valori.

În plus, se poate observa că se obține o îmbunătățire a complexității – de la $O(n^2)$ la $O(n * \log n)$ – dacă valorile sunt sortate descrescător înainte de a face

alegerile. Folosirea unei preprocesări simple (de exemplu, o sortare) în etapa inițială pentru a reduce complexitatea algoritmului este o proprietate a mai multor algoritmi construiți folosind tehnica Greedy.

Complexitate: $O(n * \log n) + O(n) = O(n * \log n)$

2. Planificarea temelor de către studenți

Se folosește un heap cu minim în vârf pentru a prioritiza temele în funcție de punctaj și de proximitatea termenului limită. Alternativ, pentru valori întregi și mici, se pot sorta temele după punctaje, în ordine descrescătoare și, în aceasta ordine, li se caută un slot de rezolvare, pornind de la termenul limită și căutând spre stânga.

Dacă în loc de maximizarea punctajului total obținut, se dorește maximizarea numărului de teme ce pot fi rezolvate, se sortează lista de intervale de rezolvare după termenul limită și se alege întotdeauna tema care se termină cel mai curând, în așa fel încât să nu se suprapună cu nicio altă temă aleasă în prealabil.

Complexitate: $O(n * \log n)$

3. Numere de tip palindrom

Se observă că cifrele mai mici trebuie să ocupe poziții cât mai semnificative, ceea ce asigură că suma celor două numere să fie minimă. Pentru a realiza acest lucru se vor completa *în paralel* cele două numere cu cifrele parcurse în ordine crescătoare stabilind în care din cele două numere se vor poziționa. De asemenea, trebuie avut în vedere că niciunul din cele două numere să nu înceapă cu cifra 0.

Complexitate: $O(L + l)$

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). Introduction to Algorithms (3rd ed.). Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) Introducere în Analiza Algoritmilor. Bucuresti: Polirom

[3] Metoda Greedy, materiale suplimentare disponibile la adresa:
<http://www3.algorithmdesign.net/handouts/Greedy.pdf>

III Programare Dinamică

III.1 Importanță și aplicații practice

Similar cu Greedy, tehnica de Programare Dinamică este folosită, în general, pentru rezolvarea problemelor de optimizare. Programarea dinamică are un câmp larg de aplicare, aici amintind genetica (aliniera de secvențe de nucleotide sau proteine, *sequence alignment*), căutarea optimă (arbori optimi la căutare), teoria grafurilor (algoritmul Floyd-Warshall), metode de antrenare a rețelelor neurale (Adaptive Critic training strategy), limbaje formale și automate (algoritmul Cocke-Younger-Kasami, care analizează dacă și în ce fel un șir poate fi generat de o gramatică independent de context), sau implementarea bazelor de date (algoritmul Selinger pentru optimizarea interogărilor relaționale).

III.2 Prezentarea generală a problemei

Programarea dinamică presupune rezolvarea unei probleme prin descompunerea ei în subprobleme similare, rezolvarea acestora și combinarea subsoluțiilor. Spre deosebire de divide et impera, subproblemele nu sunt disjuncte, ci de multe ori acestea chiar sunt identice sau se suprapun.

Pentru a evita recalcularea porțiunilor care se suprapun, rezolvarea se face pornind de la cele mai mici subprobleme și, folosindu-ne de rezultatul acestora, iterativ calculăm subprobleme de dimensiune mai mare folosind soluții mai mici, deja calculate. Cele mai mici subprobleme sunt denumite subprobleme unitare, acestea putând fi rezolvate într-o complexitate constantă (de exemplu, cea mai mare subsecvență dintr-o mulțime de un singur element).

Pentru a nu recalcula soluțiile subproblemelor care ar trebui rezolvate de mai multe ori, pe ramuri diferite, se reține soluția subproblemelor folosind o tabelă (matrice uni, bi sau multi-dimensională în funcție de problemă) cu rezultatul fiecărei subprobleme. Tehnica utilizată, denumită **memorizare**, determină și memorează „valoarea” soluției pentru fiecare din subproblemele deja rezolvate. Mergând de la subprobleme mici la subprobleme din ce în ce mai mari ajungem la soluția optimă, la nivelul întregii probleme.

Tehnica de Programare Dinamică este caracterizată prin flexibilitatea ei, dar funcția de optimizare a soluțiilor subproblemelor este diferită de la o problemă la alta. După calcularea valorii pentru toate subproblemele se poate determina efectiv mulțimea de elemente care compun soluția. Prin aplicarea tehnicii de programare dinamică se determină *una* din soluțiile optime ale problemei, deși aceasta poate avea mai multe soluții optime. În cazul în care se dorește determinarea tuturor soluțiilor optime, algoritmul trebuie combinat cu unul de backtracking în vederea construcției tuturor soluțiilor. Metoda de combinare a subsoluțiilor și recurența generată de către aceasta variază în funcție de problemă, acesta fiind una dintre principalele dificultăți în găsirea unei soluții folosind programare dinamică.

Aplicarea tehnicii de Programare dinamică poate fi descompusă în următoarea secvență de pași:

1. Identificarea structurii și a metricilor utilizate în caracterizarea soluției optime pentru problema principală și pentru subprobleme;
2. Determinarea unei metode de calcul recursiv pentru a afla soluția optimă fiecărei subprobleme plecând de la subprobleme mai mici;
3. Calcularea „bottom-up” a soluțiilor subproblemelor (de la subproblemele cele mai mici la cele mai mari);
4. Reconstrucția soluției optime a unei probleme de dimensiune mai mare pornind de la rezultatele subproblemelor obținute anterior.

Programarea Dinamică este o tehnică de programare extrem de flexibilă. În continuare prezentăm câteva probleme semnificative pentru înțelegerea acestei tehnici.

Cel mai lung subșir strict crescător

O problemă clasică de Programare Dinamică presupune determinarea celui mai lung subșir strict crescător dintr-un șir de n numere. Un subșir al unui șir este format din elemente (nu neapărat consecutive) ale șirului respectiv, în ordine crescătoare a aparițiilor/indicilor în șir.

Exemplu: pentru șirul 24 12 15 15 8 19 cel mai lung subșir strict crescător este șirul 12 15 19.

Soluție: Se observă, în primul rând, că o abordare greedy nu poate determina nici măcar elementul de început într-un mod corect. Totuși, problema se poate rezolva într-o manieră directă, inefficientă, folosind un algoritm care alege toate combinațiile de numere din șir, validează că șirul obținut este strict

crescător și îl reține pe cel de lungime maximă, dar această abordare are complexitatea temporală $O(n!)$. Cu optimizări, este posibilă atingerea unei complexități de $O(2^n)$.

O metodă de rezolvare mai eficientă folosește *Programarea Dinamică*. Începem prin a stabili pentru fiecare element lungimea celui mai lung subșir strict crescător care începe cu primul element și se termină în elementul respectiv. Numim această valoare $best_i$ și aplicăm formula recursivă $best_i = 1 + \max(best_j)$, cu $j < i$ și $elem_j < elem_i$.

Aplicând acest algoritm obținem:

elem	24	12	15	15	8	19
best	1	1	2	2	1	3

Pentru numerele 24 sau 12 nu există nici un alt element în stânga lor strict mai mic decât ele, de aceea au $best$ egal cu 1. Pentru elementele 15 se poate găsi în stânga lor numărul 12 strict mai mic decât ele. Pentru numărul 19 se găsește elementul 15, strict mai mic decât el. Cum 15 deja este capăt pentru un subșir soluție de 2 elemente, putem spune că 19 este capătul pentru un subșir soluție de 3 elemente.

Cum pentru fiecare element din mulțime trebuie să găsim un element mai mic decât el și cu $best$ maxim, avem o complexitate $O(n)$ pentru fiecare element. Practic, rezolvarea unei subprobleme ($best_i$) necesită o complexitate $O(n)$. La nivel global, fiind n subprobleme ($1 \leq i \leq n$), rezultă o complexitate $O(n^2)$. Se pot obține și rezolvări cu o complexitate mai mică folosind structuri de date mai avansate. Atât soluția în $O(n^2)$, cât și o soluție optimizată cu complexitatea $O(n * \log n)$ pot fi găsite pe infoarena [4]. Tot acolo se poate găsi și o listă de probleme mai dificile ce folosesc tehnica Programării Dinamice, adaptată în diferite forme.

Pentru a găsi elementele care alcătuiesc subșirul strict crescător, putem să reținem și o „cale de întoarcere”. Reconstrucția astfel obținută are complexitatea $O(n)$. Exemplu: subproblema care se termina în elementul 19 are subșirul de lungime maximă 3 și a fost calculată folosind subproblema care se termină cu elementul 15 (oricare din ele). Subșirul de lungime maximă care se termină în 15 a fost calculat folosindu-ne de elementul 12. Numărul 12 marchează sfârșitul reconstrucției, fiind cel mai mic element din subșir.

O altă problemă cu o aplicare clasică a Programării Dinamice este și *determinarea celui mai lung subșir comun a două șiruri de caractere*. Descrierea problemei, indicații de rezolvare și o modalitate de evaluare a soluțiilor poate fi găsită pe infoarena [5]. Similar, o problemă care admite o

varietate mare de soluții este cea a subsecvenței de sumă maximă; enunțul acesteia poate fi găsit la [6].

Al n -lea termen din șirul lui Fibonacci

Șirul lui Fibonacci se definește ca fiind secvența de numere generată de recurența:

$$F_N = F_{N-1} + F_{N-2}, \text{ cu } F_0 = 0 \text{ și } F_1 = 1$$

După cum se observă, pentru a calcula următorul element din șir ne sunt necesare doar precedentele două numere. De asemenea, după ce am calculat noul termen, trebuie să le reținem tot pe ultimele două numere pentru a putea reaplica aceeași metodă în continuare.

Implementarea calculului celui de-al n -lea termen Fibonacci folosind direct relația de recurență anterioară necesită o complexitate $O(n)$. În continuare, vrem să îmbunătățim această implementare, reducând complexitatea, prin introducerea unei metode algebrice/matriciale utile și pentru rezolvarea mai eficientă a altor probleme (a se vedea exercițiile propuse).

Practic, cunoscând termenii F_{N-1} și F_{N-2} putem să obținem următoarea ecuație în forma matricială:

$$\begin{bmatrix} F_{N-1} \\ F_N \end{bmatrix} = A \times \begin{bmatrix} F_{N-2} \\ F_{N-1} \end{bmatrix}, \text{ unde } A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

Folosind metoda algebrică/iterativă pentru rezolvarea acestei recurențe în formă matricială, se observă ușor relația de mai jos, care poate fi demonstrată prin inducție:

$$\begin{bmatrix} F_{N-1} \\ F_N \end{bmatrix} = A^{N-1} \times \begin{bmatrix} F_0 \\ F_1 \end{bmatrix}$$

În concluzie, pentru a determina cel de-al N -lea termen Fibonacci:

- calculăm A^{N-1} în timp logaritm, folosind ridicarea la putere prin divide-et-impera.
- înmulțim matricea anterioară cu vectorul alcătuit din primele 2 valori ale șirului lui Fibonacci.

Complexitatea algoritmului este $O(\log n)$.

Observații:

- complexitatea calculelor matriciale poate afecta (prin constanta asociată) în mod semnificativ timpul de execuție; totuși, în această

situație, nu se pune problema întrucât matricea A are dimensiunea 2×2 .

- deoarece valorile șirului cresc repede, calculele se pot efectua într-o clasă de resturi sau pe numere mari, altfel se va depăși precizia numerelor întregi.

Implementarea acestei probleme poate fi testată pe infoarena [7].

III.3 Concluzii și observații

Greedy și Programarea Dinamică sunt tehnici flexibile și ușor de înțeles la nivel conceptual. Totodată, cu ajutorul lor se pot rezolva probleme foarte complexe într-un mod eficient. În viitor, este posibil să întâlniți algoritmi de Programare Dinamică pe arbori sau Greedy pe stări [8], unde fiecare stare poate fi inclusiv o matrice. Conceptele discutate aici rămân neschimbate. Pe de altă parte, deși Programarea Dinamică pare ușor de înțeles, pentru utilizarea ei în practică, pentru probleme noi, este necesar să aveți experiență în determinarea subproblemelor și a modalității de combinare recursivă a soluțiilor acestora. Această experiență se poate acumula doar cu rezolvarea unui număr cât mai mare și mai variat de probleme.

Aspectul cel mai important de reținut este că soluțiile găsite trebuie să reprezinte optimul global și nu doar local. Astfel, se pot confunda ușor problemele care se rezolvă cu Greedy cu cele care se rezolvă prin Programare Dinamică.

În final, deși în cadrul ultimelor două capitole (Greedy și Programare Dinamică) au fost discutate în special probleme de optimizare, tehnicile sunt utile și în rezolvarea altor tipuri de probleme (de exemplu, probleme de combinatorică sau de decizie, vezi și problema celui de-al n -lea termen din șirul lui Fibonacci).

III.4 Probleme propuse

1. Problema rucsacului (varianta discretă)

Un camion poate transporta T tone de mobilă. Există n piese de mobilă, fiecare caracterizată de greutatea $G[i]$ și de valoarea $V[i]$ adusă de transportarea sa. Să se decidă ce piese de mobilă vor fi transportate pentru a

aduce o valoare maximă (soluție optimă), știind ca nu avem voie să luăm bucăți dintr-o piesă de mobilă.

2. Aranjarea cărților în bibliotecă

Se dau n dulapuri aflate într-o bibliotecă, precum și numărul cărților din fiecare dulap. Se dorește mutarea dulapurilor (cu toate cărțile aferente) într-o nouă bibliotecă astfel încât primul dulap este mutat întotdeauna, iar celelalte dulapuri se mută doar dacă nu sunt așezate unul lângă celălalt în biblioteca inițială. De asemenea, se dorește aflarea soluției care maximizează numărul cărților din noua bibliotecă.

3. Parantezarea unei expresii booleene

Se dă o expresie booleană exprimată prin stringurile “true”, “false”, “and”, “or”, “xor”. Numărați modurile în care se pot așeza parantezele astfel încât rezultatul expresiei parantezate să fie “true”.

De exemplu, pentru expresia:

```
true and false xor true
```

există două modalități de parantezare astfel încât rezultatul să fie “true”:

```
((true and false) xor true)
(true and (false xor true))
```

4. Înmulțirea optimă a unui lanț de matrici

Întrucât înmulțirea matricilor este asociativă, determinați modul optim de realizare al înmulțirilor unui șir $A_1 \dots A_n$ de n matrici astfel încât numărul total de înmulțiri scalare să fie minimă.

Exemplu: Fie A_1 o matrice 10×30 ; $A_2 - 30 \times 5$; $A_3 - 5 \times 60$:

$(A_1 * A_2) * A_3 = (10 \times 30 \times 5) + (10 \times 5 \times 60) = 1500 + 3000 = 4500$ operații de înmulțire scalari

$A_1 * (A_2 * A_3) = (30 \times 5 \times 60) + (10 \times 30 \times 60) = 9000 + 18000 = 27000$ operații de înmulțire scalari

III.5 Indicații de rezolvare

1. Problema rucsacului (varianta discretă)

Se folosește o matrice care se completează pe linii, iar fiecare linie modelează o lume în care camionul poate transporta 1 tonă, 2 tone, ... n tone.

Putem modela o subproblemă ca fiind $best[i][j]$ = valoarea maximă pe care o poate transporta un camion de tonaj maxim i folosind doar primele j piese de mobilă. Relația de recurență este:

$$best[i][j] = \max\{best[i][j-1], best[i-G[j]] + V[j]\}, \text{ dacă } i - G[j] \geq 0$$

$$best[i][j] = best[i][j-1], \text{ dacă } i - G[j] < 0$$

$$best[0][j] = 0; best[i][0] = 0$$

Soluția problemei va fi oferită de către $best[T][n]$. O observație interesantă este că această soluție poate fi aplicată doar dacă greutatea obiectelor sunt numere naturale, întrucât acestea intră în definiția relației de recurență (vezi $i - G[j]$). În cazul general, problema rucsacului este NP-completă și nu acceptă o soluție polinomială.

Complexitate: $O(n * T)$

2. Aranjarea cărților în bibliotecă

Fie $c_max[i]$ = numărul maxim de cărți ce pot fi transferate utilizând cărțile existente pe dulapurile de la 1 la $i-2$, astfel încât să fie selectat și dulapul i . Formula de recurență obținută este $c_max(i) = carti(i) + \max\{c_max(k) \mid k = 1, i-2\}$, ceea ce duce la o complexitate $O(i)$ pentru pas și $O(n^2)$ în total.

Totuși, relația de recurență de mai sus se poate optimiza observând că, la fiecare pas i , trebuie ales maximul dintre $c_max(i-2)$ și $c_max(i-3)$. Răspunsul este dat de maximul din vectorul $c_max[0...n]$.

Complexitate: $O(n)$

3. Parantezarea unei expresii booleene

Se folosește o matrice pentru a stoca soluții parțiale pentru subprobleme, cu semnificația $A[i][j]$ este numărul de parantezări asupra sub-expresiei formată din termenii pornind de la i și terminand cu j , astfel încât rezultatul sub-expresiei să fie *true*. Pozițiile se pot referi doar la elementele operanzi (de pe

pozițiile pare: 0, 2, 4 etc.), nu și la operatori. Pentru simplitate, se poate utiliza o matrice separată care să stocheze numărul de parantezări ale sub-expresiilor în așa fel încât rezultatul să fie *false*. Cele două matrice vor fi simetrice; prin urmare, acestea pot fi completate pe subdiagonale, doar deasupra diagonalei principale, caz în care rezultatul final se va regăsi în colțul din dreapta-sus al matricei *true*.

Complexitate: $O(n^2)$

4. Înmulțirea optimă a unui lanț de matrici

Se folosește o matrice care se completează iterativ, peste diagonala principală, în care fiecare celulă modelează înmulțirea optimă a matricilor între indecșii i și j , precum și o structură auxiliară în care se memorează dimensiunea matricii rezultate din înmulțirea tuturor matricilor $A_i \dots A_j$. Ulterior, putem modela o subproblemă ca fiind $best[i][j]$ = numărul minim de operații necesare pentru a înmulți matricile $A_i \dots A_j$. Relația de recurență este:

$$best[i][j] = \min\{best[i][k] + best[k+1][j] + \text{costul înmulțirii matricii } A_{i\dots k} \text{ cu } A_{k+1\dots j}\}$$

$$best[i][i] = 0;$$

Se poate observa ușor că aplicând relația de recurență pentru subproblema de dimensiune 1 ($best[i][i+1]$) obținem:

$$best[i][i+1] = \text{costul înmulțirii matricii } A_i \text{ cu } A_{i+1} = nr_linii(A_i) * nr_coloane(A_i) * nr_coloane(A_{i+1})$$

Soluția problemei inițiale va fi oferită de către $best[0][n]$.

Complexitate: $O(n^3)$

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). Introduction to Algorithms (3rd ed.). Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) Introducere în Analiza Algoritmilor. București: Polirom
- [3] Programea dinamică, materiale suplimentare disponibile la adresa: <http://ww3.algorithmdesign.net/handouts/DynamicProgramming.pdf>

[4] Problema Subșirului crescător maximal, disponibilă la

<http://infoarena.ro/problema/scmax>

[5] Problema Celui mai lung subșir comun, disponibilă la

<http://infoarena.ro/problema/cmlsc>

[6] Problema Subsecvenței de sumă maximă, disponibilă la

<http://infoarena.ro/problema/ssm>

[7] Problema celui de-al k -lea termen Fibonacci, disponibilă la

<http://infoarena.ro/problema/ssm>

[8] When Greedy Algorithms are Perfect: the Matroid, disponibilă la

<https://jeremykun.com/2014/08/26/when-greedy-algorithms-are-perfect-the-matroid/>

IV Backtracking și Optimizări

IV.1 Importanță și aplicații practice

Backtracking-ul (BKT) este o tehnică de programare în general ineficientă, care este folosită când alte scheme de algoritmi mai eficiente nu pot fi aplicate. Rezolvarea prin backtracking a problemelor [1-3] este o tehnică de tipul „încearcă și verifică eroarea” („trial and error”) sau, mai tehnic vorbind, o căutare neinformată („oarbă”) în spațiul stărilor unei probleme (configurațiile posibile ale problemei sunt stări iar arcele sunt tranzițiile între configurații). Din nefericire, pentru multe probleme din realitate (de exemplu, cele NP-complete) și din domeniul inteligenței artificiale nu s-au descoperit algoritmi mai eficienți decât BKT care să rezolve întotdeauna perfect (corect, optimal) problema. Pentru a reduce ineficiența acestei clase de algoritmi au fost făcute multe cercetări, soluțiile încercând să profite de particularitățile și constrângerile (restricțiile) între variabilele problemei de rezolvat [7-9] sau să aplice diverse euristici pentru îmbunătățirea BKT orb.

IV.2 Descrierea problemei și a rezolvărilor

Exact ca în cazul strategiilor de parcurgere în lățime/adâncime (descrise în capitolul VI), backtracking-ul are la bază expandarea tentativă a unei căi în spațiul stărilor (cu avansuri și reveniri, implicit într-un arbore top-down), construcția soluției făcându-se într-o manieră incrementală. Abstract vorbind, la fiecare nod al căii explorate se face asignarea unei valori la o variabilă și se testează viabilitatea soluției parțiale obținute. Dacă acesta nu este validă, se revine la ultima alegere corectă și se selectează o altă valoare.

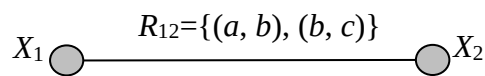
Prin natura sa, BKT-ul poate fi implemenat comod recursiv, în arborele expandat top-down aplicându-se operații de tipul *pruning* (tăierea unei ramuri) dacă soluția parțială la care s-a ajuns nu este validă.

Notațiile utilizate în continuare în descrierea schemei backtracking sunt următoarele:

- $X_1, \dots, X_N$ sunt cele N variabile ale problemei;
- $D_1, \dots, D_N$ domeniile aferente fiecărei variabile;
- U - întreg care reprezintă indicele variabilei curent selectate pentru a i se atribui o valoare;

- F - vector indexat după indicii variabilelor, în care sunt memorate selecțiile de valori făcute de la prima variabilă și până la variabila curentă.

Reprezentarea grafică a unei relații (restricții, constrângeri) binare $R_{12}=\{(a, b), (b, c)\}$ între două variabile X_1 și X_2 cu același domeniu $\{a, b, c\}$ se poate face după cum urmează:



Astfel, raportat la domeniul inițial al variabilelor, după aplicarea constrângerii, valorile admisibile sunt doar $X_1 = a$ și $X_2 = b$, respectiv $X_1 = b$ și $X_2 = c$. Dacă se reprezintă grafic toate restricțiile se obține un graf de restricții. Trebuie precizat că pot exista și restricții n -are [8, 9] între mai mult de două variabile ale problemei.

O prezentare generică a celui mai simplu algoritm de backtracking, cel cronologic (sau „orb”) recursiv poate fi următoarea:

BKT (U, F)

```
// vectorul F conține asignările variabilelor problemei
// U este id-ul variabilei de la pasul curent din BKT
foreach (V in  $X_U$ )
    // pentru fiecare valoare din domeniul variabilei  $X_U$ 
    F[U] = V
    // verificăm corectitudinea asignării făcute
    if (Verifica(U, F))
        if (U < N)
            // dacă nu am ajuns la ultima variabilă
            // mergem mai departe
            BKT(U+1, F)
        else
            // am găsit o soluție a problemei
            // toate asignările sunt corecte
            Afișează valorile din vectorul F
            // dacă vrem toate soluțiile problemei
            // renunțăm la break-ul următor
            Break
// când se iese din funcție, înseamnă că am încercat toate
// valorile pentru variabila curentă și trebuie să revenim
```

Funcția **Verifica** testează dacă soluția parțială memorată în vectorul soluție F este validă și, în general, dacă s-a ajuns la o soluție finală. BKT poate fi folosit pentru a găsi toate soluțiile unei probleme (caz în care, la găsirea unei

soluții finale aceasta este afișată și se continuă căutarea) sau prima soluție întâlnită (caz în care se tipărește și oprește căutarea, ieșind forțat din funcția **Verifica**).

```
Verifica (U,F)
    test = true
    if (Soluție?(F))
        Afișează valorile din vectorul F
        if (se dorește doar o soluție)
            stop // se iese din toată recursivitatea
        else
            test=Valid?(F)
            if test == false
                break
    return test
```

În cazul în care problema poate fi descrisă prin restricții (ca mai sus) și soluția constă doar în satisfacerea tuturor restricțiilor, o variantă de a implementa această funcție este:

```
Verifică (U,F)
    test = true
    I = U - 1
    while (I > 0)
        test = Relație(I, F[I], U, F[U])
        I = I - 1
        if (test == false)
            break
    return test
```

Trebuie precizat că, în general, verificarea poate include și alte teste decât doar respectarea restricțiilor.

Complexitatea algoritmului: complexitatea temporală este de $O(b^d)$, iar cea spațială $O(d)$, unde b se numește *factor de ramificare* (numărul mediu de stări ulterioare în care nodul curent poate fi expandat, depinde de numărul de valori din domeniul nodului curent) și d este *adâncimea soluției* (în general, egal cu numărul de variabile).

Pornind de la versiunea inițială a algoritmului BKT cronologic, putem aduce o **serie de îmbunătățiri** în următoarele direcții:

- **Utilizarea euristicilor** în vederea optimizării numărului de teste prin luarea în considerare a următoarelor idei:

1. *Ordonarea variabilelor*
 2. *Ordonarea valorilor.*
- **Algoritmi hibrizi** care îmbunătățesc performanțele rezolvării prin reducerea numărului de teste; aici putem identifica următoarele subcategorii:
 1. *Tehnici prospective:*
 1. Căutare cu predicție completă
 2. Căutare cu predicție parțială
 3. Căutare cu verificare predictivă
 2. *Tehnici retrospective:*
 1. Backtracking cu salt
 2. Backtracking cu marcare
 - **Algoritmi de satisfacerea restricțiilor (CSP - Constraint Satisfaction Problem)** prin verificarea consistenței nodurilor, arcelor sau a căilor în graful de restricții.

Dintre metodele enumerate mai sus ne vom concentra asupra tehnicilor prospective, existând în ambele cazuri o îmbunătățire considerabilă la nivelul apelurilor recursive și al intrărilor în stivă pentru majoritatea problemelor, precum și asupra CSP cu îmbunătățirea aferentă a consistenței reprezentării.

Pe de altă parte, trebuie avut în vedere că aceste îmbunătățiri se obțin uneori cu o complexitate mai mare la fiecare pas din recursivitate. Drept urmare, sunt situații când aceste tehnici, deși reduc numărul de apeluri recursive și de soluții parțiale, pot duce la timpi de rezolvare mai mari decât BKT cronologic. În general, nu există o rețetă de succes și pentru fiecare problemă trebuie analizat în particular utilitatea metodelor prospective, retrospective sau de tip CSP.

IV.2.1 EURISTICI PENTRU BACKTRACKING

Ordonarea variabilelor urmărește reordonarea variabilelor legate prin restricții explicite (specificate de mulțimea de restricții definită în problemă) astfel încât numărul de operații ulterioare să fie minim. Astfel sunt *preferate mai întâi variabilele care apar într-un număr mare de restricții (most constrained variable first) sau/și au domenii de valori cu cardinalitate mică (least remaining values first)*.

Ordonarea valorilor pleacă de la premisa că nu toate valorile din domeniul variabilelor apar în toate restricțiile. Și în acest caz sunt preferate mai întâi valorile cele mai restricționate, cu cele mai puține atribuiri posibile.

IV.2.2 TEHNICI PROSPECTIVE

Principiul tehnicilor prospective este simplu: fiecare pas explorat spre soluție nu trebuie să ducă la blocare. Astfel, la fiecare atribuire a variabilei curente cu o valoare corespunzătoare din domeniu, toate variabilele ulterioare sunt verificate pentru a depista eventuale condiții de blocare. Anumite valori ale variabilelor neinstantiate pot fi eliminate deoarece nu vor putea niciodată să facă parte din soluția finală.

Următorii algoritmi prospectivi analizați implementează strategia de căutare neinformată cu realizarea unor grade diferite de k -consistență.

Backtracking cu predicție completă

Predicție(U, F, D)

```
foreach (L of D[U])
    F[U] = L;
    if (U < N)
        DNEW ← Verifică_Inainte(U, D[U], D);
        if (DNEW != null)
            DNEW = Verifica_Viitoare(U, DNEW);
        if (DNEW != null)
            Predictie(U+1, F, DNEW);
```

Verifica_Inainte(U, L, D)

```
inițializează DNEW
for (U2 = U+1..N)
    foreach (L2 ∈ D[U2])
        if (Relatie(U, L, U2, L2))
            introduce L2 în DNEW[U2];
    if (DNEW[U2] == vidă)
        return null;
return DNEW;
```

Verifica_Viitoare(U, DNEW)

```
for (U1 = U+1..N)
    foreach (L1 din DNEW[U1])
        for (U2 = U+1..N)
            gasit = false;
            foreach (L2 din DNEW[U2])
                if (Relatie(U1, L1, U2, L2))
                    gasit = true;
```

```

                                break; // foreach L2
// nu s-a găsit o valoare consistentă
// pentru U2
if (!gasit)
    elimina L1 din DNEW[U1]
    break; // for U2
if DNEW[U1] = vidă
    return null
return DNEW

```

Backtracking-ul cu predicție parțială [3] presupune modificarea doar a funcției Verifică_Viitoare din prisma domeniului de vizibilitate a variabilei U_2 care acum variază exclusiv de la U_1+1 , nu direct de la $U+1$. Rezultatul imediat este înjumătățirea numărului de operații efectuate la nivelul funcției.

Verifica_Viitoare(U, DNEW)

```

...
for (U1 = U+1..N)
    foreach (L1 din DNEW[U1])
        for (U2 = U1+1..N)
            foreach (L2 din DNEW[U2])
                ...

```

Backtracking-ul cu verificare predictivă [3] elimină apelul Verifica_Viitoare(U, DNEW) complet din funcția de Predicție

Predicție(U, F, D)

```

...
DNEW = Verifică_Inainte(U, D[U], D)
if DNEW != null
    then DNEW ← Verifica_Viitoare(U, DNEW)
if DNEW != null
    ...

```

Discuția care se ridică imediat este *care dintre cele 3 metode este mai eficientă?* Părerile sunt împărțite în sensul că uneori costul rafinărilor ulterioare poate fi mai mare decât costul expandării efective a nodului curent, dar totodată se poate obține o reducere semnificativă a numărului de apeluri recursive prin eliminarea unor soluții neviabile. Certitudinea este că oricare dintre aceste metode poate reduce considerabil numărul de intrări în stivă, dar trebuie luat în considerare specificul problemei și costul operației de Verifica_Viitoare.

Un aspect important este că toate cele trei variante de tehnici prospective pot fi îmbunătățite prin introducerea de euristici, lucru echivalent cu o reordonare dinamică a variabilelor la fiecare avans în căutare. Experimental, s-a dovedit

că introducerea acestor euristici (de exemplu, selecția următoarei variabile urmărind ca aceasta să aibă cele mai puține valori rămase în domeniul propriu) furnizează rezultate foarte bune.

IV.2.3 PROBLEMA SATISFACERII RESTRICȚIILOR

Problema satisfacerii restricțiilor (CSP – Constraint Satisfaction Problem) [4-6, 7-9], în formularea cea mai generală, presupune existența unei mulțimi de variabile, a unor domenii de valori potențiale pentru fiecare variabilă și a unei mulțimi de restricții/constrângeri care specifică combinațiile de valori acceptabile ale variabilelor (exact conceptul de relații definite anterior, cu tot cu restricțiile aferente). **Scopul final** îl reprezintă determinarea unei atribuirii de valori pentru fiecare variabilă astfel încât toate restricțiile să fie satisfăcute, dacă acest lucru este posibil.

Problema satisfacerii restricțiilor este, în cazul general, o problemă grea, **NP-completă**, cu rezolvări de complexitate exponențială în raport cu numărul de variabile ale problemei. Din perspectiva strategiilor de căutare într-un spațiu de stări, traducerea problemei ar fi următoarea: pornind din starea inițială a procesului care conține restricțiile identificate în descrierea inițială a problemei, se dorește atingerea unei stări finale care a fost restricționată "suficient" pentru a rezolva problema.

Pornind de la premisa că CSP este o problemă de căutare din clasa problemelor NP-complete, aspectul de interes al optimizării curente devine reducerea cât mai puternică a timpului și a spațiului de căutare. Totodată, fiind o problemă de căutare, rezolvarea problemei satisfacerii restricțiilor poate fi făcută aplicând una din tehnicile de căutare a soluției în spațiul stărilor. Astfel, cea mai utilizată strategie de rezolvare a problemei CSP este backtracking-ul, variantă a căutării neinformate în adâncime.

În acest capitol sunt de interes problemele de tipul CSP binare, care implică constrângeri doar între două variabile ale problemei, care pot fi reprezentate printr-un graf de restricții.

Un **arc** (X_i, X_j) într-un graf de restricții orientat se numește **consistent** dacă și numai dacă pentru orice valoare $x \in D_i$ din domeniul variabilei X_i , există o valoare $y \in D_j$ din domeniul variabilei X_j , astfel încât $R_{i,j}(x,y)$ (sau $(x,y) \in R_{i,j}$, deci asignarea respectă restricția). **Graful de restricții orientat** rezultat se numește **arc-consistent** dacă toate arcele sunt consistente.

O cale de lungime m prin nodurile $i_0, \dots, i_m$ ale unui graf de restricții orientat se numește **m-consistentă** dacă și numai dacă pentru orice valoare $x \in D_{i_0}$,

din domeniul variabilei i_0 , și o valoare $y \in D_{i_m}$, din domeniul variabilei i_m , pentru care $R_{i_0, i_m}(x, y)$, există o secvență de valori $z_1 \in D_{i_1} \dots z_{m-1} \in D_{i_{m-1}}$ astfel încât $R_{i_0, i_1}(x, z_1), \dots, R_{i_{m-1}, i_m}(z_{m-1}, y)$. **Graful de restricții orientat** rezultat se numește ***m-arc-consistent***.

Arc-consistența unui graf de restricții se verifică folosind algoritmi AC-1, AC-2, AC-3, AC-4 etc. [9]. Mai jos sunt prezentați AC-1 și AC-3:

```

Verifică ( $X_k, X_m$ )
    delete = false
    foreach ( $x \in D_k$ )
        if (nu există nicio valoare  $y \in D_m$  astfel încât
             $R_{k,m}(x, y)$ )
            elimină  $x$  din  $D_k$ 
            delete = true
    return delete

```

AC-1:

```

Crează  $Q \leftarrow \{(X_i, X_j) \mid (X_i, X_j) \in \text{Mulțime arce/restricții}, i \neq j\}$ 
repeat
    modificat = false
    foreach ( $(X_i, X_j) \in Q$ )
        modificat = modificat or Verifică( $X_k, X_m$ )
until (modificat == false)

```

AC-3:

```

Crează  $Q \leftarrow \{(X_i, X_j) \mid (X_i, X_j) \in \text{Multime arce/restricții}, i \neq j\}$ 
while  $Q$  nu este vida
    Elimină din  $Q$  primul element ( $X_k, X_m$ )
    if (Verifică( $X_k, X_m$ ))
         $Q \leftarrow Q \cup \{(X_i, X_k) \mid (X_i, X_k) \in \text{Multime arce}, i \neq k, m\}$ 

```

Pornind de la următoarele notații:

- N - numărul de variabile;
- a - cardinalitatea maximă a domeniilor de valori ale tuturor variabilelor;
- e - numărul de restricții;

complexitățile algoritmilor precedenți sunt următoarele:

- Algoritmul de realizare a arc-consistenței AC-1 are în cazul cel mai defavorabil complexitatea $O(a^2 * N * e)$
- Algoritmul de realizare a arc-consistenței AC-3: complexitatea temporală este $O(e * a^3)$; complexitate spațiu: $O(e + N * a)$

IV.3 Concluzii și observații

Metodele descrise pot fi aplicate pe o plajă largă de probleme, iar optimizările prezentate pot duce la scăderi drastice la nivelul timpilor de execuție. Combinarea anumitor metode, precum tehnici prospective cu euristici duce la rezultate și mai bune, demonstrate în practică. Astfel, majoritatea problemelor care presupun parcurgeri în spațiul stărilor pot fi abordate pornind de la unul dintre algoritmi descriși. Trebuie avut în vedere, însă, că soluțiile de tip backtracking au în general o complexitate exponențială, chiar dacă se folosesc tehnici prospective și euristici, drept urmare ele trebuie folosite doar pentru probleme foarte dificile (de exemplu, probleme NP-complete).

Algoritmi de programare bazată pe restricții au o foarte largă aplicabilitate, de la rezolvarea unor probleme de satisfacere a restricțiilor binare, cum a fost descris în acest capitol, la satisfacerea de restricții n -are, cu valori discrete sau continue, la probleme de asignare de valori sau chiar la sisteme experte [9]. Când o rețea de restricții nu poate fi satisfăcută, se poate recurge la o relaxare a acestora [9]. Sisteme de prelucrare a restricțiilor au fost integrate în medii [8] sau limbaje de programare, cum ar fi Prolog [10].

IV.4 Probleme propuse

Sudoku

Jocul clasic rezolvat prin BKT la nivelul căruia aplicăm diverse optimizări este Sudoku. Astfel, avem o matrice 9×9 subdivizată în 9 sub-matrice identice de dimensiuni 3×3 , denumite regiuni. Regula jocului este simplă: fiecare rând, coloană sau regiune nu trebuie să conțină decât o dată cifrele de la 1 la 9. Formulată altfel, fiecare ansamblu trebuie să conțină cifrele de la 1 la 9 o singură dată.

Se cere să se modeleze această problemă folosind următoarele abordări:

- Backtracking simplu;
- BKT și o metodă prospectivă (preferabil predicție parțială sau completă);
- BKT și o euristică la alegere.

Pentru fiecare dintre soluțiile implementate se va analiza impactul la nivelul timpului total de execuție (care pot fi influențați și de alte procese din sistemul de operare, respectiv alocări și procese interne ale JVM, etc.), precum și al numărului de intrări în recursivitate.

IV.5 Indicații de rezolvare

Varianta simplă de backtracking presupune introducerea tuturor valorilor posibile $\{1...9\}$, pe rând, în fiecare celulă liberă și verificarea consistenței atribuirii la nivel de linie, coloană și regiune. În cazul încălcării oricărei reguli, se încearcă o nouă valoare sau se revine în recursivitate, dacă nu mai există nicio variantă posibilă pentru variabila curentă.

Prima optimizare presupune introducerea unei metode prospective care va filtra, la fiecare iterație, numărul de valori potențiale care pot fi introduse în anumite celule. Astfel, vor fi identificate și eliminate în prealabil potențialele inconsistențe la nivel de linie, coloană sau regiune (valori care nu pot fi utilizate în iterațiile ulterioare), iar dacă domeniul de valori al unei celule libere devine vid, este necesară revenirea din recursivitate.

Suplimentar, se poate introduce și euristica de a reordona variabilele pornind de la cele mai restricționate (cu cele mai puține valori potențiale a fi introduse într-o anumită celulă), reducând astfel și mai mult numărul de intrări inutile în recursivitate.

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.) (2009). Introduction to Algorithms (3rd ed.). Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) Introducere în Analiza Algoritmilor. Bucuresti: Polirom
- [3] Cursul de Inteligență Artificială, Prof. Ing. Adina Magda Florea, Facultatea de Automatică și Calculatoare, Universitatea Politehnica din București
- [4] Donald E. Knuth. (2011) The Art of Computer Programming, Boston, MA: Addison-Wesley
- [5] Tutorial CSP, disponibil la <http://4c.ucc.ie/web/outreach/tutorial.html>

- [6] The Complexity of Some Polynomial Network Consistency Algorithms for Constraint Satisfaction Problems, disponibil la <http://cse.unl.edu/~choueiry/Documents/AC-MackworthFreuder.pdf>
- [7] Ștefan Trăușan-Matu, M. Bărbuceanu (1990) Prelucrări bazate pe rețele de restricții în inteligența artificială, în Revista Româna de Informatică, vol. 11, nr.1, pag. 9-26.
- [8] Ștefan Trausan-Matu, M. Barbuceanu, Gh. Ghiculete (1992) The Integration of Powerful and Flexible Constraint Representation and Processing into an Object-Oriented Programming Environment, in J.C.Rault (ed), Actes de "Representations Par Objets", La Grande Motte, Franta, iunie 1992, pag.201-210. Disponibil la https://www.researchgate.net/publication/239537714_The_Integration_of_Powerful_and_Flexible_Constraint_Representation_and_Processing_into_an_Object-Oriented_Programming_Environment
- [9] Ștefan Trăușan-Matu (1995) Reprezentarea și prelucrarea restricțiilor, Disponibil la https://www.researchgate.net/publication/306960461_Reprezentarea_si_prelucrarea_restricțiilor
- [10] CLP(B): Constraint Logic Programming over Boolean Variables. Disponibil la <http://www.swi-prolog.org/man/clpb.html>

V Algoritmul Minimax pentru jocuri

V.1 Importanță și aplicații practice

Algoritmul Minimax [1] și variantele sale îmbunătățite (de exemplu, Negamax și tăierea Alpha-Beta) [2] sunt folosite în diverse domenii precum teoria jocurilor (*Game Theory*) [3], teoria jocurilor combinatorice (*Combinatorial Game Theory – CGT*) [4], teoria deciziei (*Decision Theory*) și statistică. Astfel, diferite variante ale algoritmului sunt necesare în proiectarea și implementarea de aplicații legate de inteligență artificială, economie, dar și în domenii precum științe politice sau biologie.

V.2 Descrierea problemei și a rezolvărilor

Algoritmii Minimax permit abordarea unor probleme ce țin de teoria jocurilor combinatorice. CGT este o ramură a matematicii ce se ocupă cu studierea jocurilor în doi (*two-player games*) [4], în care participanții își modifică rând pe rând pozițiile în diferite moduri, prestabilite de regulile jocului, pentru a îndeplini una sau mai multe condiții de câștig. Exemple de astfel de jocuri sunt: șah, go, dame (*checkers*), X și O (*tic-tac-toe*), etc. CGT nu studiază jocuri ce presupun implicarea unui element aleator (șansa) în derularea jocului, precum poker, blackjack, zaruri etc. Astfel decizia abordării unor probleme rezolvabile prin metode de tip Minimax se datorează în principal simplității atât conceptuale, cât și raportat la implementarea propriu-zisă. Algoritmul poate fi însă adaptat și pentru jocuri în care intră în calcul și elemente de șansă.

V.2.1 ALGORITMUL MINIMAX

Strategia pe care se bazează ideea algoritmului Minimax este că jucătorii implicați adoptă următoarele strategii [1]:

- Jucătorul 1 (max; în general jucătorul aflat la mutare) va încerca mereu să-și maximizeze propriul câștig prin mutarea pe care o are de făcut;
- Jucătorul 2 (min) va încerca mereu să minimizeze câștigul jucătorului 1 la fiecare mutare.

Abordarea se axează pe jocurile cu câștig de sumă zero (*zero-sum*) [5], acest lucru garantând, printre altele, că orice câștig al Jucătorului 1 este egal cu modulul sumei pierdute de Jucătorul 2. Cu alte cuvinte, cât pierde Jucătorul 2, atât câștigă Jucătorul 1, și invers. Putem scrie acest lucru astfel:

$$\begin{aligned}\text{Câștig_Jucător_1} &= |\text{Pierdere_Jucător_2}| \\ |\text{Pierdere_Jucător_1}| &= \text{Câștig_Jucător_2}\end{aligned}$$

Un alt mod de a descrie jocurile de sumă zero este că scorul tuturor jucătorilor este zero (dacă unul câștigă, celălalt pierde):

$$\begin{aligned}\text{Câștig_Jucător_1} + \text{Pierdere_Jucător_2} &= \\ \text{Loss_Player_1} + \text{Pierdere_Jucător_2} &= 0\end{aligned}$$

Reprezentarea spațiului soluțiilor

În general spațiul soluțiilor pentru un joc în doi de tip *zero-sum* se reprezintă ca un arbore, fiecărui nod fiindu-i asociată o stare a jocului în desfășurare (*game state*). Drept exemplu, considerăm jocul 7-nim, în cadrul căreia avem inițial un teanc de 7 monezi care trebuie divizat iterativ de jucători în sub-grupuri inegale ca număr de monezi, din ce în ce mai mici. Dacă se ajunge la sub-mulțimi de 1 sau 2 monezi, acestea nu mai pot divizate, iar jucătorul care nu mai poate efectua mutări pierde. Se constată că există o strategie optimă prin care un jucător câștigă întotdeauna. Dacă primul jucător alege mutarea 4-3 (care duce către starea din dreapta din primul nivel MAX din Figura 5.1), atunci el va câștiga sigur, indiferent de mutarea aleasă ulterior de către adversar.

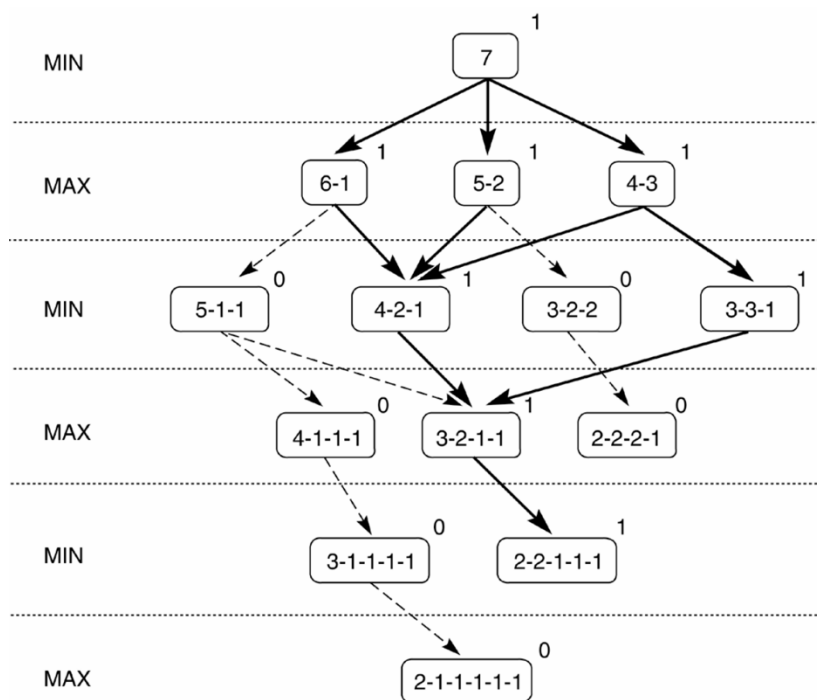


Figura 5.1 – Minimax exhaustiv pentru jocul 7-Nim [6].

Metodele de reprezentare a arborelui de joc variază în funcție de paradigma de programare aleasă, de limbajul de programare, precum și de gradul de optimizare avut în vedere.

Având noțiunile de bază asupra strategiei celor doi jucători, precum și a reprezentării spațiului soluțiilor problemei, putem formula o prima variantă a algoritmului Minimax.

Funcția de *evaluare* este o euristică care asignează câștigul potențial într-un anumit moment (de exemplu, evaluarea stării curente și codificarea +/-1 în funcție de câștigul jucătorului sau al oponentului, estimarea numărului de mutări până la câștig, poziționarea mai bună/strategică a pieselor pe o tablă de șah, numărul de linii potențial câștigătoare la nivelul cărora mai trebuie completate 1-2 valori în cadrul unui joc de tip X și O). Mai mult, funcția de evaluare a stării se face din perspectiva jucătorului *max*, aflat la mutare. Din acest motiv, fiind un joc de sumă zero, adversarul, jucătorul *min*, folosește scorul *-evaluare()* pentru starea curentă. Funcția *evaluare()* poate codifica +1 câștig; -1 pierdere; 0 nedefinit, sau poate reprezenta un avantaj al mutării respective (de exemplu, centralitatea pieselor, mobilitatea și poziționarea strategică a pieselor la șah, suprafața teritoriului deținut la un joc GO).

```
max(adâncime)
    if (adâncime == 0)
        return evaluare();
    int max = -∞;
    for (all moves)
        scor = mini(adâncime-1);
        if (scor > max)
            max = scor;
    return max;

min(adâncime)
    if (adâncime == 0)
        return -evaluare();
    int min = +∞;
    for (all moves)
        score = maxi(adâncime-1);
        if (score < min)
            min = score;
    return min;
```

Argumentarea utilizării unei adâncimi maxime

Datorita spațiului de stări mare, de multe ori copleșitor ca volum, o inspectare completă a acestuia nu este fezabilă și devine impracticabilă din punctul de vedere al timpului consumat sau chiar a memoriei alocate (se vor discuta aceste aspecte în secțiunea de complexitate). Astfel, de cele mai multe ori este preferată o abordare care parcurge arborele numai până la o anumită adâncime maximă (*depth*) d . Această abordare permite examinarea arborelui suficient de în profunzime pentru a putea lua decizii informate (coerente) pentru desfășurarea jocului într-un timp rezonabil. Totuși, dezavantajul major este că, pe termen lung, se poate dovedi ca decizia luată explorând doar până la adâncimea d nu este global favorabilă jucătorului în cauză, în special datorită faptului că funcția de evaluare a unei stări nu poate estima perfect rezultatul final al jocului.

De asemenea, se observă recursivitatea indirectă. Prin convenție acceptăm că începutul algoritmului să fie cu funcția *max*. Astfel, se analizează succesiv diferite stări ale jocului din punctul de vedere al celor doi jucători până la adâncimea d . Rezultatul întors este scorul final al mutării celei mai bune, jucătorul aflat la mutare urmând să execute una dintre mutările aferente acestui scor.

V.2.2 NEGAMAX

Negamax este o variantă a minimax care se bazează pe următoarea observație: fiind într-un joc *zero-sum* în care câștigul unui jucător este egal cu modulul sumei pierdute de celalalt jucător și invers, se remarcă faptul că fiecare jucător încearcă să-și maximizeze propriul câștig la fiecare pas. Într-adevăr, se poate spune că jucătorul *mini* încearcă de fapt să maximizeze în modul suma pierdută de *max*. Astfel, se poate formula următoarea implementare ce profită de observația de mai sus și utilizează formula $\max(a, b) = -\min(-a, -b)$ [7]:

```
negamax(adâncime)
    if (adâncime == 0)
        return evaluare();
    int max =  $-\infty$ ;
    for (all moves)
        scor = -negamax(adâncime-1);
        if( score > max )
            max = scor;
    return max;
```

Printre avantajele acestei formulări fata de algoritmul Minimax standard prezentat anterior se numără:

- Claritatea sporită a codului;
- Eleganța implementării;
- Ușurința în întreținere și extindere a funcționalității.

Din punctul de vedere al complexității temporale sau al numărului de stări explorate la un pas, Negamax nu diferă absolut deloc de Minimax (ambele examinează același număr de stări în arborele de stări). Totuși, din prisma beneficiilor anterioare, este de preferat o implementare ce folosește Negamax față de una bazată pe Minimax în rezolvarea unor probleme ce țin de această tehnică.

V.2.3 TĂIEREA ALPHA-BETA

Algoritmii Minimax și Negamax sunt algoritmi exhaustivi de căutare (*exhaustive search algorithms*) care găsesc soluția optimă examinând întreg spațiul de soluții al problemei. Acest mod de abordare este extrem de ineficient în ceea ce privește efortul de calcul necesar, mai ales considerând contextul în care multe stări de joc inutile sunt explorate (este vorba de acele

stări care nu pot fi niciodată atinse de doi jucători care joacă optim, datorită încălcării principiului de maximizare a câștigului la fiecare rundă).

O îmbunătățire substanțială a Mini/Nega-max îl reprezintă tăierea Alpha-beta. Acest algoritm încearcă să optimizeze Mini/Nega-max profitând de o observație importantă: pe parcursul examinării arborelui de stări se pot elimina subarbori întregi, corespunzători unei mutări M , dacă pe parcursul analizei se dovedește faptul că mutarea M este mai slabă calitativ decât cea mai bună mutare curentă. Astfel, considerăm că se pornește cu o primă mutare $M1$. După analiza în totalitate a acestei mutări și atribuirea unui scor aferent, se continuă cu analiza mutării $M2$. Dacă în analiza ulterioară, se identifică că adversarul are cel puțin o mutare care transformă $M2$ într-o mutare mai slabă decât $M1$, atunci orice alte variante (subarbori) ce corespund mutării $M2$ nu mai trebuie analizate. De exemplu, în Figura 5.2, în momentul identificării nodului 4 din al doilea subarbor de pe nivelul MIN, este clar că valoarea de pe nivelul superior de tip MIN va fi ≤ 4 . Dar cum jucătorul este interesat de maxim, iar 4 este mai mic decât 5 identificat în prealabil din expandarea primului sub-arbore, jucătorul MAX nu va alege această cale și nu are sens expandarea ulterioară a celui de-al doilea subarbor (nodurile care nu sunt evaluate sunt marcate cu „X”).

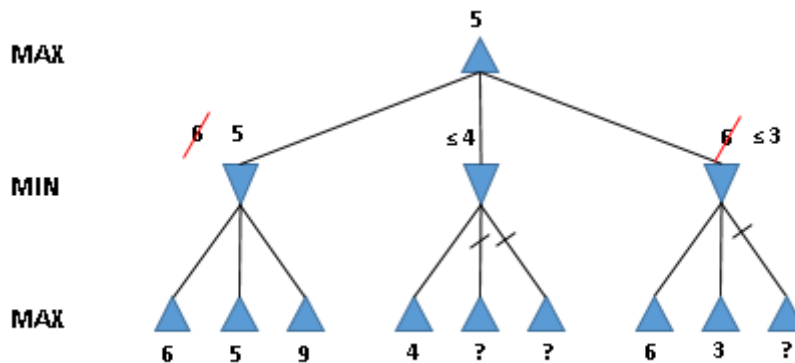


Figura 5.2 – Exemplu de tăieri alpha-beta – căile tăiate în arborele de joc nu sunt explorate indiferent de valorile existente în nodurile descendente

Motivație: În situația generică descrisă anterior există **cel puțin** o variantă în care adversarul obține un câștig mai bun decât dacă jucătorul curent ar fi executat mutarea $M1$. Nu este important cu cât este mai slabă mutarea $M2$ față de $M1$ întrucât, în conformitate cu principiul de maximizare al câștigului folosit de fiecare jucător, adversarul va alege exact acea mutare ce îi va asigura un câștig maximal. Dacă există o variantă și mai bună pentru el este

irelevant, deoarece jucătorul curent este interesat dacă cea mai slabă mutare bună a lui este mai bună decât mutarea noastră curent analizată.

O observație importantă se poate realiza analizând modul de funcționare al acestui algoritm: este extrem de importantă ordonarea mutărilor după valoarea câștigului. În cazul ideal, în care cea mai bună mutare a jucătorului curent este analizată prima, toate celelalte mutări, fiind mai slabe, vor fi eliminate timpuriu din căutare. În cel mai defavorabil caz, în care mutările sunt ordonate crescător după câștigul furnizat, deci cea mai proastă mutare este evaluată prima, iar cea mai bună ultima, Alpha-beta are aceeași complexitate cu Mini/Nega-max, neobținând nicio îmbunătățire. În practică, se constată o îmbunătățire vizibilă în folosirea algoritmului Alpha-beta față de Mini/Nega-max [8].

Rolul mutărilor analizate la început presupune stabilirea unor plafoane de minim și maxim legate de cât de bune/slabe pot fi mutările. Astfel, plafonul de minim, denumit *alpha*, stabilește că o mutare nu poate fi mai slabă decât valoarea acestui plafon. Plafonul de maxim, denumit *beta*, este folosit la verificarea condiției în care o mutare este prea bună pentru a fi luată în considerare. Depășirea plafonului de maxim înseamnă că o mutare este atât de bună încât adversarul nu ar fi permis-o; cu alte cuvinte, există mai sus în arbore o mutare pe care adversarul ar fi putut să o efectueze pentru a nu ajunge în situația curent analizată. Astfel *alpha* și *beta* furnizează o fereastră folosită pentru a filtra mutările posibile pentru cei doi jucători. Evident, aceasta fereastră se va actualiza pe măsură ce se analizează mai multe mutări. De exemplul plafonul minim *alpha* se mărește pe măsură ce sunt identificate mutări mai bune („better worst best moves”). Așadar, în implementare ținem seama și de aceste două plafoane. În conformitate cu principiul Minimax, plafonul de minim al unui jucător (*alpha*-ul) este plafonul de maxim al celuilalt jucător (*beta*-ul lui) și invers.

În continuare prezentăm o implementare conceptuală a algoritmului Alpha-beta, atât pentru Minimax, cât și pentru Negamax [8]:

Varianta Minimax:

```
alphaBetaMax(alpha, beta, adâncime)
    if (adâncime == 0)
        return evaluate();
    for (all moves)
        score = alphaBetaMin(alpha, beta, adâncime-1);
        if( score >= beta )
            return beta; // tăiere beta
        if( score > alpha )
            alpha = score;
```

```

    return alpha;

alphaBetaMin(alpha, beta, adâncime)
    if (adâncime == 0)
        return -evaluare();
    for (all moves)
        score = alphaBetaMax(alpha, beta, adâncime-1);
        if( score <= alpha )
            return alpha; // tăiere alpha
        if( score < beta )
            beta = score;
    return beta;

```

Varianta Negamax:

```

alphaBeta(alpha, beta, adâncime)
    if (adâncime == 0)
        return evaluare();
    for (all moves)
        score = -alphaBeta(-beta, -alpha, adâncime-1);
        if(score >= beta)
            return beta; // tăiere beta
        if(score > alpha)
            alpha = score;
    return alpha;

```

Din nou se remarcă claritatea și coerența sporită a variantei Negamax.

V.2.4 COMPLEXITATE

Pentru a evalua complexitatea algoritmilor prezentați anterior trebuie introduse câteva noțiuni ce țin de terminologia algoritmilor de explorare în spațiul stărilor, care sunt similare cu cele de la algoritmii de tip backtracking, după cum urmează:

- Factor de ramificare (*branching factor*) – notat cu ***b***, reprezintă în medie numărul de descendenți ai unui nod oarecare, neterminal, al arborelui de stări;
- Adâncimea arborelui de joc explorat (depth) – notat cu ***d***, reprezintă adâncimea până la care se face căutarea în arborele de stări. Orice nod de adâncime ***d*** este considerat terminal;
- *Ply* (plural *plies*) – reprezintă un nivel al arborelui.

Folosind termenii de mai sus putem spune ca un arbore cu un factor de ramificare *b*, care va fi examinat până la un nivel *d*, va furniza b^d noduri

terminale ce trebuie procesate. Un algoritm Mini/Nega-max clasic analizează toate stările terminale posibile, și drept urmare va avea complexitatea $O(b^d)$, deci exponențială. În cazul tăierii alpha-beta, după cum s-a menționat anterior, trebuie evaluate cazurile cel mai favorabile, respectiv cel mai defavorabile.

În cazul cel mai *favorabil*, în care mutările sunt ordonate descrescător după câștig (deci ordonate optim, cu prima stare evaluată fiind cea mai favorabilă, de câștig maxim pentru jucătorul aflat la mutare), rezultă o complexitate $O(b * 1 * \dots * b * 1)$ pentru d par sau $O(b * 1 * \dots * b)$ pentru d impar. Restrângând ambele expresii rezultă o complexitate $O(b^{d/2})$. Astfel, deși tot exponențială, complexitatea în acest caz este egală cu radical din complexitatea obținută cu un algoritm mini/nega-max naiv ($\sqrt{b^d} = b^{d/2}$). Explicația este că pentru jucătorul 1, aflat la mutare, trebuie examinate toate mutările posibile pentru a putea găsi mutarea optimă. Însă, pentru fiecare mutare examinată, nu este necesară decât cea mai bună mutare a jucătorului 2 pentru a tăia restul de mutări ale jucătorului 1, în afară de prima (prima fiind și cea mai bună).

Prin urmare, într-un caz ideal, algoritmul Alpha-beta poate explora de 2 ori mai multe niveluri în arborele de stări față de un algoritm mini/nega-max naiv în același timp. Cazul cel mai *defavorabil* a fost deja discutat, în prezentarea Alpha-beta, și este identificat atunci când mutările sunt ordonate crescător după câștigul furnizat unui jucător; astfel, este necesară examinarea tuturor nodurilor pentru găsirea celei mai bune mutare. În consecință, complexitatea devine egală cu cea a unui algoritm Mini/Nega-max naiv.

V.3 Concluzii și observații

Minimax este un algoritm care analizează spațiul soluțiilor unui joc de tip *zero-sum*, dar care poate fi extins ușor și pentru alte tipuri de jocuri. Complexitatea Mini/Nega-max este una exponențială $O(b^d)$, făcând astfel imposibilă examinarea exhaustivă a tuturor stărilor posibile dintr-un joc complex. Din considerente practice, limitele algoritmilor Mini/Nega-max sunt undeva în jurul a 7-8 niveluri în arborele de stări pe mașini standard pentru un joc precum șahul (dar numărul de niveluri trebuie particularizat pentru fiecare problemă în parte). Totuși, această explorare poate aduce un câștig informațional semnificativ în selecția unei mutări bune. Suplimentar față de optimizările prezentate în acest capitol (alpha-beta), există mai multe metode posibile pentru reducerea complexității, precum Negascout sau Transposition Tables [7].

Încheiem acest capitol cu menționarea faptului că există și alți algoritmi pentru rezolvarea jocurilor, una dintre cele mai populare fiind Monte-Carlo Tree Search [9]. Această metodă, cuplată cu diverse alte tehnici moderne de învățare automată, au permis construirea primului program din lume care a câștigat un meci de GO în fața campionului mondial. Jocul GO are un factor de ramificare mediu semnificativ mai mare față de alte jocuri ($b = 250$), precum șahul ($b = 35$) sau „X și O” standard ($b = 9$) [10], acest lucru făcând explorarea în spațiul stărilor foarte costisitoare.

V.4 Probleme propuse

X și O

Aplicația presupune aplicarea unor algoritmi de tip Minimax în scopul rezolvării jocului de X și O (*tic-tac-toe*). Astfel, se dorește implementarea algoritmul Mini/Nega-max astfel încât calculatorul să poată juca împotriva unui jucător uman. Adicional, trebuie extins algoritmul anterior prin utilizarea optimizării de tip tăietură alpha-beta (*alpha-beta pruning*).

Notă: acest joc, desi rezolvabil și prin metode mai puțin costisitoare din punct de vedere al efortului de calcul, însă este ales în scop didactic. Mai mult, varianta sa generalizată este mult mai complicată și exercițiul poate fi extins si pentru aceasta.

V.5 Indicații de rezolvare

Întrucât spațiul stărilor este limitat și numărul de mutări posibile scade pentru fiecare nivel deoarece matricea de stări este completată succesiv, implementarea poate fi utiliza adâncimea maximă (9), atât în cazul mini/nega-max normal, cât și în cazul aplicării tăierii alpha-beta. Funcția de evaluare va lua 3 valori posibile: +1 dacă jocul este câștigat de jucătorul curent, -1 dacă jocul este câștigat de oponent și 0 pentru toate stările în care nu se poate determina un câștigător. Pentru simplitate, se poate conveni din start dacă jucătorul uman este „X” sau „O”, și dacă acesta efectuează prima mutare.

Referințe

- [1] Hazewinkel, Michiel, ed. (2001), "Minimax principle", Encyclopedia of Mathematics, Springer.
- [2] George T. Heineman, Gary Pollice, and Stanley Selkow (2008). Chapter 7: Path Finding in AI. Algorithms in a Nutshell. Oreilly Media. pp. 217–223.
- [3] Michael Maschler, Eilon Solan & Shmuel Zamir (2013). Game Theory. Cambridge University Press.
- [4] Albert, Michael H.; Nowakowski, Richard J.; Wolfe, David (2007). Lessons in Play: An Introduction to Combinatorial Game Theory. A K Peters Ltd.
- [5] Raghavan, T. E. S. (1994). Handbook of Game Theory - volume 2, chapter Zero-sum two-person games, Edited by Aumann and Hart, Elsevier Amsterdam, pp. 735–759.
- [6] Note de curs CS4811 - Artificial Intelligence, Nilufer Onder, Michigan Technological University, disponibil la <http://www.cs.mtu.edu/~nilufer/classes/cs4811/2016-spring/lecture-slides/cs4811-ch05-adversarial-search.pdf>
- [7] Negamax, disponibil la <https://chessprogramming.wikispaces.com/Negamax>
- [8] Alpha-Beta, disponibil la <https://chessprogramming.wikispaces.com/Alpha-Beta>
- [9] Browne, Cameron B., et al. "A survey of monte carlo tree search methods." IEEE Transactions on Computational Intelligence and AI in Games 4.1 (2012): 1-43.
- [10] Branching factor, disponibil la https://en.wikipedia.org/wiki/Branching_factor

VI Parcurgerea Grafurilor. Sortarea Topologică

VI.1 Importanță și aplicații practice

Grafurile sunt utile pentru a modela o gamă variată de probleme din lumea reală, iar algoritmi de parcurgere a grafurilor se regăsesc implementați în multiple aplicații practice precum: rețele de calculatoare (de exemplu, stabilirea unei configurații fără bucle), regăsirea de informații în pagini web (de exemplu, Google PageRank [1]), rețele sociale (de exemplu, calculul centralității [2]), hărți cu drumuri (de exemplu, drumul minim între două puncte) sau modelare grafică (de exemplu, Gephi - <https://gephi.org/> sau D3JS - <https://d3js.org/>).

VI.2 Descrierea problemei și a rezolvărilor

Un graf poate fi modelat drept o pereche de două mulțimi $G = (V, E)$. Mulțimea V conține nodurile/vârfurile grafului (*vertices*), iar mulțimea E conține muchiile/arcele (*edges*), fiecare muchie stabilind o relație de vecinătate între două noduri. O mare varietate de probleme se modelează folosind grafuri, iar rezolvarea acestora presupune explorarea nodurilor grafului. O parcurgere își propune să viziteze într-un mod sistematic fiecare nod al grafului, exact o singură dată, pornind de la un nod ales, numit în continuare nod *sursă*.

Reprezentarea în memorie a grafurilor se face, de obicei, cu liste de adiacență sau cu o matrice de adiacență. Se pot folosi însă și alte structuri de date, de exemplu o listă de tuple (*hashmap*) $\langle\langle\text{sursă, destinație}\rangle, \text{cost}\rangle$.

Pe parcursul rulării algoritmilor de parcurgere, un nod poate avea 3 culori, codificând 3 stări posibile:

- *Alb* = nod nedescoperit încă de parcurgere;
- *Gri* = nodul a fost descoperit și este în curs de procesare;
- *Negru* = nodul a fost complet procesat.

Se poate face o analogie cu o pată neagră care se extinde pe un spațiu alb. Nodurile gri se află pe frontiera petei negre și conține nodurile care mai pot aduce informații utile parcurgerii.

Algoritmii de parcurgere pot fi considerați și ca algoritmi de căutare a unei căi de la un nod inițial la un alt nod. În acest caz, ei pot fi caracterizați prin completitudine și optimalitate. Un algoritm de căutare complet va descoperi întotdeauna o cale, dacă aceasta există. Un algoritm de explorare optimal va descoperi calea optimă din perspectiva numărului de pași care trebuie efectuați (sau de muchii folosite între cele două noduri).

VI.2.1 PARCURGerea ÎN LĂȚIME – BFS

Parcurerea în lățime (*Breadth-First Search - BFS*) [3, 4] este un algoritm de căutare pentru grafuri în care, atunci când se ajunge într-un nod oarecare v , nevizitat, se vizitează toate nodurile nevizitate adiacente lui v , abia apoi toate vârfurile nevizitate adiacente vârfurilor adiacente lui v , etc. *Atenție!* BFS depinde de nodul de start al parcurgerii. În consecință, plecând dintr-un nod sursă dat, se va parcurge doar componenta conexă din care acesta face parte.

Dacă se aplică algoritmul BFS asupra fiecărei componente conexe a grafului, se obține o pădure de arbori de acoperire ai grafului, numiți *arbori de lățime*, câte unul pentru fiecare componentă conexă. Pentru a putea construi acești arbori, se păstrează pentru fiecare nod din graf identitatea nodului din care a fost descoperit, nod numit *părinte*.

În cazul în care nu există o funcție de lungime (sau de cost) asociată muchiilor grafului, BFS va determina și drumurile minime de la rădăcină la oricare nod. În acest caz, prin drum minim înțelegem numărul minim de muchii parcurse de la sursă (care este și rădăcina arborelui de lățime) la toate nodurile din graf, drept urmare se poate considera că fiecare muchie are un cost egal cu 1. Arborele de lățime are rădăcina în sursa parcurgerii și va conține aceste drumuri minime către fiecare nod din graf ce poate fi atins din sursă.

Pentru implementarea BFS se folosește o coadă. În momentul adăugării în coadă, un nod trebuie colorat din alb în gri (a fost descoperit și urmează să fie prelucrat).

Algoritmul de explorare BFS este complet și optimal, semnificând că dacă există un drum de la sursă până la un nod oarecare din graf, algoritmul îl va descoperi în mod sigur – mai mult, acest drum va avea costul minim, în sensul discutat anterior, dintre toate drumurile posibile.

```
BFS( $s$ ,  $G$ ) {
    foreach ( $u \in V$ )
         $p(u) = \text{null}$ ; // initializări
         $\text{dist}(u) = \text{inf}$ ;
         $c(u) = \text{alb}$ ;
```

```

dist(s) = 0;      // distanța până la sursa este 0
c(s) = gri;       // se începe prelucrarea nodului,
                  // deci culoarea devine gri
Q = {}; //se folosește o coadă cu nodurile de prelucrat
Q = Q + {s}; // se adaugă sursa în coadă
while (!empty(Q)) //cât timp există noduri de prelucrat
    u = top(Q); //se determină nodul din vârful cozii
    foreach (v ∈ succesorii(u)) // pentru toți vecinii
        if (c(v) == alb) // nodul nu este în coada
            // actualizăm structura de date
            dist(v) = dist(u) + 1;
            p(v) = u;
            c(v) = gri;
            Q = Q + {v};
    c(u) = negru; //nodul a fost terminat
    Q = Q - {u}; //nodul este eliminat din coadă

```

Complexitate:

- Folosind liste de adiacență (algoritmul de mai sus): $O(|E|+|V|)$;
- Folosind matrice de adiacență: $O(|V|^2)$.

Un exemplu de rulare a principalilor pași ai parcurgerii în lățime pentru un graf orientat este oferit în Figura 6.1.

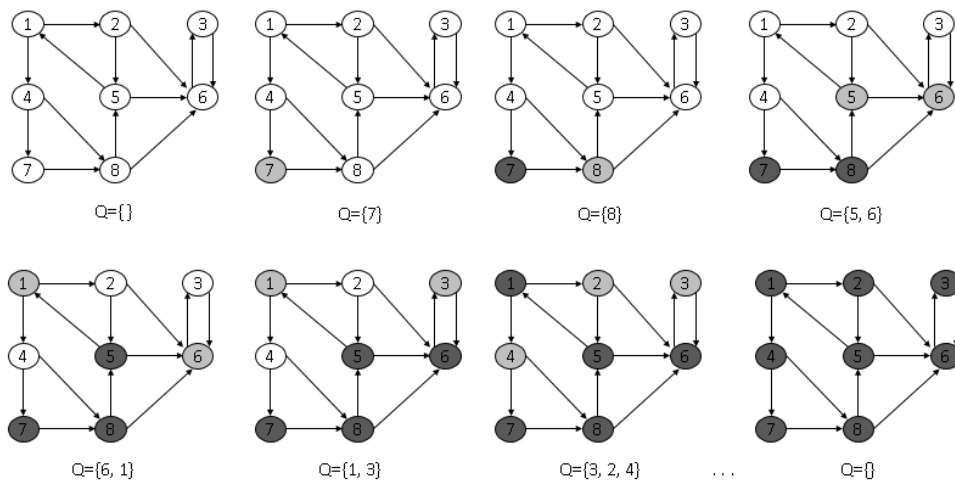


Figura 6.1 – Parcurgere în lățime pornind din nodul sursă $s=7$

VI.2.2 PARCURGEREA ÎN ADÂNCIME – DFS

Parcurgerea în adâncime (*Depth-First Search - DFS*) [3, 4], spre deosebire de BFS, nu mai pornește de la un nod dat, ci consideră toate nodurile nevizitate la un moment dat. Se pleacă inițial de la un nod oarecare. Se alege primul vecin nevizitat (alb) al nodului în curs de explorare și se marchează și acesta ca fiind în curs de procesare (gri). Apoi și pentru acest nod se caută primul vecin nevizitat, și așa mai departe. În momentul în care nodul curent nu mai are vecini nevizitați, se marchează ca fiind procesat (negru) și se revine la nodul anterior. Pentru acest nod, se caută din nou următorul vecin nevizitat. Algoritmul continuă până când toate nodurile grafului au fost procesate. Dacă graful are mai multe componente conexe, când algoritmul termină de vizitat una dintre ele alege un nod de start nevizitat din celelalte componente conexe, dacă acesta mai există.

În urma aplicării algoritmului DFS, se obține pentru fiecare componentă conexă câte un arbore de acoperire, numit *arbore de adâncime*. Pentru a putea construi acest arbore, se păstrează pentru fiecare nod dat identitatea părintelui său, similar cu procedeul folosit de către BFS.

Pentru fiecare nod v se vor reține:

- timpul descoperirii – $d[v]$;
- timpul finalizării – $f[v]$;
- părintele – $p[v]$;
- culoarea – $c[v]$.

Algoritmul de explorare **DFS** nu este nici complet (în cazul unei căutări pe un subarbore infinit, algoritmul va cicla și nu va descoperi drumurile nici către noduri mai apropiate, dar încă nevizitate înainte de a ajunge în subarboarele infinite), nici optimal (nu găsește drumul de lungime minimă de la nodul start la alt nod din graf).

Spre deosebire de BFS, pentru implementarea DFS se folosește o stivă (abordare LIFO în loc de FIFO). Deși se poate face direct această înlocuire în algoritmul anterior folosit de către BFS, de cele mai multe ori este mai intuitivă folosirea recursivității pentru a simula stiva.

DFS (G)

```
V = noduri(G)
foreach (u ∈ V)
    // initializare structura date
    c(u) = alb;
    p(u) = null;
```

```

timp = 0;
foreach (u ∈ V)
    if (c(u) == alb) explorare(u);

```

explorare(u)

```

d(u) = ++timp; // timpul de descoperire al nodului u
c(u) = gri; // nod în curs de explorare
foreach (v ∈ succesorii(u)) // prelucrare vecini
    if (c(v) == alb) // nu au fost prelucrat deja
        p(v) = u;
        explorare(v);
c(u) = negru; // am terminat de prelucrat nodul curent
f(u) = ++timp; // timpul de finalizare al nodului u

```

Complexitate:

- Folosind liste de adiacență (algoritmul de mai sus): $O(|E|+|V|)$;
- Folosind matrice de adiacență: $O(|V|^2)$.

În Figura 6.2 este evidențiat un exemplu de parcurgere în adâncime, împreună cu timpii de descoperire și de finalizare ai nodurilor la fiecare pas. Pentru simplificarea pozei, unii pași mai puțin importanți au fost omiși.

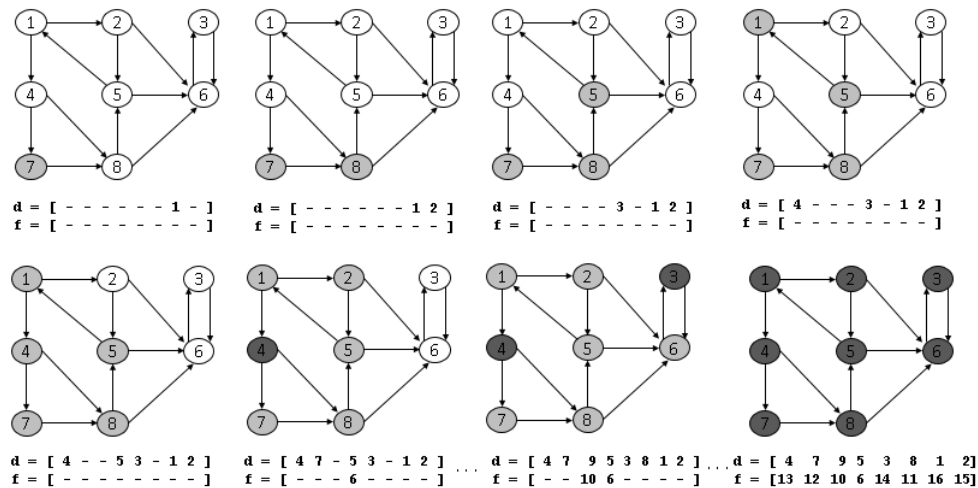


Figura 6.2 – Parcurgere în adâncime, cu evidențierea timpilor de descoperire și de finalizare de la fiecare pas.

VI.2.3 SORTAREA TOPOLOGICĂ

Dându-se un graf orientat aciclic (*Directed Acyclic Graph – DAG*), sortarea topologică [3, 4] realizează o aranjare liniară (o listă) a nodurilor în funcție de muchiile dintre ele. Orientarea muchiilor corespunde unei relații de ordine de la nodul sursă către cel destinație. Astfel, dacă (u, v) este una dintre arcele grafului, u trebuie să apară înaintea lui v în înșiruire (practic, indicele lui u din sortare trebuie să fie mai mic decât indicele nodului v). Dacă graful ar fi ciclic, nu ar putea exista o astfel de înșiruire întrucât nu se poate stabili o ordine între nodurile care alcătuiesc un ciclu conform definiției de mai sus.

Grafic, sortarea topologică poate fi văzută ca o metodă de generare a unei ordini a nodurilor, astfel încât dacă acestea ar fi plasate de-a lungul unei linii orizontale conform acestei ordonări, toate arcele grafului să fie direcționate de la un nod aflat mai în stânga către unul aflat mai în dreapta. Pentru un exemplu, puteți consulta Figura 6.3 care prezintă mai multe sortări topologice ale aceluiași graf.

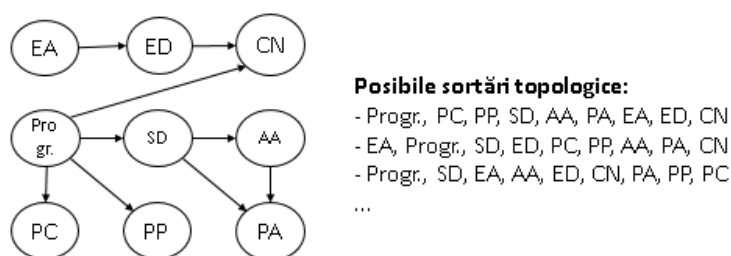


Figura 6.3 – Graf orientat aciclic împreună cu trei posibile sortări topologice

Doi algoritmi cunoscuți pentru rezolvarea sortării topologice sunt:

a) Algoritmul bazat pe DFS:

- parcurgere DFS pentru determinarea timpilor de finalizare;
- sortare descrescătoare a nodurilor grafului în funcție de timpul de finalizare.

Pentru a evita sortarea nodurilor în funcție de timpul de finalizare, se poate folosi o stivă în care sunt introduse nodurile atunci când acestea sunt finalizate (colorate din gri în negru) și parcurgerea în adâncime se termină pentru nodul respectiv. La afișarea ulterioară a nodurilor din stivă, primul nod scos din stivă va fi ultimul introdus (cu timpul de finalizare cel mai mare), iar ultimul nod scos din stivă va avea timpul de finalizare cel mai mic (fiind primul introdus).

b) Un alt algoritm clasic este cel propus de către **Kahn**:

```
Kahn (G) {
    V = noduri(G);
    L = {}; // lista care va conține elementele sortate
    // inițializare S cu nodurile care nu au in-muchii
    S = {};
    foreach (u ∈ V)
        if (u nu are in-muchii)
            S = S + {u};
    while (!empty(S)) // cât timp există noduri de prelucrat
        u = random(S); // se scoate un nod din mulțimea S
        L = L + {u}; // se adaugă u la lista finală
        foreach (v ∈ succesori(u)) // pentru toți vecinii
            sterge (u, v) din G; // șterge muchia u-v
            if (v nu are in-muchii)
                S = S + {v}; // se adaugă v la mulțimea S
    if (G are muchii)
        print(eroare); // graf ciclic
    else
        print(L); // ordinea conform sortării topologice
}
```

Complexitate: $O(|E|+|V|)$ folosind liste de adiacență.

VI.3 Concluzii și observații

Grafurile sunt foarte importante pentru reprezentarea și rezolvarea unui set variat de probleme. Cele mai uzuale moduri de reprezentare a unui graf presupun utilizarea listelor de adiacență sau a matricelor de adiacență.

Cele două moduri uzuale de parcurgere (căutare neinformată) a unui graf sunt:

- BFS – parcurgere în lățime;
- DFS – parcurgere în adâncime.

Sortarea topologică este o modalitate de aranjare a nodurilor în funcție de muchiile dintre ele. În funcție de nodul de *start* din cadrul DFS se pot obține sortări diferite, păstrând însă proprietățile generale ale sortării topologice. În general, sortarea topologică este o etapă de preprocesare utilă pentru a ajunge la soluții mai eficiente pentru rezolvarea anumitor probleme (de exemplu, calculul drumurilor minime) pentru grafuri orientate aciclice.

VI.4 Probleme propuse

1. Trebuie să evadez! Nu mai rezist!

Se dă un tablou bidimensional reprezentând suprafața unei camere. Camera poate să conțină obstacole, reprezentate în matrice printr-o celulă de valoare 1, prin care nu se poate circula. Celulele cu valoarea 0 reprezintă elementele libere ale camerei prin care se poate trece. Dându-se un punct interior de start, să se decidă care este cel mai scurt drum până la o ieseire. Ieșirile pot să fie doar pe marginea tabloului, și sunt simbolizate prin cifra 2. Să se afișeze și drumul până la una din cele mai apropiate ieșiri.

2. Cum ar trebui studiate optim materiile?

Se dă setul de mai jos de dependențe între materii (materia din stânga trebuie studiată într-un semestru anterior celei din dreapta). Să se precizeze o ordine de studiere a materiilor din curriculum.

Analiza Algoritmilor → Proiectarea Algoritmilor
Programarea Calculatoarelor → Proiectarea Algoritmilor
Structuri de Date → Proiectarea Algoritmilor
Proiectarea Algoritmilor → Algoritmi Paraleli și Distribuți
Protocoale de Comunicatie → Algoritmi Paraleli și Distribuți
Structuri de Date → Analiza Algoritmilor
Programarea Calculatoarelor → Protocoale de Comunicatie
Protocoale de Comunicatie → Rețele Locale
Algoritmi Paraleli și Distribuți → Algoritmi și Prelucrări Paralele
Arhitectura Sistemelor de Calcul → Algoritmi și Prelucrări Paralele
Matematica 1 → Matematica 2
Matematica 2 → Matematici speciale
Matematica 1 → Fizică
Fizică → Electrotehnică
Fizică → Electronică Analogică
Electronică Analogică → Electronică Digitală

VI.5 Indicații de rezolvare

1. Trebuie să evadez! Nu mai rezist!

Întrucât se dorește îndeplinirea criteriului de optimalitate se va aplica o parcurgere în lățime din poziția inițială, considerând totodată constrângerile impuse de celulele adiacente indisponibile (marcate cu 1). Labirintul poate fi modelat ca un graf, dacă considerăm că pentru orice celulă (i, j) , se poate construi ușor funcția $succesori(i, j)$ care să întoarcă celulele adiacente în care se poate trece.

2. Cum ar trebui studiate optim materiile?

Se aplică o sortare topologică urmată de o alocare greedy pe semestre. Preferabil, se poate opta pentru includerea materiilor cu cele mai multe dependențe ulterioare în primele semestre. Pentru această evaluare se vor realiza parcurgeri suplimentare în adâncime pornind din fiecare materie și se vor prioritiza materiile în ordinea descrescătoare a adâncimii maxime. Opțional, se poate realiza o combinație de Kahn și DFS pentru identificarea alocării optime.

Referințe

- [1] Brin, S.; Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems* 30: 107–117.
- [2] D'Andrea, Alessia et al. (2009). An Overview of Methods for Virtual Social Network Analysis. In Abraham, Ajith et al. *Computational Social Network Analysis: Trends, Tools and Research Advances*. Springer.
- [3] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press.
- [4] Giumale, C. (2004) *Introducere în Analiza Algoritmilor*. Bucuresti: Polirom

VII Aplicații DFS

VII.1 Importanță și aplicații practice

Parcurgerea în adâncime joacă un rol extrem de important în identificarea componentelor biconexe [1, 2] cu aplicații importante în rețelistică (o componentă biconexă asigură redundanța), importanța persoanelor într-o rețea socială, tare conexitate (de exemplu, în rezolvarea problemei 2-SAT). Totodată, descompunerea grafurilor orientate în componente tare conexe este utilizată în data mining, compilatoare sau calcul științific.

VII.2 Noțiuni teoretice

Tare conexitate. Un graf *orientat* este tare conex, dacă oricare ar fi două vârfuri u și v , ele sunt tare conectate („*strongly connected*”), adică există drum atât de la u la v , cât și de la v la u . Interpretând această definiție, ajungem la concluzia că într-un graf tare conex trebuie să existe un ciclu (eventual care nu este simplu) care să conțină toate vârfurile grafului.

O componentă tare conexă (CTC) este un subgraf maximal tare conex al unui graf orientat, adică o submulțime de vârfuri $U \subsetneq V$, astfel încât pentru orice u și v din U , ele sunt tare conectate. Dacă fiecare componentă tare conexă este redusă într-un singur nod, se va obține un graf orientat aciclic. Acest graf se mai numește și graful componentelor tare conexe.

Un punct (nod) de articulație (punct critic, „*cut vertex*”, „*articulation point*”) este un nod al unui graf *neorientat* a cărui eliminare duce la creșterea numărului de componente conexe ale acelui graf.

O punte (muchie critică, „*cut edge*”, „*bridge*”) este o muchie a unui graf *neorientat* a cărei eliminare duce la creșterea numărului de componente conexe ale acelui graf.

Biconexitate. Un graf biconex este un graf *neorientat* conex cu proprietatea că eliminând oricare nod al acestuia, graful rămâne conex. Un graf biconex se mai numește și **2-conectat**, iar definiția poate fi extinsă și pentru grafuri orientate (între orice 2 noduri u și v trebuie să existe cel puțin două căi disjuncte/fără a avea nici un nod în comun).

O **componentă biconexă** a unui graf *neorientat* este o mulțime maximală de noduri care respectă proprietatea de biconexitate.

VII.3 Componente tare conexe

Pornind de la definiție, pentru a afla componenta tare conexă din care face parte un nod v se va parcurge în adâncime graful pentru a găsi o mulțime de noduri S ce sunt accesibile din v . Se va parcurge apoi graful transpus (obținut prin inversarea arcelor din graful inițial), determinând astfel o nouă mulțime de noduri T ce sunt accesibile din v în graful transpus. Intersecția dintre S și T va reprezenta o componentă tare conexă. De remarcat că graful inițial și cel transpus au aceleași componente tare conexe.

Pornind de la această idee, în continuare sunt prezentați doi algoritmi cu complexitate liniară $O(|E| + |V|)$, care folosesc implementări diferite pentru a calcula CTC cât mai eficient. O implementare brută ar necesita calculul nodurilor ce pot fi atinse din fiecare nod v atât în graful inițial, cât și în cel transpus, ducând la o complexitate mai mare în cel mai defavorabil caz: $O(|V||E| + |V|^2)$.

VII.3.1 ALGORITMUL LUI KOSARAJU

Algoritmul propus de către Kosaraju folosește două parcurgeri în adâncime (prima pe graful inițial și a doua pe graful transpus - G^T) și o stivă pentru a reține ordinea terminării parcurgerii nodurilor grafului original (evitând astfel o sortare a nodurilor după timpul de finalizare la terminarea parcurgerii).

CTC (G= (V, E))

```

S = stiva vida
// prima parcurgere folosind graful inițial
culoare[1..n] = alb
while (există un nod v din V care să fie alb)
    DFS(G, v, S)
// a doua parcurgere pe graful transpus
culoare[1..n] = alb
while (!empty(S))
    v = pop(S)
    if (culoare[v] == alb)
        DFS_T(GT, v)
        /* toate nodurile ce pot fi vizitate din v fac
        parte din aceeași CTC */

```

DFS (G, v, S)

```

culoare[v] = gri

```



```

foreach ((v, u) ∈ E)
    if (culoare[u] == alb)
        DFS(G, u, S)
push(S, v) //nodul expandat este pus pe stiva
culoare[v] = negru

```

DFS_T(G, v, S)

//similar cu DFS(G, v, S): fără a introduce nodurile în stiva S, dar cu reținerea CTC într-o structură de tip listă de liste de vârfuri (câte o listă pentru fiecare CTC).

Complexitate (folosind liste de adiacență): $O(|V| + |E|)$, fiind două parcurgeri în adâncime apelate succesiv.

În Figura 7.1 este reprezentat un exemplu de rulare a celor două parcurgeri în adâncime folosite de către algoritmul lui Kosajaru pentru determinarea CTC. Se pot observa ordinea parcurgerii nodurilor pentru graful transpus G^T , precum și componentele tare conexe rezultate.

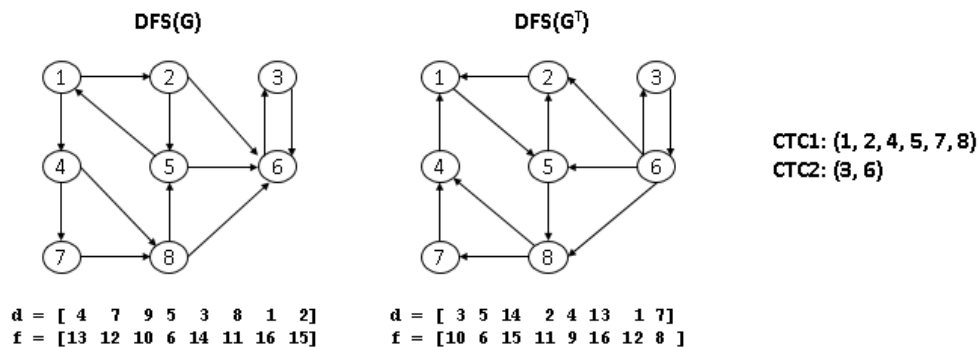


Figura 7.1 – Algoritmul lui Kosaraju pentru determinarea componentelor tari conexe (CTC) aplicat asupra unui graf orientat cu 2 CTC

VII.3.2 ALGORITMUL LUI TARJAN

Algoritmul lui Tarjan [3] folosește o singură parcurgere DFS și o stivă. Ideile de bază ale algoritmului sunt că o parcurgere în adâncime pornește dintr-un nod de start și că toate componentele tare conexe formează subarbori ai arborelui de adâncime rezultat, iar rădăcinile acestor subarbori sunt strămoși (adică primul nod descoperite de parcurgere din fiecare CTC) pentru componentele tare conexe. Nodurile sunt puse pe o stivă, în ordinea vizitării. Când parcurgerea a terminat de vizitat un subarbore, deci când se finalizează un strămoș pentru o CTC, nodurile sunt scoase din stivă și se determină pentru

fiecare nod dacă este rădăcina unei componente tare conexe. Dacă un nod este rădăcina unei componente, atunci el și toate nodurile scoase din stivă înaintea lui formează acea componentă tare conexă.

Algoritmul este centrat pe identificarea dacă un nod este sau nu rădăcină a unei componente tare conexe. Pentru a face asta, fiecărui nod i se atribuie un index în urma parcurgerii în adâncime ($idx[v]$, acesta este similar cu timpul de descoperire), ce numără nodurile în ordinea în care sunt descoperite.

În plus, pentru fiecare nod se ține și o valoare $lowlink[v] = \min\{idx[u] \mid u \text{ este accesibil din } v\}$. Prin urmare, v este rădăcina unei componente tare conexe dacă și numai dacă $lowlink[v] = idx[v]$, toate celelalte noduri din acea CTC având $idx[v] > lowlink[v]$ (întrucât au fost descoperite mai târziu ca strămoșul sau rădăcina CTC). Vectorul $lowlink[1..|V|]$ se calculează în timpul parcurgerii în adâncime.

CTC_Tarjan($G=(V, E)$)

```
index = 0
S = {}
foreach (v ∈ V)
    if (idx[v] nu e definit) // nu a fost vizitat
        Tarjan(G, v)
```

Tarjan(G, v)

```
idx[v] = index
lowlink[v] = index
index = index + 1
push(S, v)
foreach ((v, u) ∈ E)
    if (idx[u] nu e definit) // nodul u nu a fost vizitat
        Tarjan(G, u)
        lowlink[v] = min(lowlink[v], lowlink[u])
    else
        if (u este în S) // dacă muchia (v, u) este
                        // muchie înapoi
            lowlink[v] = min(lowlink[v], idx[u])
if (lowlink[v] == idx[v])
    // v este rădăcina unei CTC
    print "O nouă CTC:"
    repeat
        u = pop(S)
        print u
    until (u == v)
    println
```

Complexitate: $O(|V| + |E|)$

VII.4 Puncte de articulație

Pentru determinarea punctelor de articulație într-un graf neorientat se folosește o parcurgere în adâncime modificată, reținându-se informații suplimentare pentru fiecare nod. Fie T un arbore de adâncime descoperit de parcurgerea grafului. Atunci, un nod v este punct de articulație dacă:

- v este rădăcina lui T și v are doi sau mai mulți copii. Dacă v este eliminat din graf, nodurile din subarborii dominați de acești copii (cel puțin doi) vor deveni deconectate.
- v nu este rădăcina lui T și are un copil u în T , astfel încât nici un nod din subarborii dominați de u ($Arb(u)$) nu este conectat cu un strămoș al lui v printr-o muchie înapoi. Drept urmare, din nici un nod din $Arb(u)$ nu se poate ajunge pe altă cale pe un nivel superior lui v în arborele de adâncime. Astfel, dacă v este eliminat din graf, în mod sigur nodurile din $Arb(u)$ vor fi deconectate de toți strămoșii lui v (și există cel puțin unul, întrucât v nu este rădăcina arborelui de adâncime).

Fie $d[v]$ = timpul de descoperire a nodului u ; notăm cu $low[v] = \min\{d[v]\} \cup \{d[x] \mid (v, x) \text{ este o muchie înapoi de la un descendent } v \text{ al lui } u\}$. Se poate observa cu ușurință că notațiile sunt foarte similare cu cele de la algoritmul lui Tarjan pentru descoperirea CTC într-un graf orientat. De fapt, aceasta este o variație a algoritmului respectiv, introdusă tot de Tarjan, pentru descoperirea punctelor critice în grafuri neorientate.

Pentru a putea identifica punctele de articulație care se încadrează în cel de-al doilea caz, se va calcula $low[v]$ pentru fiecare vârf, în timpul parcurgerii în adâncime. Nodul v este punct de articulație dacă și numai dacă $low[u] \geq d[v]$, pentru un copil u al lui v în T .

Funcția low se poate calcula folosind relația de recurență:

$$low[u] = \min(\{d[u]\} \cup \{d[v] \mid (u, v) \text{ este o muchie înapoi} \} \cup \{low[w] \mid w \text{ copil al lui } v \text{ în arborele de adâncime}\})$$

Puncte_articulatie($G = (V, E)$)

```
timp = 0
foreach (v ∈ V)
    if (d[v] nu e definit)
        dfsCV(G, v)
```

dfsCV(G, v)

```
d[v] = timp
timp = timp + 1
```

```

low[v] = d[v]
copii = { } // multimea viza
foreach ((v, u) ∈ E)
    if (u nu este parintele lui v)
        if (d[u] nu e definit)
            copii = copii ∪ {u}
            dfsCV(G, u)
            low[v] = min(low[v], low[u])
        else
            low[v] = min(low[v], d[u])

if (v radacina arborelui and |copii| ≥ 2)
    v este punct de articulatie
else
    if (∃ u ∈ copii cu low[u] ≥ d[v])
        v este punct de articulatie

```

Figura 7.2 prezintă un exemplu de aplicare a algoritmului lui Tarjan pentru determinarea punctelor de articulație dintr-un graf neorientat. În cadrul acestuia sunt scoase în evidență valorile vectorilor $d[u]$ și $low[u]$.

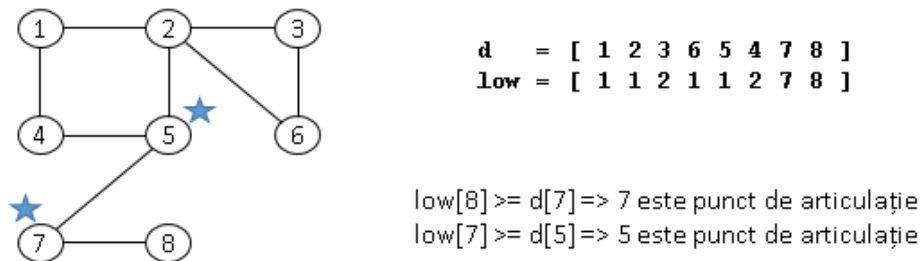


Figura 7.2 – Algoritmul lui Tarjan pentru determinarea punctelor de articulație ale unui graf neorientat

VII.5 Punți

Pentru a determina muchiile critice (punțile) se folosește tot o parcurgere în adâncime modificată, foarte similară cu cea de la punctele de articulație. Se pornește de la următoarea observație: muchiile critice sunt muchiile care nu apar în niciun ciclu. Prin urmare, o muchie de întoarcere descoperită în timpul parcurgerii nu poate fi critică, deoarece o astfel de muchie închide

întotdeauna un ciclu. Astfel, trebuie verificat doar dacă muchiile de arbore (în număr de $|V| - 1$) fac parte dintr-un ciclu.

În continuare este folosit tot vectorul $low[v]$ (definit la punctul anterior), iar dacă din nodul u se poate ajunge pe un nivel mai mic sau egal cu nivelul lui v , atunci muchia de arbore (v, u) nu este critică; în caz contrar, ea este critică. Drept urmare, se poate modifica doar funcția de parcurgere recursivă așa cum este prezentat în continuare.

```
dfsB(G, v, parinte)
    d[v] = timp
    timp = timp + 1
    low[v] = d[v]
    foreach ((v, u) ∈ E)
        if (u nu este parintele lui v)
            if (d[u] nu este definit)
                dfsB(G, u, v)
                low[v] = min(low[v], low[u])
                if (low[u] > d[v])
                    (v, u) este muchie critica
            else
                low[v] = min(low[v], d[u])
```

Complexitate: $O(|V| + |E|)$

VII.6 Componente biconexe

Împărțirea în componente biconexe nu introduce ca și până acum (conexitate, tare conexitate) o partiție a nodurilor grafului, ci o partiție a muchiilor acestuia. Se remarcă că o componentă biconexă este o componentă care nu are puncte critice. Astfel, se poate adapta algoritmul de aflare a punctelor critice, reținând și o stivă cu toate muchiile directe și de întoarcere parcurse până la un moment dat. La întâlnirea unui nod critic v , se formează noua componentă biconexă extrăgând din stivă muchiile corespunzătoare. Nodul v este critic dacă a fost găsit un copil u din care nu se poate ajunge pe un nivel mai mic în arborele de adâncime, pe un alt drum care folosește muchii de întoarcere ($low[u] \geq d[v]$). Atunci când este identificat un astfel de nod u , toate muchiile aflate în stivă până la muchia (u, v) inclusiv formează o nouă componentă biconexă.

Complexitate: $O(|V| + |E|)$

VII.7 Concluzii și observații

Algoritmul de parcurgere în adâncime poate fi modificat pentru calculul componentelor tare conexe, a punctelor de articulație, a punților și a componentelor biconexe. Complexitatea tuturor acestor algoritmi este cea a parcurgerii: $O(|V| + |E|)$. Acestea sunt doar câteva dintre aplicațiile cele mai cunoscute ale DFS.

VII.8 Probleme propuse

Into the Matrix

Tocmai s-a lansat o noua rețea socială, dedicată exclusiv gamerilor. Întrucât toată lumea dorea să discute cu prietenii lor din comunitate într-un mod cât mai privat, comunicarea este unidirecțională prin canale dedicate. Astfel, atunci când un jucator intră prima dată pe un server, indică toate persoanele pe care le consideră prieteni și stabilește relațiile outbound. Ulterior, terțe persoane pot crea tuneluri permanente în sens invers pentru a permite comunicația bidirecțională între 2 noduri.

- 1) Identificați toate clanurile de pe un server. (Mesajele între oricare 2 membri dintr-un clan pot fi rutate prin canalele deschise)
- 2) Descoperiți ce relații există între clanuri. Care ar fi cel mai numeros clan dacă s-ar adauga o legătură artificială între oricare doi jucatori?

Ulterior, pentru a spori ușurința utilizării, canalele de comunicație au devenit bi-direcționale și, pentru a evalua relațiile dintre jucători, dorim să identificăm următoarele elemente cheie din fiecare clan:

- 3) Care sunt jucătorii critici a căror eliminare ar duce la distrugerea comunicației în cadrul clanului?
- 4) Care canale între 2 jucători sunt critice la nivelul clanului?

VII.9 Indicații de rezolvare

- 1) Graful este orientat și se aplica algoritmul lui Kosaraju sau Tarjan pentru identificarea componentelor tare conexe.
- 2) Varianta cea mai elegantă presupune contracția muchiilor (toate nodurile dintr-un CTC și muchiile aferente sunt contractate într-un singur nod),

urmând ca muchia adăugată să creeze un ciclu în graful anterior și care să maximizeze numărul total de membri din CTC-uri incluse în ciclul nou creat. Soluția finală presupune adăugarea unei muchii înapoi între oricare dintre membrii celor două sub-comunități anterioare unificate prin muchia dintre CTC-uri.

3, 4). Jucătorii critici reprezintă punctele de articulație din graful neorientat, iar canalele critice sunt identificate sub formă de punți.

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) *Introducere în Analiza Algoritmilor*. Bucuresti: Polirom
- [3] Tarjan, R. E. (1972), "Depth-first search and linear graph algorithms", *SIAM Journal on Computing* 1 (2): 146–160

VIII Drumuri minime

VIII.1 Importanță și aplicații practice

Algoritmii pentru determinarea drumurilor minime [1, 2] au multiple aplicații practice și reprezintă clasa de algoritmi pe grafuri cel mai des utilizată în vederea: rutării în cadrul unei rețele (telefonice, de calculatoare etc.), găsirii drumului minim dintre două locații (Google Maps, GPS, etc.), stabilirea unei agende de zbor în vederea asigurării unor conexiuni optime, sau asignarea unui peer / server de fișiere în funcție de metricile definite pe fiecare linie de comunicație.

VIII.2 Concepte

VIII.2.1 COSTUL UNEI MUCHII ȘI AL UNUI DRUM

Fiind dat un graf orientat $G = (V, E)$, se consideră funcția $w: E \rightarrow R$, numită funcție de cost, care asociază fiecărei muchii o valoare numerică. Domeniul funcției poate fi extins, pentru a include și perechile de noduri între care nu există muchie directă, caz în care valoarea costului este $+\infty$ (infiniț). Costul unui drum p format din muchiile $p_{12} p_{23} \dots p_{(n-1)n}$, având costurile $w_{12}, w_{23}, \dots, w_{(n-1)n}$, este aditiv $w_p = w_{12} + w_{23} + \dots + w_{(n-1)n}$.

VIII.2.2 DRUMUL DE COST MINIM

Costul minim al drumului dintre două noduri este minimul dintre costurile tuturor drumurilor existente între cele două noduri. Deși, în cele mai multe cazuri, costul este o funcție cu valori nenegative, există situații în care un graf cu muchii de cost negativ are relevanță practică (de exemplu, jocurile care au atât câștiguri, cât și costuri (pierderi) între două stări). O parte din algoritmi pot determina drumul corect de cost minim inclusiv pe astfel de grafuri. Totuși, nu are sens căutarea drumului minim în cazurile în care graful conține cicluri de cost negativ – un drum minim ar avea lungimea infinită, întrucât costul său s-ar reduce la fiecare reparcurgere a ciclului, iar costul ar tinde către $-\infty$.

VIII.2.3 RELAXAREA UNEI MUCHII

Relaxarea unei muchii (v_1, v_2) constă în a testa dacă se poate reduce costul ei, trecând printr-un nod intermediar u . Fie w costul inițial al muchiei de la v_1 la v_2 , w_{1u} costul muchiei de la v_1 la u , și w_{u2} costul muchiei de la u la v_2 . Dacă $w > w_{1u} + w_{u2}$, muchia directă este înlocuită cu succesiunea de muchii (v_1, u) , (u, v_2) . Toți algoritmi prezentați în continuare se bazează pe mecanismul de relaxare pentru a determina drumul minim.

Alternativ, relaxarea unei muchii poate fi văzută ca o metodă de a scade estimarea distanței cele mai bune descoperite până la un moment dat de la un nod predefinit s la un alt nod v , estimare reținută în $d[v]$. Relaxând o muchie oarecare (u, v) încercăm să vedem dacă putem îmbunătăți estimarea $d[v]$ folosind muchia (u, v) . Pseudocodul este foarte simplu, iar comportamentul este ilustrat în Figura 8.1.

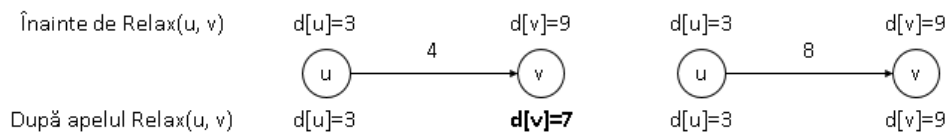


Figura 8.1 – Două exemple în care este relaxată muchia (u, v) . În stânga, estimarea $d[v]$ este îmbunătățită, iar în exemplul din dreapta nu este modificată.

```
Relax(u, v):
    // relaxează muchia (u, v)
    if (d[v] > d[u] + w[u, v])
        d[v] = d[u] + w[u, v]
```

VIII.3 Drumuri minime de sursă unică

Algoritmi din această secțiune determină drumul de cost minim de la un nod sursă către restul nodurilor din graf, pe bază de relaxări repetate ale muchiilor. Diferențele între acești algoritmi sunt date de numărul de relaxări efectuate pentru fiecare muchie și de ordinea în care sunt făcute aceste relaxări.

VIII.3.1 ALGORITMUL DIJKSTRA

În primul rând, trebuie precizat că algoritmul Dijkstra poate fi folosit doar pentru grafuri care au toate muchiile nenegative. Algoritmul este de tip Greedy: optimul local folosit este reprezentat de costul drumului cel mai bun descoperit până la momentul respectiv dintre nodul sursă s și un nod v . Pentru fiecare nod, se reține un cost estimat $d[v]$, inițializat la început cu costul muchiei $s-v$, sau cu $+\infty$, dacă nu există muchie. După cum se poate observa, această estimare este pesimistă, valoarea ei scăzând repetat prin relaxări de muchii până se atinge valoarea drumului de cost minim pentru fiecare vârf.

Algoritmul selectează, în mod repetat, nodul u care are, la momentul respectiv, costul estimat minim (față de nodul sursă). În continuare, se încearcă relaxarea muchiilor (u, v) , operație ce va îmbunătăți costurile $d[v]$ pentru anumite vârfuri v adiacente lui u .

Astfel, dacă $d[v] > d[u] + w_{uv}$, după relaxarea muchiei (u, v) , $d[v]$ ia valoarea $d[u] + w_{uv}$ îmbunătățind astfel estimarea $d[v]$. Pentru a ține evidența muchiilor care trebuie relaxate, se folosesc două structuri: S (mulțimea de vârfuri deja vizitate) și Q (o coadă cu priorități, în care nodurile se află ordonate după distanța față de sursă) din care este mereu extras nodul aflat la distanță minimă. În S se află inițial doar sursa, iar în Q doar nodurile spre care există muchie directă de la sursă, deci care au $d[nod] < +\infty$.

Algoritmul se încheie când coada Q devine vidă, sau când S conține toate nodurile. Pentru a putea determina și muchiile din care este alcătuit drumul minim căutat, nu doar costul său final, este necesară utilizarea unui vector de părinți P (cu un înțeles similar cu vectorul de părinți folosit la parcurgeri). Pentru nodurile care au muchie directă de la sursă, $P[nod]$ este inițializat cu sursa, pentru restul cu *null*.

Pseudocodul pentru determinarea drumului minim de la o sursă către celelalte noduri utilizând algoritmul lui Dijkstra, și care apoi afișează drumul de la sursă la un nod destinație *dest*, este următorul:

Dijkstra(sursa, dest):

```

Q = {}
foreach (nod ∈ V) // V = mulțimea nodurilor
    selectat[nod] = false
    if ((sursa, nod) ∈ E) // E = mulțimea
                                // muchiilor
        // inițializăm distanța până la nodul respectiv
        d[nod] = w[sursa, nod]
        insereaza(Q, nod, d[nod])
        // părintele nodului devine sursa

```

```

        P[nod] = sursa
    else
        d[nod] = +∞ // distanță infinită
        P[nod] = null // nu are părinte
    selectat[sursa] = true
    d[sursa] = 0
    // relaxari succesive
    while(!empty(Q))
        u = extrage_min(Q)
        selectat[u] = true
        foreach (nod ∈ vecini[u]) // (*)
            /* dacă drumul de la sursă la nod prin u este mai
               mic decât cel curent */
            if (!selectat[nod] & d[nod] > d[u] + w[u, nod])
                // actualizează distanța, părintele și
                poziția nodului în coada de priorități */
                d[nod] = d[u] + w[u, nod]
                P[nod] = u
                actualizeaza (Q, nod, d[nod])

    // găsierea drumului efectiv între sursa și dest
    initializeaza Drum = {}
    nod = P[dest]
    while (nod != null)
        inserează nod la începutul lui Drum
        nod = P[nod]

```

Reprezentarea grafului ca matrice de adiacență duce la o implementare ineficientă pentru orice graf care nu este complet, datorită parcurgerii vecinilor nodului u , din linia (*), care se va executa în $|V|$ pași pentru fiecare extragere din Q , iar pe întreg algoritmul vor rezulta $|V|^2$ pași. Este preferată reprezentarea grafului cu liste de adiacență, pentru care numărul total de operații cauzate de linia (*) va fi egal cu $|E|$.

Complexitatea algoritmului este $O(|V|^2 + |E|)$ în cazul în care coada cu priorități este implementată ca o căutare liniară într-un vector. În acest caz funcția $extrage_min(Q)$ se execută în timp $O(|V|)$, iar $actualizeaza(Q)$ în timp $O(1)$.

O variantă mai eficientă presupune implementarea cozii ca heap binar. Funcția $extrage_min(Q)$ se va executa în timp $O(\log(|V|))$; funcția $actualizeaza(Q)$ se va executa tot în timp $O(\log(|V|))$, dar trebuie cunoscută poziția cheii nod în heap, pentru ca elementul să fie reindexat în heap. Complexitatea obținută este $O(|E| \lg |V|)$ pentru un graf conex, care este mai eficientă pentru grafuri rare decât implementarea folosind vectori, dar mai ineficientă pentru grafuri dense.

Cea mai eficientă implementare se obține folosind un heap Fibonacci [3] pentru coada cu priorități. Aceasta este o structura de date complexă, dezvoltată în mod special pentru optimizarea algoritmului lui Dijkstra, caracterizată de un timp amortizat de $O(\log(|V|))$ pentru operația *extrage_min(Q)* și numai $O(1)$ pentru *actualizeaza(Q)*. Complexitatea obținută este $O(|V|\log(|V|) + |E|)$, foarte bună pentru grafuri rare și pentru grafuri dense.

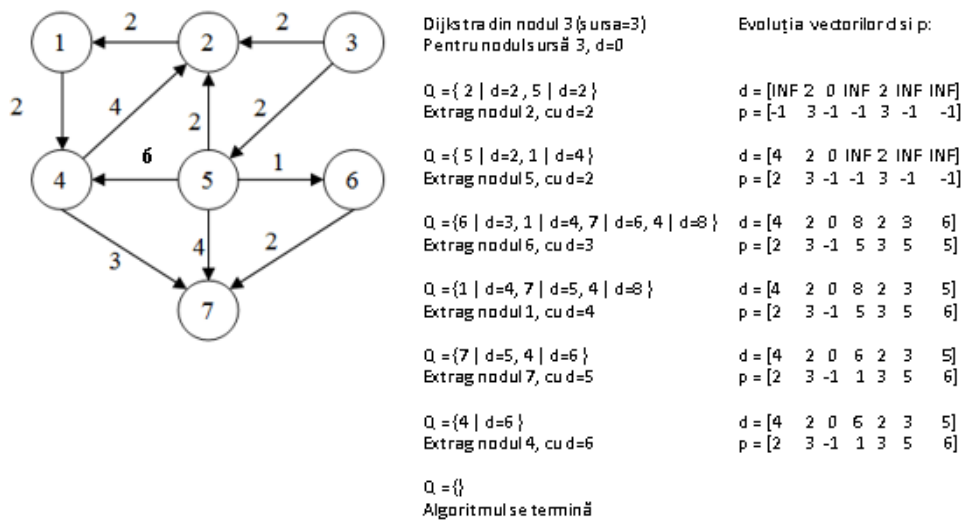


Figura 8.2 – Funcționarea algoritmului lui Dijkstra pentru determinarea drumurilor minime de sursă unică, cu evidențierea nodurilor din coadă și a valorilor din vectorii $d[u]$ și $p[u]$

VIII.3.2 ALGORITMUL BELLMAN – FORD

Algoritmul Bellman-Ford poate fi folosit pentru a calcula drumurile de cost minim de la o sursă la toate celelalte vârfuri și pentru grafuri ce conțin muchii de cost negativ, dar nu poate fi folosit pentru grafuri ce conțin cicluri de cost negativ (când căutarea unui drum minim nu are sens). Cu ajutorul său se poate determina, de asemenea, dacă un graf conține cicluri de cost negativ.

Algoritmul folosește același mecanism de relaxare a muchiilor utilizat în cadrul algoritmului lui Dijkstra, dar, spre deosebire de acesta, nu optimizează o soluție folosind un criteriu de optim local, ci parcurge fiecare muchie de un număr suficient de ori, egal cu $(|V| - 1)$, și încearcă să o relaxeze de fiecare dată, pentru a îmbunătăți distanța până la nodul destinație al muchiei curente.

Motivul pentru care se face acest lucru este că drumul minim dintre sursă și orice nod destinație poate să treacă prin maximum $|V|$ noduri (adică toate nodurile grafului) și $(|V| - 1)$ muchii. Acest lucru se întâmplă pentru că drumurile minime nu conțin cicluri (nici pozitive, nici negative). Prin urmare, relaxarea tuturor muchiilor de $(|V| - 1)$ ori este suficientă pentru a propaga la toate nodurile informația despre distanța minimă de la sursă. Dacă la sfârșitul acestor $|E| * (|V| - 1)$ relaxări, mai poate fi îmbunătățită o distanță, atunci graful are un ciclu de cost negativ și problema nu are soluție.

Menținând notațiile anterioare, pseudocodul algoritmului este:

BellmanFord(sursa) :

```
// initializari
foreach (nod  $\in V$ ) //  $V$  = multimea nodurilor
    dacă muchie[sursa, nod]
        d[nod] = w[sursa, nod]
        P[nod] = sursa
    altfel
        d[nod] =  $+\infty$ 
        P[nod] = null
d[sursa] = 0
p[sursa] = null
// relaxari succesive; intrucat anterior este realizată deja
// o relaxare, sunt suficiente mai  $|V|-1$  (nr maxim de muchii
// intre oricare 2 noduri in graf) - 1
for (i = 1 to  $|V|-2$ )
    foreach ((u, v)  $\in E$ ) //  $E$  = multimea muchiilor
        if (d[v] > d[u] + w(u,v))
            d[v] = d[u] + w(u,v)
            p[v] = u;
// dacă se mai pot relaxa muchii
foreach ((u, v)  $\in E$ )
    if (d[v] > d[u] + w(u,v))
        fail ("exista cicluri negativ")
```

Complexitatea algoritmului este în mod evident $O(|E| * |V|)$.

VIII.4 Drumuri minime între oricare două noduri

VIII.4.1 ALGORITMUL FLOYD-WARSHALL

Algoritmii din aceasta secțiune determină drumul de cost minim dintre oricare două noduri dintr-un graf. Pentru a rezolva această problemă s-ar putea aplica unul din algoritmii de mai sus, considerând ca sursa fiecare nod pe rând, dar o astfel de abordare ar fi ineficientă. Algoritmul Floyd-Warshall

este bazat pe schema de programare dinamică. El compară toate drumurile posibile din graf dintre fiecare 2 noduri, și poate fi utilizat și în grafuri cu muchii de cost negativ.

Estimarea drumului optim poate fi reținut într-o structură tridimensională $d[v_1, v_2, k]$, cu semnificația – costul minim al drumului de la sursa v_1 la destinația v_2 , folosind ca noduri intermediare doar noduri din mulțimea $\{1 \dots k\}$. Dacă nodurile sunt numerotate de la 1, atunci $d[v_1, v_2, 0]$ reprezintă costul muchiei directe de la v_1 la v_2 , considerând $+\infty$ dacă aceasta nu există.

Pornind cu valori ale lui k de la 1 la $|V|$, scopul presupune identificarea celui mai scurt drum de la fiecare v_1 la fiecare v_2 , folosind doar noduri intermediare din mulțimea $\{1 \dots k\}$. La fiecare increment, este comparat costul deja estimat al drumului de la v_1 la v_2 până în acel moment, deci $d[v_1, v_2, k-1]$ obținut la pasul anterior, cu costul drumurilor de la v_1 la k și de la k la v_2 , adică $d[v_1, k, k-1] + d[k, v_2, k-1]$, obținute tot la pasul anterior. În final, $d[v_1, v_2, |V|]$ va conține costul drumului minim de la orice nod v_1 la orice nod v_2 .

Pseudocodul acestui algoritm este:

FloydWarshall(G) :

```

n = |V|
foreach ((i, j) ∈ (1..n, 1..n))
    d[i, j, 0] = w[i, j] // costul muchiei, sau infinit
for (k = 1 to n)
    foreach ((i, j) ∈ (1..n, 1..n))
        d[i, j, k] = min(d[i, j, k-1], d[i, k, k-1] + d[k, j, k-1])

```

Complexitatea temporală este $O(|V|^3)$, iar cea spațială este tot $O(|V|^3)$.

O complexitate spațială cu un ordin mai mic se obține observând că la fiecare pas nu este nevoie decât de matricea de la pasul precedent $d[i, j, k-1]$ și cea de la pasul curent $d[i, j, k]$. O observație și mai bună este că, de la un pas $k-1$ la k , estimările lungimilor nu pot decât să scadă, deci putem să lucrăm pe o singură matrice. Astfel, spațiul de memorie necesar este redus la $|V|^2$.

Rescris, pseudocodul algoritmului arată astfel dacă se integrează și stocarea nodului următor din calea de la u la v (această construcție este similară cu cea care salvează părintele de la algoritmi discutați anterior, doar că se memorează nodul următor pe drumul curent):

FloydWarshall(G) :

```

n = |V|
foreach ((i, j) ∈ (1..n, 1..n))
    d[i, j] = w[i, j] // costul muchiei, sau infinit
    urmator[i, j] = j

```

```

for (k = 1 to n)
    foreach (i,j) în (1..n,1..n)
        if (d[i, j] > d[i, k] + d[k, j])
            d[i,j] = d[i,k]+d[k,j])
            urmator[i,j] = urmator[i,k]

//tiparire cale de la nodul u la nodul v
path(u,v)
    tiparire u
    while (u!=v)
        u = urmator[u,v]
        tiparire u

```

Un exemplu de graf orientat pe care este aplicat algoritmul Floyd-Warshall este reprezentat în Figura 8.3. În acest exemplu, sunt afișate toate matricile intermediare pentru drumuri minime ce conțin doar noduri din mulțimea $\{1 \dots k\}$.

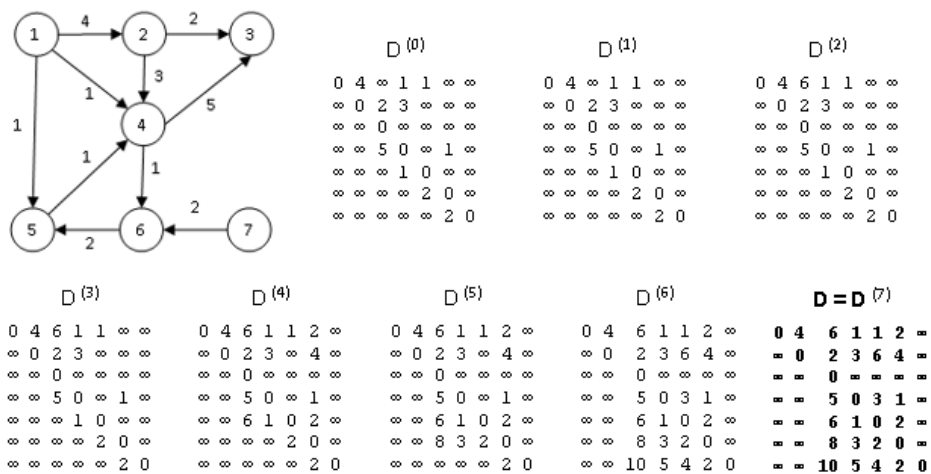


Figura 8.3 – Algoritmul Floyd-Warshall aplicat pentru un graf cu 7 noduri. Matricea din dreapta reprezintă drumurile minime între toate perechile de noduri.

VIII.5 Cazuri speciale

1. Pentru un graf **neorientat, fără cicluri** (un arbore) există un singur drum între oricare două noduri, care poate fi aflat printr-o simplă parcurgere DFS/BFS. Folosind diferite preprocesări [4], se poate calcula distanța între oricare două noduri în timp constant, *Complexitate*: $O(1)$.

2. Pentru un graf **orientat, fără cicluri** (un DAG [5]), răspunsul poate fi în continuare aflat printr-un DFS pornind de la sortarea topologică a grafului, dar trebuie ținut cont de faptul că pot exista mai multe drumuri între două noduri (este necesară salvarea pe parcurs a costului minim pentru a ajunge într-un anumit nod). *Complexitate:* $O(|V|+|E|)$

3. Pentru un graf unde toate muchiile au **cost egal**, distanța minimă de la un nod sursă la orice alt nod se poate determina printr-o parcurgere BFS (de asemenea, ținând cont de faptul că pot exista mai multe drumuri până la un anumit nod). *Complexitate:* $O(|V|+|E|)$

4. Pentru grafuri orientate, **rare** (relativ puține muchii), putem folosi algoritmul lui Johnson [6] pentru calcularea distanței minime între oricare două noduri din graf. În acest caz (graf rar), algoritmul lui Johnson are o complexitate mai mică decât algoritmul Floyd-Warshall.

Complexitate: $O(|V|^2 \log(|V|) + |V||E|)$

VIII.6 Concluzii și observații

Dijkstra

- Calculează drumurile minime de la o sursă către celelalte noduri;
- Nu poate fi folosit dacă există muchii de cost negativ;
- Complexitate minimă este $O(|V|\log(|V|) + |E|)$ utilizând heap-uri Fibonacci; în general $O(|V|^2 + |E|)$.

Bellman – Ford

- Calculează drumurile minime de la o sursă către celelalte noduri;
- Detectează existența ciclurilor de cost negativ;
- Complexitate: $O(|V| |E|)$.

Floyd – Warshall

- Calculează drumurile minime între oricare două noduri din graf;
- Poate fi folosit în grafuri cu cicluri de cost negativ, dar nu le detectează implicit (exercițiu: cum se modifică algoritmul pentru a permite acest lucru?);
- Complexitate $O(|V|^3)$.

VIII.7 Probleme propuse

Zburând prin Europa

1. Andrei primește o hartă a Europei pe care sunt marcate toate zborurile posibile de la o localitate la alta, respectiv costurile acestora. Ca primă evaluare, Andrei își dorește să afle dacă două localități sunt conectate, indiferent de numărul de escale sau de linia aeriană.
2. Ulterior, Andrei își dorește să afle toate localitățile aflate pe drumul optim de la un oras la altul.
3. Întrucât Andrei se simte depășit de numărul foarte mare de localități de pe hartă, vă rugăm să identificați costul unei rute optime de la o singură localitate (cea în care se află Andrei) la oricare alta localitate.
4. Discutând cu un prieten, Andrei descoperă că pe anumite rute companiile oferă reduceri semnificative, clienții fideli primind chiar recompense financiare dacă aleg anumite rute. Determinați în aceste circumstanțe costul optim de la localitatea curentă la oricare alta localitate.

În final, lui Andrei îi vine o idee ingenioasă și vă rugăm îl ajutați să descopere dacă exista posibilitatea de a specula recompensele liniilor aeriene pentru a circula la infinit între două localități, fără a plăti. În cazul identificării unei astfel de rute, vă rugăm să o afișați.

VIII.8 Indicații de rezolvare

- 1) Pentru implementarea închiderii tranzitive se aplică algoritmul Floyd-Warshall puțin modificat: $d[i, j] = (d[i, j] \text{ OR } (d[i, k] \text{ AND } d[k, j]))$.
- 2) Se aplică algoritmul Floyd-Warshall cu păstrarea părinților.
- 3) Se aplică algoritmul Dijkstra întrucât costurile muchiilor sunt pozitive.
- 4) În locul algoritmului Dijkstra trebuie aplicat algoritmul Bellman-Ford datorită costurilor negative, algoritm care permite totodată identificarea ciclurilor de cost negativ.

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). Introduction to Algorithms (3rd ed.). Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) Introducere în Analiza Algoritmilor. Bucuresti: Polirom
- [3] Fredman, Michael Lawrence; Tarjan, Robert E. (1987). "Fibonacci heaps and their uses in improved network optimization algorithms" (PDF). Journal of the Association for Computing Machinery 34 (3): 596–615.
- [4] Bender, Michael A.; Farach-Colton, Martín; Pemmasani, Giridhar; Skiena, Steven; Sumazin, Pavel (2005). "Lowest common ancestors in trees and directed acyclic graphs" (PDF). Journal of Algorithms 57 (2): 75–94.
- [5] Christofides, Nicos (1975), Graph theory: an algorithmic approach, Academic Press, pp. 170–174.
- [6] Johnson, Donald B. (1977), "Efficient algorithms for shortest paths in sparse networks", Journal of the ACM 24 (1): 1–13

IX Arbori minimi de acoperire

IX.1 Importanță și aplicații practice

Găsirea unui arbore minim de acoperire [1, 2] pentru un graf are aplicații în domenii variate. De exemplu, în rețele de calculatoare, telefonie, cablu TV, electricitate, sau drumuri rutiere se urmărește frecvent interconectarea mai multor puncte, cu un cost redus. STP (Spanning Tree Protocol) este un protocol de rutare care previne apariția buclelor într-o rețea locală (LAN), și se bazează pe crearea unui arbore de acoperire. Singurele legături active sunt cele care apar în acest arbore, iar astfel se evită bucele. Totodată, arborii minimi de acoperire sunt utili în algoritmi de aproximare pentru probleme NP-dure (problema comis-voiajorului, arbori Steiner), sau în clustering pentru detectarea de clustere cu forme neregulate [3].

IX.2 Descrierea problemei și a rezolvărilor

Dându-se un graf conex neorientat $G = (V, E)$, se numește *arbore de acoperire* al lui G un subgraf $G' = (V, E')$ care conține toate vârfurile grafului G și o submulțime minimă de muchii $E' \subseteq E$ cu proprietatea că unește toate vârfurile și nu conține cicluri. Cum G' este conex și aciclic, el este un arbore. Pentru un graf oarecare, există mai mulți arbori de acoperire.

Dacă asociem o funcție de cost ($w: E \rightarrow R$) pentru muchiile din G , fiecare arbore de acoperire va avea asociat un cost egal cu suma costurilor muchiilor conținute. Un arbore care are costul asociat mai mic sau egal cu costul oricărui alt arbore de acoperire se numește *arbore minim de acoperire* (*minimum spanning tree*, AMA) al grafului G . Un graf poate avea mai mulți arbori minimi de acoperire. Dacă toate costurile muchiilor din graf sunt diferite, există un singur AMA.

Primul algoritm pentru determinarea unui arbore minim de acoperire a fost scris în 1926 de Otakar Boruvka. În prezent, cei mai folosiți algoritmi sunt algoritmul lui Prim și cel al lui Kruskal. Toți trei sunt algoritmi greedy, și rulează în timp polinomial în funcție de numărul de noduri și arce. La fiecare pas, pentru a construi arborele se alege cea mai bună variantă posibilă la momentul respectiv. Generic, algoritmul pentru determinare a unui AMA se poate scrie astfel:

```

ArboreMinimDeAcoperire (G(V, E), w)
    MuchiiAMA =  $\emptyset$ ;
    while (|MuchiiAMA| != |V| - 1)
        găsește o muchie (u, v) care este sigură pentru
        MuchiiAMA;
        MuchiiAMA = MuchiiAMA  $\cup$  {(u, v)};
    return MuchiiAMA;

```

O muchie sigură este o muchie care se poate adăuga unei submulțimi de muchii ale unui arbore minim de acoperire, astfel încât noua mulțime obținută să aparțină tot unui arbore minim de acoperire. Inițial, *MuchiiAMA* este o mulțime vidă. La fiecare pas, se adaugă câte o muchie sigură, deci *MuchiiAMA* rămâne o submulțime a unui AMA. În consecință, la sfârșitul rulării algoritmului (când muchiile din mulțime unesc toate nodurile din graf), *MuchiiAMA* va conține de fapt arborele minim de acoperire dorit.

IX.2.1 ALGORITMUL KRUSKAL

Algoritmul a fost dezvoltat în 1956 de Joseph Kruskal. Determinarea arborelui minim de acoperire se face prin reuniuni de subarbori minimi de acoperire neconectați (disjuncți). Inițial, se consideră că fiecare nod din graf este un subarbore separat cu un singur nod. Apoi, la fiecare pas se selectează muchia de cost minim care unește doi subarbori disjuncți, și se realizează unirea celor doi subarbori. Muchia respectivă se adaugă la mulțimea *MuchiiAMA*, care la sfârșit va conține chiar muchiile din arborele minim de acoperire.

```

Kruskal(G(V, E), w)
    MuchiiAMA =  $\emptyset$ ;
    foreach (v  $\in$  V)
        MakeSet(v); //fiecare nod e un arbore diferit
    //sortează muchiile în ordinea crescătoare a costului
    //w(u, v)
    sort(E);
    //muchiiile se parcurg în ordinea data de sortare
    foreach ((u, v)  $\in$  E)
        if (FindSet(u) != FindSet(v))
            // capetele muchiei fac parte din
            // subarboridisjuncți
            // adaugă muchia la arbore și unește subarborii
            // corespunzători lui u și lui v
            MuchiiAMA = MuchiiAMA  $\cup$  {(u, v)};
            Union(u, v);
    return MuchiiAMA;

```

Bucloa principală *for* poate fi înlocuită cu o buclă *while*, în care se verifică dacă în MuchiiAMA există mai puțin de $|V| - 1$ muchii, pentru că orice arbore de acoperire are $|V| - 1$ muchii, iar la fiecare pas se adaugă o muchie sigură.

Complexitate: Timpul de execuție depinde de implementarea structurilor de date pentru mulțimi disjuncte (cu operațiile *MakeSet(u)*, *FindSet(u)* și *Union(u, v)*) [1]. Vom presupune că se folosește o pădure cu mulțimi disjuncte. Inițializarea se face într-un timp $O(|V|)$. Sortarea muchiilor în funcție de cost se face în $O(|E|\log|E|)$. În bucla principală se execută $|E|$ operații care presupun două operații de găsim a subarborilor din care fac parte extremitățile muchiilor și eventual o reuniune a acestor arbori, într-un timp $O(|E|)$.

Astfel, complexitatea totală este: $O(|V|) + O(|E|\log|E|) + O(|E|\log|E|) = O(|E|\log|E|)$. Cum $|E| \leq |V|^2$, și $\log(|V|^2) = 2 \log(|V|) = O(\log(|V|))$, rezultă o complexitate $O(|E|\log|V|)$.

În Figura 9.1 este evidențiată evoluția algoritmului lui Kruskal. Mulțimile disjuncte (care conțin mai mult de un nod) de la fiecare pas au culori diferite, iar muchiile din arborele minim de acoperire sunt îngroșate.

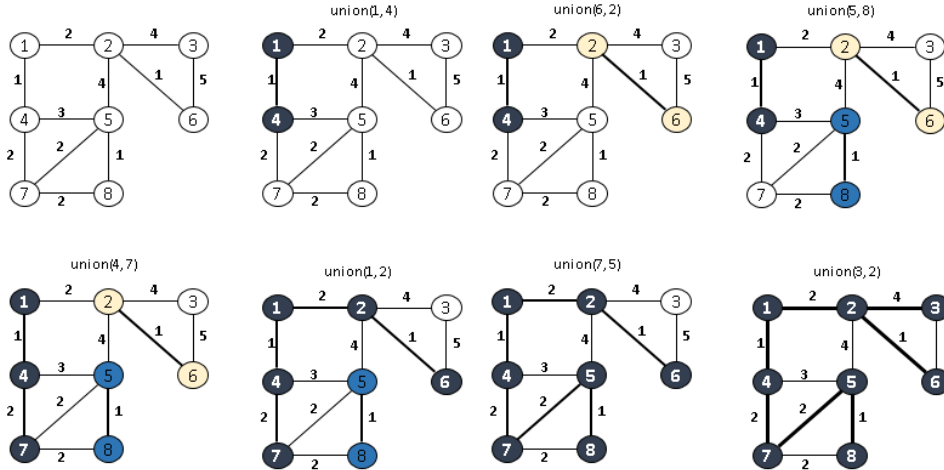


Figura 9.1 – Determinarea arborelui minim de acoperire folosind algoritmul lui Kruskal

IX.2.2 ALGORITMUL PRIM

Algoritmul a fost prima oară dezvoltat în 1930 de matematicianul ceh Vojtěch Jarník, și independent în 1957 de informaticianul Robert Prim, al cărui nume

l-a păstrat. Algoritmul consideră inițial că fiecare nod este un subarbor independent, similar cu algoritmul lui Kruskal. Însă, spre deosebire de acesta, nu se construiesc mai mulți subarbori care se unesc și în final ajung să formeze AMA, ci există un arbore principal, iar la fiecare pas se adaugă acestuia muchia cu cel mai mic cost care unește un nod din acest arbore principal cu un nod din afara sa. Nodul rădăcină al arborelui principal se alege arbitrar. În momentul în care s-au adăugat muchii care ajung în toate nodurile grafului, arborele AMA dorit a fost identificat. Abordarea seamănă cu algoritmul Dijkstra de găsim drumul minim între două noduri ale unui graf, doar că lipsește relaxarea muchiilor de-a lungul unei căi.

Pentru o implementare eficientă, următoarea muchie de adăugat la arborele principal trebuie să fie ușor de selectat. Vârfurile care nu sunt în arbore trebuie sortate în funcție de distanța până la acesta (de fapt costul minim al unei muchii care leagă nodul dat de un nod din arborele principal). Pentru aceasta, se poate folosi o structură de heap. Presupunând că (u, v) este muchia de cost minim care unește nodul u cu un nod v din arbore, se vor reține două informații:

- $d[u] = w[u, v]$ distanța minimă de la u la arbore
- $p[u] = v$ predecesorul lui u în drumul minim de la arbore la u , reținând practic muchia $(v, u) = (u, v)$, graful fiind neorientat.

La fiecare pas se va selecta nodul u cel mai apropiat de arborele principal, reunind apoi arborele principal cu subarborii corespunzător nodului selectat (care conține întotdeauna doar nodul respectiv). Se verifică apoi dacă există noduri mai apropiate de u decât de nodurile care erau anterior în arborele principal, caz în care trebuie modificate distanțele, dar și predecesorul. Modificarea unei distanțe impune și refacerea structurii de heap.

Prim($G(V, E)$, w , $root$)

```

MuchiiAMA =  $\emptyset$ ;
foreach ( $u \in V$ )
     $d[u] = \text{INF}$ ; //inițial distanțele sunt infinit
     $p[u] = \text{null}$ ; //și nu există predecesori
     $d[\text{root}] = 0$ ; //distanța de la rădăcină la arbore e 0
H = Heap( $V, d$ ); //se construiește heap-ul
while (!empty(H)) //cât timp mai sunt noduri neadăugate
     $u = \text{GetMin}(H)$ ; //se selectează cel mai apropiat nod u
    //se adaugă muchia care unește u cu un nod din AMA
    MuchiiAMA = MuchiiAMA  $\cup \{(u, p[u])\}$ ;
    //pentru toate nodurile adiacente lui u se verifică
    dacă trebuie făcute modificări
    foreach ( $v \in \text{succesori}(u)$ )
        if ( $w[u][v] < d[v]$ )

```

```

d[v] = w[u][v];
p[v] = u;
Heapify(v, H); //refacerea structurii de heap
MuchiiAMA = MuchiiAMA \ {(root, p[root])};
return MuchiiAMA;

```

Complexitate: Inițializările se fac în $O(|V|)$. Bucla principală *while* se execută de $|V|$ ori. Procedura *GetMin()* are nevoie de un timp de ordinul $O(\log|V|)$, deci toate apelurile vor dura $O(|V|\log|V|)$. Bucla *for* este executată în total de $O(|E|)$ ori, deoarece suma tuturor listelor de adiacență este $2|E|$. Modificarea distanței, a predecesorului, și refacerea heapului se execută într-un timp de $O(1)$, $O(1)$, respectiv $O(\log|V|)$. Astfel, bucla interioară *for* durează $O(|E|\log|V|)$.

În consecință, timpul total de rulare este $O(|V|\log|V| + |E|\log|V|)$, adică $O(|E|\log|V|)$, aceeași complexitate cu algoritmul Kruskal. Totuși, timpul de execuție al algoritmului Prim se poate îmbunătăți până la $O(|E| + |V|\log|V|)$, folosind heap-uri Fibonacci sau la $O(|V|^2)$ pentru grafuri dense, folosind o implementare cu vector (în loc de heap) pentru memorarea distanțelor $d[v]$.

Figura 9.2 oferă un exemplu al funcționării algoritmului lui Prim pentru un graf cu sursa nodul 1, împreună cu evoluția arborelui minim parțial de acoperire și a cozii de priorități (heap-ului) Q .

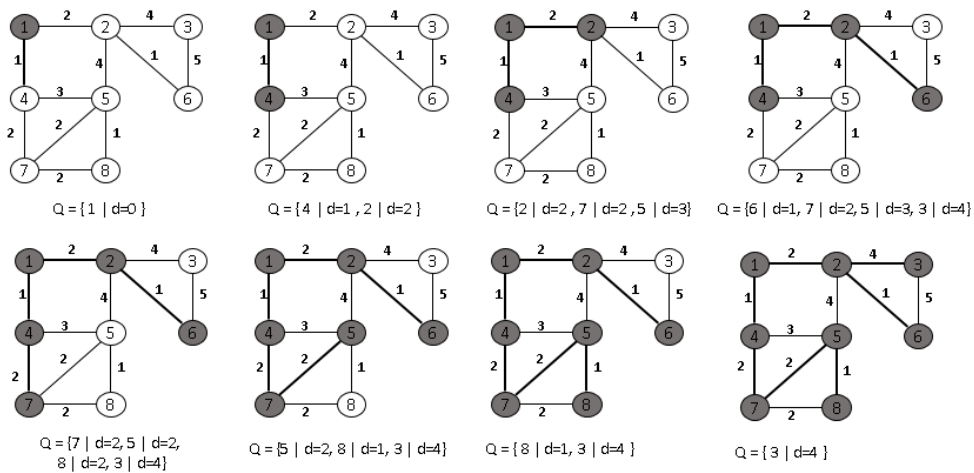


Figura 9.2 – Determinarea arborelui minim de acoperire folosind algoritmul lui Prim

IX.3 Concluzii și observații

Un arbore minim de acoperire al unui graf este un arbore care conține toate nodurile și, în plus, acestea sunt conectate prin muchii care asigură un cost total minim. Determinarea unui arbore minim de acoperire pentru un graf este o problemă cu aplicații în foarte multe domenii: rețele, clustering, prelucrare de imagini. Cei mai cunoscuți algoritmi, Prim și Kruskal, rezolvă problema în timp polinomial. Performanța algoritmilor depinde de modul de reprezentare a grafului și de structure de date folosite.

IX.4 Probleme propuse

Cablare optimă în rețele de date

Prietenul nostru Andrei a fost asignat drept noul șef al departamentului de rețelistică al companiei Nuke Cola. Sediul companiei are arondate N sucursale, iar Andrei trebuie să conecteze asigure conectivitate între toate locațiile folosind o lungime minimă de fibră optică, lucru care duce implicit la reducerea costurilor totale.

În cazul în care directorul general nu este de acord cu planul său, Andrei vrea să aibă un plan de backup, acesta reprezentând cea de-a doua cea mai bună soluție din punctul de vedere al lungimii totale de fibră optică utilizată.

Vă rugăm să îl ajutați pe Andrei să găsească configurația optimă având drept intrare distanțele între locațiile care pot fi conectate direct una cu cealaltă.

IX.5 Indicații de rezolvare

Cablare optimă în rețele de date

Inițial trebuie identificat arborele minim de acoperire (AMA) folosind fie algoritmul Prim, fie Kruskal.

Ulterior, trebuie identificat cel de-al doilea arbore minim de acoperire (2AMA) utilizând următoarea construcție optimizată. Pentru fiecare muchie $(u, v) \notin \text{AMA}$, se determină ciclul care s-ar obține prin adăugarea muchiei (u, v) la AMA, urmând ca un candidat pentru 2AMA să se obțină prin eliminarea muchiei de cost maxim, diferită de (u, v) , din acest ciclu nou creat. Astfel, în continuare este asigurată conectivitatea tuturor nodurilor (se elimină o

muchie de cost maxim dintr-un ciclu), iar $2AMA$ este costul minim al tuturor arborilor construiți anterior pornind de la fiecare muchie care nu este în AMA -ul inițial. Pentru a optimiza construcția ciclurilor conținând muchia (u, v) , se poate considera muchia de cost maxim din drumul de la u la v în AMA , cost care poate fi determinat cu ușurință folosind o parcurgere în lățime (BFS) în AMA .

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). Introduction to Algorithms (3rd ed.). Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) Introducere în Analiza Algoritmilor. Bucuresti: Polirom
- [3] Prim, H. (2011) A Minimum Spanning Tree Based Clustering Algorithm for High Throughput Biological Data. Ph.D. Dissertation. Mississippi State Univ., Mississippi State, MS, USA. Advisor(s) Burak Eksioglu.

X Flux Maxim

X.1 Importanță și aplicații practice

Un graf orientat poate fi utilizat pentru modelarea unui proces de transport într-o rețea între un producător s și un consumator t . Destinația nu poate consuma mai mult decât produce, iar cantitatea trimisă pe o cale nu poate depăși capacitatea aferentă de transport de pe muchiile folosite. Rețelele de transport folosite pentru modelarea problemelor de flux maxim [1-3] se regăsesc în multe situații reale, cum ar fi curgerea unui lichid în sisteme cu țevi, deplasarea pieselor pe benzi rulante, deplasarea curentului prin rețele electrice sau transmiterea informațiilor prin rețele de comunicare [4].

X.2 Descrierea problemei și a rezolvărilor

O problemă des întâlnită într-o rețea de transport este cea a găsirii fluxului maxim posibil prin arcele rețelei astfel încât:

- să nu fie depășite capacitățile arcelor;
- fluxul să se conserve pe drumul său de la producătorul s și consumatorul t .

Definiție 3.1

O rețea de transport este un graf orientat $G=(V, E)$ cu proprietățile:

- există două noduri speciale în V : s este nodul sursă (sau producătorul de flux) și t este nodul terminal (sau consumatorul de flux);
- este definită o funcție totală de capacitate $c:V \times V \rightarrow \mathbb{R}_+$ astfel încât:
 1. $c(u, v) = 0$ dacă $(u, v) \notin E$
 2. $c(u, v) \geq 0$ dacă $(u, v) \in E$
- pentru orice nod $v \in V \setminus \{s, t\}$ există cel puțin o cale $s \rightarrow v \rightarrow t$.

Definiție 3.2

Fluxul în rețeaua $G = (V, E)$ este o funcție totală $f: V \times V \rightarrow \mathbb{R}$ cu proprietățile:

- Restricție de capacitate:

$$f(u, v) \leq c(u, v), \forall (u, v) \in V \times V \text{ (fluxul printr-un arc nu poate depăși capacitatea acestuia)}$$
- Antisimetrie:

$$f(u, v) = -f(v, u), \forall u \in V, \forall v \in V$$
- Conservarea fluxului:

$$\sum_{v \in V} f(u, v) = 0, \quad \forall u \in V \setminus \{s, t\}, v \in V$$

Un flux negativ de la u la v este unul virtual (sau teoretic), el nu reprezintă un transport efectiv (sau net, practic), ci doar sugerează că există un transport fizic de la v la u (este o convenție asemănătoare cu cea făcută pentru intensitățile curenților într-o rețea electrică). Ultima proprietate ne spune că la trecerea printr-un nod intermediar, în afară de sursă și drenă, fluxul se conservă: suma fluxurilor ce intră într-un nod este 0 (ținând cont de convenția de semn stabilită).

Capacitatea reziduală a unui arc direct, definită ca $c_f(u, v) = c(u, v) - f(u, v)$, este cantitatea de flux adițional care mai poate fi transportat de la u la v , fără a depăși capacitatea $c(u, v)$.

Definiție 3.3

Fie o rețea de flux $G = (V, E)$, iar f fluxul prin G la un moment de timp dat. Numim **rețea reziduală** a lui G , indusă de fluxul f , o rețea de flux notată cu $G_f = (V, E_f)$, astfel încât

$$E_f = \{(u, v) \in V \times V \mid c_f(u, v) = c(u, v) - f(u, v) > 0\}$$

Este important de observat că E_f și E pot fi disjuncte: un arc rezidual (u, v) apare în rețeaua reziduală doar dacă capacitatea sa reziduală este strict pozitivă (ceea ce nu implică existența arcului în rețeaua originală, întrucât capacitate reziduală pozitivă se poate obține și cu $c(u, v) = 0$ și flux $f(u, v) < 0$).

Un **drum de ameliorare** (cale reziduală între s și t) este o cale $p = (u_1, u_2, \dots, u_k)$, unde $u_1 = s$ și $u_k = t$, în graful rezidual cu $c_f(u_i, u_{i+1}) > 0, \forall i = 1, 2, \dots, k-1$. Practic, un drum de ameliorare va reprezenta o cale în graf prin care se mai poate pompa flux adițional de la sursă la destinație. Astfel, capacitatea reziduală a unui drum de ameliorare p este cantitatea maximă de flux ce se poate transporta de-a lungul lui:

$$c_f(p) = \min\{c_f(u, v) \mid (u, v) \in p\}$$

Ulterior prezentării noțiunilor necesare pentru formalizarea problemei de flux maxim într-un graf, putem să prezentăm și cea mai utilizată metodă de rezolvare.

X.2.1 ALGORITMUL FORD-FULKERSON

Algoritmul Ford-Fulkerson este o metodă iterativă de găsire a fluxului maxim într-un graf care pleacă de la următoarea idee: cât timp mai există un drum de ameliorare (o cale de la sursă la destinație) în rețeaua reziduală aferentă fluxului curent prin graf, încă mai putem pompa pe această cale un flux suplimentar egal cu capacitatea reziduală a căii.

Acest algoritm reprezintă mai mult o schemă de rezolvare pentru că nu detaliază modul în care se alege drumul de ameliorare din rețeaua reziduală.

```
Ford_Fulkerson ( $G(V, E)$ ,  $s$ ,  $t$ )
   $f\{u, v\} = 0, \forall (u, v) \in V \times V$ 
   $|f_{max}| = 0$ 
  while ( $\exists p(s \rightarrow t)$  în  $G_f$  astfel încât  $c_f(u, v) > 0, \forall (u, v) \in p$ )
    identifică  $c_f(p) = \min\{c_f(u, v) \mid (u, v) \in p\}$ 
    foreach  $((u, v) \in p)$ 
       $f(u, v) = f(u, v) + c_f(p)$ 
       $f(v, u) = -f(u, v)$ 
       $|f_{max}| += c_f(p)$ 
  return  $|f_{max}|$ 
```

Complexitatea metodei în cazul cel mai defavorabil va fi $O(E * |f_{max}|)$ pentru că în ciclul *while* putem găsi, în cel mai rău caz, doar căi care duc la creșterea fluxului cu doar o unitate de flux la fiecare pas.

Corectitudinea algoritmului derivă din teorema **Flux maxim - tăietura minimă**. Numim o **tăietură a unui rețele de transport** o partiție (S, T) a nodurilor sale ($S, T \subset V, S \cup T = V, S \cap T = \emptyset$) cu proprietatea $s \in S$ și $t \in T$.

Teorema flux maxim – tăietura minimă: Pentru o rețea de flux $G(V, E)$ următoarele afirmații sunt echivalente:

- Fluxul f asigură fluxul maxim în G de la s la t ;
- Rețeaua reziduală G_f nu conține drumuri de ameliorare;
- Există o tăietură (S, T) a lui G astfel încât fluxul net prin tăietură este egal cu capacitatea acelei tăieturi.

Observație: Fluxul prin orice tăietură este egal cu cel maxim pentru că nu există o altă cale pe care ar putea ajunge flux de la sursă la destinație și care să nu treacă prin tăietură (ar încălca tocmai definiția ei); sau, altfel spus, valoarea unui flux transportat într-o rețea este egală de fluxul care trece prin orice tăietură. Pe de altă parte, fiecare tăietură are o capacitate diferită, în funcție de arcele din rețea care o traversează. Astfel, fluxul total va fi mărginit de cea mai mică capacitate a unei tăieturi. Dacă este îndeplinit punctul 3 al teoremei, atunci știm că acea tăietură nu poate fi decât una de capacitate minimă. Ultima încercare de a găsi o cale de la sursă la drenă va rezulta în găsirea doar a elementelor mărginite de o astfel de tăietură în cadrul căreia fluxul prin toate muchiile este la capacitate maximă (deci muchiile care traversează tăietura minimală au capacitate reziduală zero când s-a atins fluxul maxim în rețea).

După cum am menționat anterior, performanța algoritmului Ford-Fulkerson depinde de modul în care va fi ales drumul de ameliorare la fiecare pas al algoritmului. Se poate întâmpla ca o alegere nepotrivită a acestuia să ducă la timpi de execuție foarte mari, pentru grafuri unde cantitatea de flux maxim transportat în rețea ($|f_{\max}|$) este mare.

X.2.2 IMPLEMENTAREA EDMONDS-KARP

După cum reiese din pseudocodul anterior, metoda Ford-Fulkerson nu definește o soluție pentru alegerea drumului de ameliorare pe baza căruia se modifică fluxul în graf la fiecare pas. Implementarea Edmonds-Karp alege întotdeauna cea mai scurtă cale folosind o căutare în lățime în graful rezidual. Se poate demonstra [1] că lungimea căilor reziduale găsite astfel crește monoton cu fiecare nouă ameliorare.

```
Edmonds-Karp ( $G(V, E), s, t$ )
   $f(u, v) = 0, \forall (u, v) \in V \times V$ 
   $|f_{\max}| = 0$ 
  while (true)
     $p(s \rightarrow t) \leftarrow \text{BFS}(G_f, s, t)$ 
    if not  $\exists p(s \rightarrow t)$ 
      break;
    find  $c_f(p) = \min \{ c_f(u, v) \mid (u, v) \in p \}$ 
    foreach  $((u, v) \in p)$ 
       $f(u, v) = f(u, v) + c_f(p)$ 
       $f(v, u) = -f(u, v)$ 
       $|f_{\max}| += c_f(p)$ 
  return  $|f_{\max}|$ 
```

Plecând de la ideea că drumurile de ameliorare găsite au lungimi din ce în ce mai mari, se poate arăta că în această implementare fluxul se mărește de cel mult $O(V^* E)$ ori. Ținând cont de faptul că BFS-ul are o complexitate $O(V+E)$, rezultă complexitatea finală a algoritmului Edmonds-Karp: $O(V^* E^2)$.

În figura 10.1 este prezentat un exemplu de rulare a acestui algoritm pentru o rețea de transport cu sursa $s=1$ și drena $t=7$. La fiecare pas, sunt evidențiate fluxul prin graf, rețeaua reziduală corespunzătoare și calea de ameliorare aleasă (muchiile sunt punctate în graful rezidual). În final, este scoasă în evidență și una dintre tăieturile de capacitate minimă din graf.

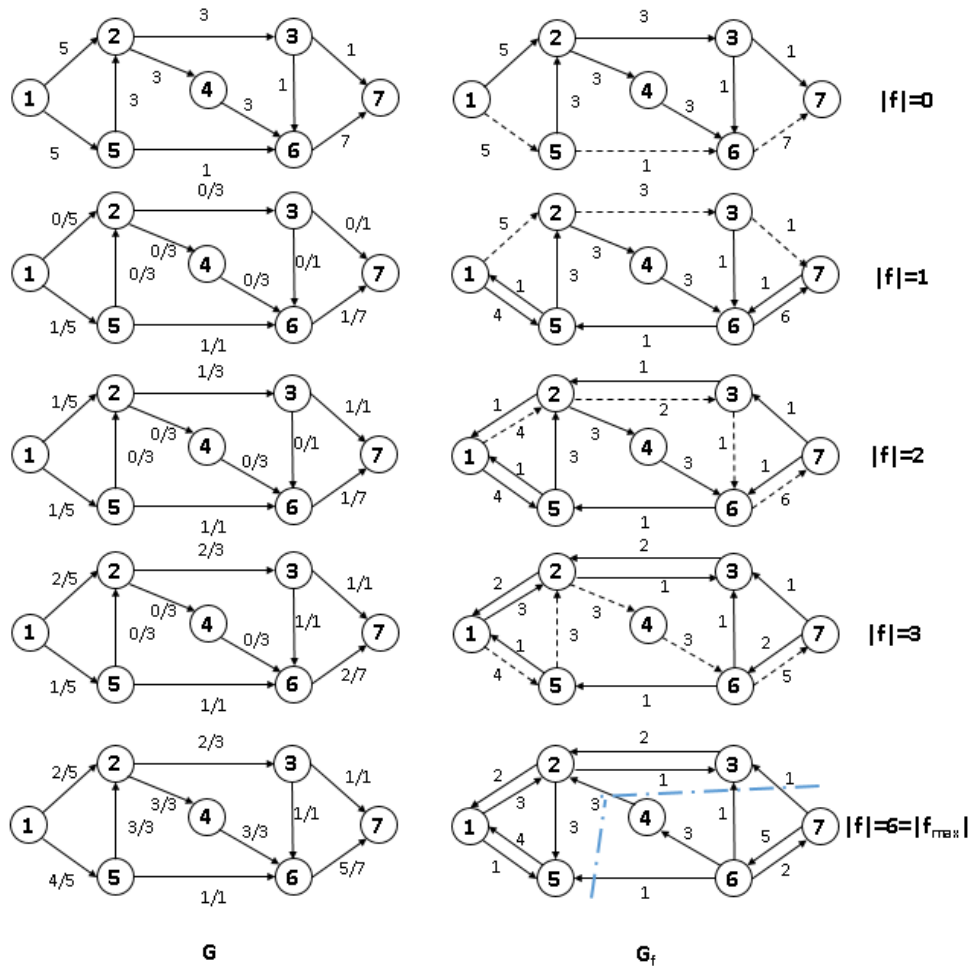


Figura 10.1 – Exemplu de rulare a algoritmului Edmonds-Karp – în stânga este reprezentată rețeaua de transport la fiecare pas, iar în dreapta graful rezidual

X.3 Variații ale problemei clasice

În rețelele de transport clasice aveam o sursă unică care putea produce orice cantitate de flux, destinația unică consuma oricât, orice nod intermediar conserva fluxul, iar singura constrângere a muchiilor era limitarea superioară a fluxului prin capacitate. Aceste condiții pot fi generalizate/modificate, însă rețelele obținute pot fi reduse (de cele mai multe ori) la una clasică.

X.3.1 SURSE ȘI DRENE MULTIPLE

Un graf cu multiple surse și/sau multiple drene poate fi redus la cel cunoscut prin adăugarea unei meta-surse legată de sursele inițiale prin muchii de capacitate nelimitată și, în mod similar, o meta-drenă legată de celelalte drene cu muchii de capacitate nelimitată. Global, comportamentul rețelei de flux nu se va schimba.

X.3.2 CUPLAJ BIPARTIT MAXIM

Fiind dat un graf neorientat $G = (V, E)$, un cuplaj este o submulțime de muchii M inclusă în E astfel încât, pentru toate vârfurile $v \in V$ există cel mult o muchie din M incidentă în v . Spunem că un vârf $v \in M$ este cuplat având cuplajul M dacă există o muchie în M incidentă în v . Un cuplaj maxim este un cuplaj de cardinalitate maximă.

În cazul grafurilor bipartite, mulțimea de vârfuri poate fi partiționată în două submulțimi $V = L \cup R$, unde L și R sunt disjuncte și toate muchiile din E sunt între noduri din L și noduri din R . Problema cuplajului maxim în grafuri bipartite poate fi rezolvată cu ajutorul noțiunii de flux, construind rețeaua de transport $G' = (V', E')$ pentru graful bipartit G . Vom alege sursa s și destinația t ca fiind noi vârfuri care nu sunt în V și vom construi $V' = V \cup \{s, t\}$. Arcele orientate ale lui G' sunt date de:

$$E' = \{(s, u): u \in L\} \cup \{(u, v): u \in L, v \in R \text{ și } (u, v) \in E\} \cup \{(v, t): v \in R\}.$$

Pentru a completa construcția, fiecare muchie din E' are atribuită capacitatea unitară.

X.3.3 REȚEA CU NODURI CE NU CONSERVĂ FLUXUL

Spre deosebire de s și t care produc/consumă oricât, un nod intermediar ar putea produce sau consuma o cantitate constantă de flux la trecerea prin el. În acest caz vom avea un invariant la nivel de nod care ia forma: $f_{\text{in}} - f_{\text{out}} = d_i$, unde d_i este cantitatea produsă suplimentar (>0) sau solicitată (<0) de un nod. Egalitatea ar putea fi transformată în:

$$(f_{\text{in}} + |d_i|) - f_{\text{out}} = 0, \text{ dacă } d_i < 0$$

$$f_{\text{in}} - (f_{\text{out}} + |d_i|) = 0, \text{ dacă } d_i > 0$$

Altfel spus, un nod ce consumă flux poate fi transformat într-unul ce conservă fluxul și are un în-arc adițional de capacitate $|d_i|$, iar un nod ce produce flux va avea un out-arc de aceeași capacitate. La nivelul întregii rețele se adaugă un nod sursă cu muchii către toate nodurile ce consumau flux și un nod destinație dinspre toate nodurile ce produceau flux în rețeaua inițială.

X.3.4 REȚEA CU LIMITE INFERIOARE DE CAPACITATE

Există cazuri în care se dorește ca valorile de pe muchiile rețelei să aparțină unui anumit interval $[inf, sup]$. Astfel, fluxul aferent unei astfel de muchii trebuie să respecte inegalitatea $inf \leq f \leq sup$.

Plecând tot de la condițiile de conservare la nivel de nod, intervalul $[inf, sup]$ se poate transla în $[0, sup-inf]$ în contextul în care nodul sursă a consumat inf unități de flux, iar nodul destinație a produs inf unități. Din exterior, entitatea nou creată este văzută ca acționând în același fel asupra fluxului ce o traversează. Ulterior, rețeaua va fi transformată aplicând modelul anterior pentru a se ajunge la rețeaua de flux clasică.

X.4 Concluzii și observații

Modelarea folosind fluxurile dintr-un graf are o aplicabilitate extrem de răspândită, fiind frecvent utilizată în reprezentarea de probleme de circulație a materialelor sau de optimizare. De asemenea, există o serie de probleme care inițial nu conțin elementele clare ale unei rețele de transport, dar care pot fi modelate folosind aceste rețele și apoi rezolvate folosind flux maxim sau cuplaj bipartit.

X.5 Probleme propuse

Networking

Pornind de la un graf orientat conex și două noduri u și v se cere să se determine următoarele elemente:

- Tăietura minimală a grafului afișată sub forma unei mulțimi de cardinal minim de muchii ce trebuie eliminate pentru a deconecta u și v .
- Pentru topologia dată, afișați numărul maxim de drumuri disjuncte între u și v , dar și drumurile în sine. Două drumuri sunt considerate disjuncte dacă nu au nicio muchie în comun.

Generarea de grafuri

Se dă o listă de N noduri, pentru fiecare cunoscându-se gradul de intrare și de ieșire al acestora (*In-degree* și *Out-degree*). Implementați un algoritm care să construiască un graf orientat G cu N noduri, care să satisfacă constrângerile de grade (de intrare și de ieșire) date. Se poate construi maxim o muchie între oricare două noduri.

X.6 Indicații de rezolvare

Networking

- Se aplică algoritmul Ford-Fulkerson, urmat de încă o parcurgere (BFS sau DFA) din nodul u pentru a detecta tăietura minimală. Această parcurgere va separa nodurile în 2 sub-mulțimi: o mulțime $S1$ cu noduri accesibile de u , respectiv mulțimea $S2=V\setminus S1$ cu noduri inaccesibile care sunt blocate de muchiile saturate. Astfel, muchiile care trebuie eliminate sunt toate muchiile din graful G care au capătul stânga în $S1$ și capătul dreapta în $S2$.
- Se aplică algoritmul Ford-Fulkerson adăugând fiecărei muchii o capacitate egală cu 1 și se afișează toate drumurile generate.

Generarea de grafuri

Se construiește un graf auxiliar pe care se aplică algoritmul Ford-Fulkerson. Acest graf are o sursă și o drenă virtuală și 2 straturi de N noduri (în total $2*N+2$ noduri), iar arcele se construiesc astfel:

- Capacitățile muchiilor de la sursa virtuală la fiecare nod din primul strat de noduri este egal cu In-Degree;

- Între cele 2 straturi de noduri se realizează un mesh/graf complet de muchii între nodurile primului strat cu capătul dreapta în al doilea strat (excluzând arcele de la nodul i din primul strat la nodul i din al doilea strat), toate de capacitate 1;
- Între fiecare nod din al doilea strat și drena virtuală se trasează arce de capacitate egal cu Out-Degree.

Raportat la modelarea anterioară, se poate opta și pentru inversarea In-Degree cu Out-Degree, urmând ca muchiile saturat din mijloc să reflecte graful solicitat.

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). Introduction to Algorithms (3rd ed.). Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) Introducere în Analiza Algoritmilor. Bucuresti: Polirom, Capitolul V Algoritmi pr grafuri: Fluxuri maxime într-un graf
- [3] A Labeling Algorithm for the maximum flow network problem, Appendix C, disponibil la <http://web.mit.edu/15.053/www/AMP-Appendix-C.pdf>
- [4] Prezentarea de aplicații specifice de flux maxim, disponibilă la <http://www.cs.princeton.edu/~wayne/cs423/lectures/max-flow-applications-4up.pdf>

XI Algoritmi euristici. Algoritmul A*

XI.1 Importanță și aplicații practice

Algoritmii de căutare euristică [1, 2] sunt folosiți în cazurile care implică găsirea unor soluții pentru probleme pentru care fie nu există un model matematic de rezolvare directă. În acest caz, este necesară o explorare a spațiului stărilor problemei pentru găsirea unui răspuns sau a unei soluții optime / sub-optime. Întrucât o mare parte dintre problemele din viața reală pornesc de la aceste premise, gama de aplicații a algoritmilor euristici este destul de largă. Proiectarea agenților inteligenți [3], probleme de planificare, proiectarea circuitelor VLSI [4], robotică, căutare web, algoritmi de aproximare pentru probleme NP-Complete [5], teoria jocurilor sunt doar câteva dintre domeniile în care căutarea informată este utilizată.

XI.2 Descrierea problemei și a rezolvărilor

XI.2.1 PREZENTARE GENERALĂ A PROBLEMEI

Primul pas în rezolvarea unei probleme folosind algoritmi euristici de explorare este definirea spațiului stărilor problemei, prin tuplul (S_i, O, S_f) – starea inițială, operatori, stări finale (corespunzătoare soluțiilor). Rezolvarea problemei folosește următoarele componente:

- **Funcția de expandare a nodurilor** – în cazul general, aceasta este o listă de perechi (acțiune, stare_rezultat). Astfel, pentru fiecare stare se enumeră toate acțiunile posibile precum și starea care va rezulta în urma aplicării respectivei acțiuni;
- **Predicatul pentru starea finală** – funcție care întoarce *adevărat* dacă o stare este soluție (scop) și *fals* altfel;
- **Funcția de cost** – atribuie o valoare numerică fiecărei căi generate în procesul de explorare. De obicei se folosește o funcție de cost pentru fiecare acțiune/tranziție, atribuind, astfel, o valoare fiecărui arc din graful stărilor.

În funcție de reprezentarea problemei, sarcina algoritmilor de căutare este de a găsi o cale pentru a ajunge din starea inițială într-o stare scop. Dacă algoritmul găsește o soluție atunci când mulțimea soluțiilor este nevidă

spunem că algoritmul este **complet**. Dacă algoritmul găsește și calea de cost minim către starea finală spunem că algoritmul este **optim**.

În principiu, orice algoritm de parcurgere sau de drumuri minime pentru grafuri discutat în capitolele anterioare poate fi utilizat pentru găsirea soluției unei probleme definite astfel. În practica, însă, mulți dintre acești algoritmi nu sunt utilizați în acest context, fie pentru că explorează mult prea multe noduri, fie pentru că nu garantează o soluție pentru grafuri definite implicit (prin stare inițială și funcție de expansiune).

Algoritmii euristici de căutare sunt algoritmi care lucrează pe grafuri definite ca mai sus și care folosesc o informație suplimentară, neconținută în definirea problemei, prin care se accelerează procesul de găsirea a unei soluții.

În cadrul explorării stărilor, fiecare algoritm generează un arbore, care în rădăcină va conține starea inițială. Fiecare nod al arborelui va conține următoarele informații:

- **Starea conținută** – $stare(nod)$;
- **Părintele nodului** – $\pi(nod)$;
- **Cost cale** – costul drumului de la starea inițială până la nod – $g(nod)$.

De asemenea, pentru fiecare nod definim și o funcție de evaluare $f(nod)$ care indică cât de promițător este un nod în perspectiva găsirii unui drum către soluție. (De obicei, cu cât $f(nod)$ este mai mic, cu atât nodul este mai promițător).

Am menționat mai sus că algoritmii de căutare euristici utilizează o informație suplimentară referitoare la găsirea soluției problemei. Această informație este reprezentată de o funcție h , unde $h(nod)$ reprezintă drumul *estimat* de la nod la cea mai apropiată stare soluție. Funcția h poate fi definită în orice mod, existând o singură constrângere:

$$h(n) = 0, \text{ unde } solutie(n) = \text{adev\c{a}rat}$$

XI.2.2 ALGORITMI DE CĂUTARE INFORMATĂ

Întrucât funcționează prin alegerea, la fiecare pas, a nodului pentru care $f(nod)$ este minim, algoritmi prezentați mai jos fac parte din clasa algoritmilor de căutare informată (există informație referitoare la direcția de urmat, iar nodul cel mai promițător la un anumit moment este explorat mereu

primul). Algoritmul *Best-First* ține cont doar de istoric (informații sigure), pe când algoritmul *A** estimează costul până la găsirea unei soluții.

De notat este faptul că algoritmi de parcurgere neinformată *BFS* și *DFS* sunt particularizări ale strategiei *Best-First*:

- pentru *BFS*: $f = \text{adâncime}(S)$
- pentru *DFS*: $f = -\text{adâncime}(S)$

Vom prezenta în continuare câțiva dintre cei mai importanți algoritmi de căutare euristică. Vom folosi pentru exemplificare, următoarea problemă: dată fiind o hartă rutieră a României – vezi Figura 11.1, sa se găsească o cale (de preferință de cost minim) între Arad și București.

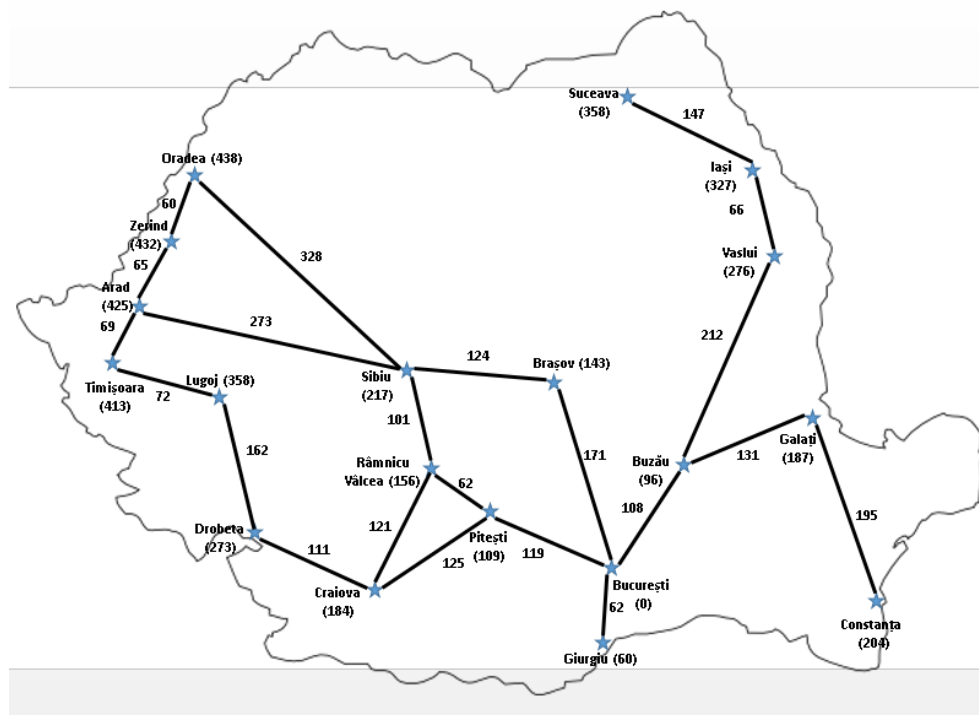


Figura 11.1 – Exemplu de graf cu distanțele rutiere între orașe – muchiile reprezintă drumurile rutiere împreună cu distanța pentru fiecare drum

Pentru această problemă, starea inițială indică că ne aflăm în orașul Arad, starea finală este dată de predicatul $\text{Oras_curent} == \text{București}$, funcția de expansiune întoarce toate orașele în care putem ajunge dintr-un oraș dat, iar funcția de cost indică numărul de km al fiecărui drum între două orașe, presupunând că viteza de deplasare este constantă. Ca euristică vom utiliza

pentru fiecare oraș distanța geometrică (în linie dreaptă) până la București. În Figura 11.1 euristica este evidențiată pentru fiecare nod din graf printr-un număr pus în paranteză în dreptul numelui de oraș.

XI.2.3 GREEDY BEST-FIRST

În cazul acestui algoritm se consideră că nodul care trebuie să fie expandat în pasul următor este cel mai apropiat de soluție, iar această aproximare utilizează funcția f definită anterior.

Pseudocodul acestui algoritm presupune:

Greedy Best-First(Sinitial, expand, h, solutie)

```

closed = {}
n = new-node()
state(n) = Sinitial
 $\pi(n)$  = nil
open = {n}
repeat
    if (open =  $\emptyset$ )
        return esec
    n = get_best(open) cu  $f(n)$  = min
    open = open - {n}
    if (solutie(state(n)))
        return build-path(n)
    if (n not in closed)
        closed = closed  $\cup$  {n}
        for each (s in expand(n))
            n' = new-node()
            state(n') = s
             $\pi(n')$  = n
            open = open  $\cup$  {n'}

```

În cadrul algoritmului se folosesc două mulțimi: *closed* – indică nodurile deja descoperite și expandate și *open* – nodurile descoperite, dar neexpandate. Mulțimea *open* este inițializată cu nodul corespunzător stării de start. La fiecare pas al algoritmului este ales din *open* nodul cu valoarea $f(n)$ cea mai mică (din acest motiv e de preferat ca *open* să fie implementată ca o coadă de prioritate). Dacă nodul se dovedește a fi o soluție a problemei, atunci este întoarsă ca rezultat calea de la starea inițială până la nod (mergând recursiv din părinte în părinte). Dacă nodul nu a fost deja explorat, atunci acesta este expandat, iar nodurile corespunzătoare stărilor rezultate sunt introduse în mulțimea *open*. Dacă mulțimea *open* rămâne fără elemente, atunci algoritmul nu a găsit nici un drum către soluție și întoarce *esec*.

Greedy Best-First urmărește mereu soluția care pare cea mai aproape de sursă. Din acest motiv nu se vor analiza stări care, deși par mai depărtate de soluție, produc o cale către soluție mai scurtă (a se vedea exemplul de rulare). De asemenea, întrucât nodurile din *closed* nu sunt niciodată reexplorate se va găsi calea cea mai scurtă către scop doar dacă se întâmplă ca această cale să fie analizată înaintea altor căi, către aceeași stare scop. Din acest motiv, algoritmul nu este optim. De asemenea, pentru grafuri infinite e posibil ca algoritmul să nu se termine, chiar dacă există o soluție. Rezultă că algoritmul nu îndeplinește nici condiția de completitudine.

În Figura 11.2 se prezintă rularea algoritmului Greedy Best-First pe exemplul dat mai sus. În primul pas algoritmul expandează nodul *Arad*, iar ca nod următor de explorat se alege *Sibiu*, întrucât are valoarea $f(n)$ minimă. Se alege în continuare *Brașov* după care urmează *București*, care este un nod final. Se observă însă că acest drum nu este însă minimal. Deși *Brașov* este mai aproape ca distanță geometrică de *București*, în momentul în care starea curentă este *Sibiu* alegerea optimală este *Râmnicu-Vâlcea*. În continuare ar fi urmat *Pitești* și apoi *București*, obținându-se un drum cu 13 km mai scurt între *Sibiu* și *București* (282 km) decât cel descoperit de Greedy Best-First (295 km).

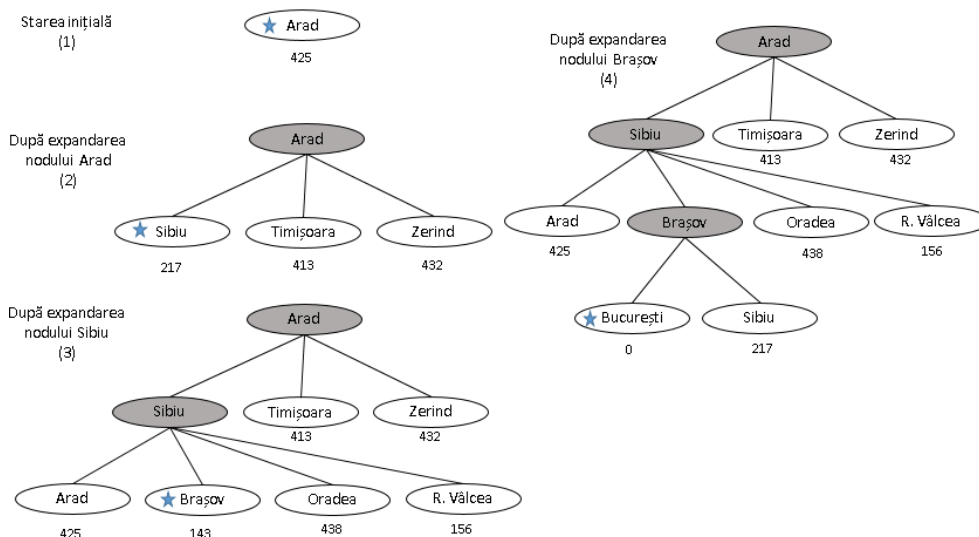


Figura 11.2 – Exemplu de explorare Greedy Best-First din nodul „Arad”

XI.2.4 A* (A STAR)

A* reprezintă cel mai cunoscut algoritm de căutare euristică. El folosește, de asemenea, o politică Best-First, însă nu suferă de aceleași defecte pe care le are Greedy Best-First definit mai sus. Acest lucru este realizat prin definirea funcției de evaluare astfel:

$$f(n) = g(n) + h(n)$$

A* evaluează nodurile combinând distanța deja parcursă până la nod cu distanța estimată de la acest nod până la cea mai apropiată stare scop sau soluție. Cu alte cuvinte, pentru un nod n oarecare, $f(n)$ reprezintă costul estimat al celei mai bune soluții care trece prin n . Această strategie se dovedește a fi completă și optimală dacă euristica $h(n)$ este admisibilă:

$$0 \leq h(n) \leq h^*(n)$$

unde $h^*(n)$ este distanța minimă de la nodul n la cea mai apropiată soluție. Cu alte cuvinte A* întoarce mereu soluția optimă, dacă o soluție există, atât timp cât algoritmul rămâne optimist și nu supraestimează distanța până la soluție pentru nici unul dintre noduri. Dacă funcția $h(n)$ nu este admisibilă, o soluție va fi în continuare găsită, dar nu se garantează optimalitatea. De asemenea, pentru a ne asigura ca vom găsi drumul optim către o soluție, chiar dacă acest drum nu este analizat primul, A* permite scoaterea nodurilor din *closed* și reintroducerea lor în *open* dacă a fost găsită o cale mai bună pentru un nod deja introdus în *closed* (drept urmare, dacă reușim să găsim un $g(n)$ mai mic pe o altă cale decât cea inițială).

Execuția algoritmului evoluează în felul următor: inițial se introduce în mulțimea *open* (organizată preferabil ca o coadă de priorități după $f(n)$) nodul corespunzător stării inițiale. La fiecare pas se extrage din *open* nodul n cu $f(n)$ minim. Dacă se dovedește ca nodul n conține chiar o stare scop atunci se întoarce calea de la starea inițială până la nodul n . Altfel, dacă nodul nu a fost explorat deja, acesta este expandat. Pentru fiecare stare rezultată, dacă aceasta nu a fost încă generată de un alt nod (nu este nici în *open*, nici în *closed*), atunci nodul este introdus în *open*. Dacă există un nod corespunzător stării generate în *open* sau în *closed*, se verifică dacă nu cumva nodul curent produce o cale mai scurtă către starea inițială s . Dacă acest lucru se întâmplă, se setează nodul curent ca părinte al nodului stării s și se corectează distanța g . Această corectare implică reevaluarea tuturor căilor care trec prin nodul s , deci acest nod va trebui reintrodus în *open* în cazul în care era inclus în *closed*.

Pseudocodul pentru algoritmul A* este prezentat în continuare:

```

A-Star(Sinitial, expand, h, solutie)
  n = new-node()
  stare(n) = Sinitial
  g(n) = 0
  π(n) = nil
  open = { n }
  closed = {}
  repeat
    if (open = ∅)
      return esec
    n = get_best(open) cu f(n) = g(n)+h(n) = min
    open = open \ {n}
    if (solutie(stare(n)))
      return build-path(n)
    if (n not in closed)
      closed = closed U {n}
      for each (s din expand(n))
        cost_n = g(n) + cost(stare(n), s)
        if (not(s in closed U open))
          n' = new-node()
          stare(n') = s
          π(n') = n
          g(n') = cost_n
          open = open U {n'}
        else
          n' = get(closed U open, s)
          if (cost_n < g(n'))
            π(n') = n
            g(n') = cost_n
            if (n' in closed)
              closed = closed \ { n' }
              open = open U { n' }

```

Algoritmul prezentat mai sus va întoarce calea optimă către soluție, dacă o soluție există și dacă euristica $h(n)$ este admisibilă.

Singurul inconvenient (care aduce calcule suplimentare, însă introduce o îmbunătățire care asigură completitudinea A*) față de Greedy Best-First este că sunt necesare reevaluările nodurilor din *closed*. Și această problemă poate fi rezolvată dacă se impune o condiție mai tare asupra euristicii h , și anume ca euristica să fie **consistentă** (sau **monotonă**):

$$h(n) \leq h(n') + \text{cost}(n, n'), n \in \text{expandare}(n)$$

Dacă o funcție este consistentă, atunci ea este și admisibilă [2]. Dacă euristica h îndeplinește și această condiție, atunci algoritmul A* este asemănător cu

Greedy Best-First, cu modificarea că funcția de evaluare este $f = g + h$, în loc de $f = h$.

În figura 11.3 se prezintă rularea algoritmului pe exemplul anterior. Se observă că euristica aleasă (distanța în linie dreaptă) este consistentă, deci admisibilă. Se observă că în pasul (6), după expandarea nodului *Brașov*, deși există o soluție în mulțimea *open*, aceasta nu este aleasă pentru explorare. Se va alege *Pitești*, întrucât $f(\text{nod}(\text{București})) = 568 > f(\text{nod}(\text{Pitești})) = 545$, semnificația acestei inegalități constând în posibilitatea de a avea un drum mai bun către *București* prin *Pitești* decât cel descoperit până atunci. În final, obținem un drum minim de lungime 555 km de la *Arad* la *București*.

XI.2.5 COMPLEXITATEA ALGORITMULUI A*

Pe lângă proprietățile de completitudine și optimalitate, A* mai are o calitate care îl face atrăgător. Pentru o euristică dată, orice algoritm de căutare complet și optim va explora cel puțin la fel de multe noduri ca A*, ceea ce înseamnă că A* este *optimal din punctul de vedere al eficienței*. În practică, însă, A* poate fi de multe ori imposibil de rulat datorită dimensiunii prea mari a spațiului de căutare. Singura modalitate prin care spațiul de căutare poate fi redus este prin găsirea unei euristici foarte bune – cu cât euristica este mai apropiată de distanța minimă reală față de o stare soluție, cu atât spațiul de căutare explorat de A* este mai redus. S-a demonstrat că spațiul de căutare începe să crească exponențial dacă eroarea euristicii față de distanța reală până la soluție nu are o creștere sub-exponențială [2]:

$$|h(n) - h^*(n)| \leq O(\log(h^*(n)))$$

Din păcate, în majoritatea cazurilor, eroarea crește liniar cu distanța până la soluție, ceea ce face ca A* să devină un algoritm mare consumator de timp, dar și de memorie. Întrucât în procesul de căutare se rețin toate nodurile deja explorate (mulțimea *closed*), în cazul unei dimensiuni mari a spațiului de căutare, cantitatea de memorie alocată căutării este în cele din urmă epuizată. Mai multe soluții au fost găsite pentru a adresa acest inconvenient. Una dintre acestea presupune utilizarea unor euristici care sunt mai strânse de distanța reală până la starea scop, deși nu sunt admisibile. Astfel, se obțin soluții mai rapide, dar nu se mai garantează optimalitatea acestora. Utilizarea acestei metode este recomandată când se dorește identificarea cât mai rapidă a unei soluții, indiferent de optimalitatea ei. Alte abordări presupun sacrificarea timpului de execuție pentru a mărgini spațiul de memorie utilizat, iar aceste alternative sunt prezentate în continuare.

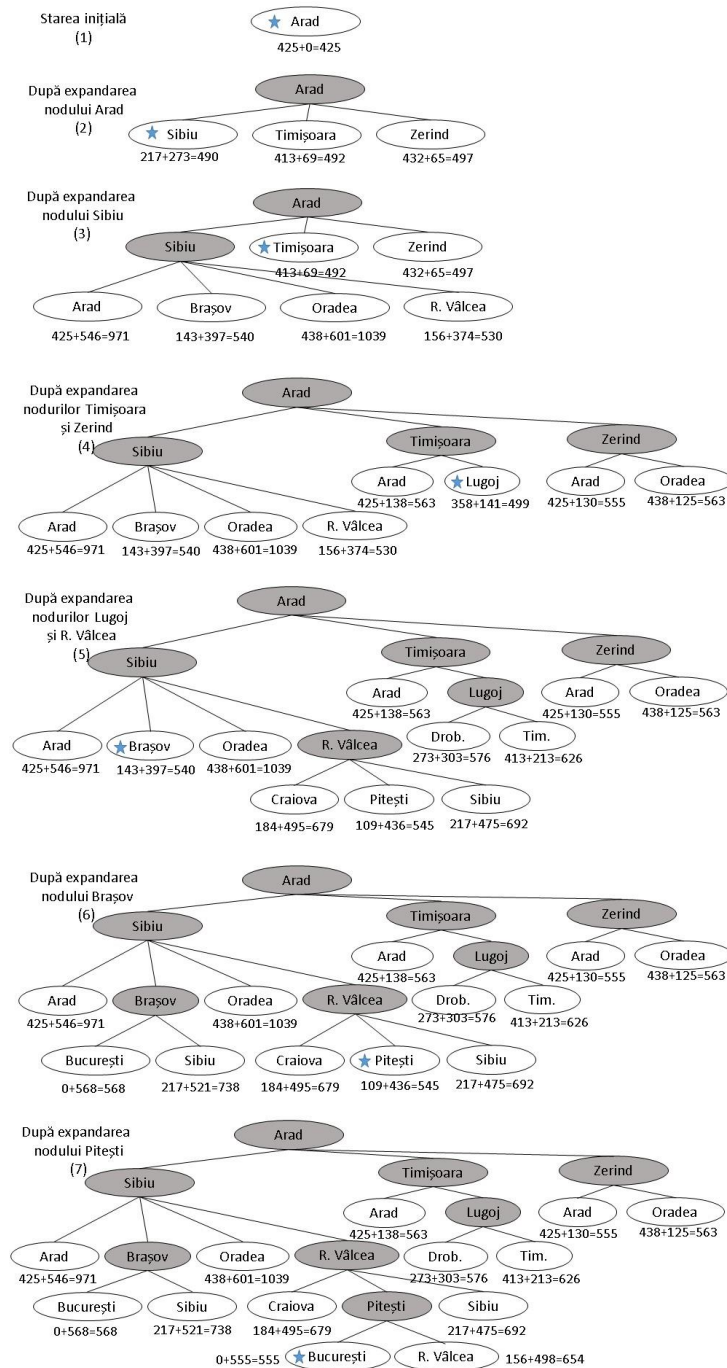


Figura 11.3 – Exemplu de explorare A* din nodul „Arad”

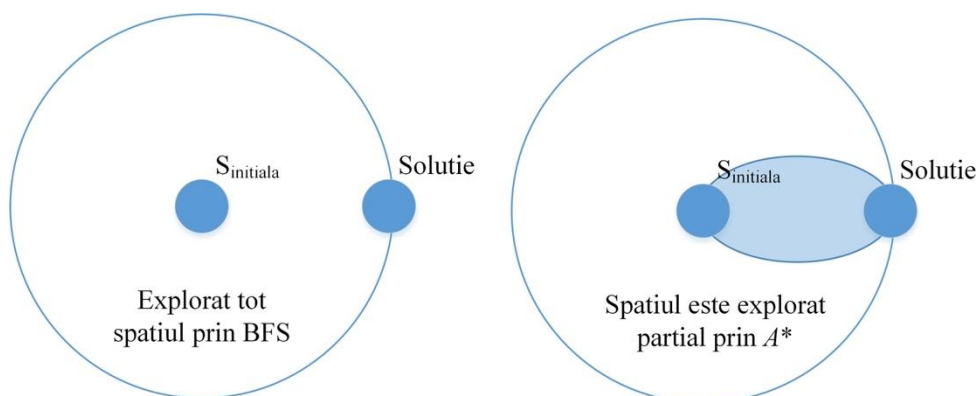


Figura 11.4 – Volumul spațiului de căutare în funcție de euristica aleasă

XI.2.6 IDA*, RBFS, MA*

A. IDA*

Iterative deepening A* [6] utilizează conceptul de adâncire iterativă în procesul de explorare a nodurilor – la un anumit pas se vor explora doar noduri pentru care funcția de evaluare are o valoare mai mică decât o limită dată, limită care este incrementată treptat până se găsește o soluție. IDA* nu mai necesită utilizarea unor mulțimi pentru reținerea nodurilor explorate și se comportă bine în cazul în care toate acțiunile au același cost. Din păcate, IDA* devine ineficient în momentul în care costurile sunt variabile.

B. RBFS

Recursive Best-First Search [7] funcționează într-un mod asemănător cu DFS, explorând la fiecare pas nodul cel mai promițător fără a reține informații despre nodurile deja explorate. Spre deosebire de DFS, se reține în orice moment cea mai bună alternativă sub forma unui pointer către un nod neexplorat. Dacă valoarea funcției de evaluare ($f = g + h$) pentru nodul curent devine mai mare decât valoarea căii alternative, drumul curent este abandonat și se începe explorarea nodului reținut ca alternativă. RBFS are avantajul ca spațiul de memorie crește doar liniar în raport cu lungimea căii analizate. Marele dezavantaj este că se ajunge de cele mai multe ori la reexpandări și reexplorări repetate ale acelorași noduri, lucru care poate fi dezavantajos mai ales dacă există multe cai prin care se ajunge la aceiași stare sau funcția *expand* este costisitoare computațional (vezi problema deplasării unui robot într-un mediu real). RBFS este optimal dacă euristica folosită este admisibilă.

C. MA*

Memory-bounded A* este o variantă a algoritmului A* în care se limitează cantitatea de memorie folosită pentru reținerea nodurilor. Există două versiuni – MA* [8] și SMA* [9] (Simple Memory-Bounded A*), ambele bazându-se pe același principiu. SMA* rulează similar cu A* până în momentul în care cantitatea de memorie devine insuficientă. În acest moment, spațiul de memorie necesar adăugării unui nod nou este obținut prin ștergerea celui mai puțin promițător nod deja explorat. În cazul în care există o egalitate în privința valorii funcției de evaluare se șterge nodul cel mai vechi. Pentru a evita posibilitatea în care nodul șters este totuși un nod care conduce la o cale optimă către soluție, valoarea f a nodului șters este reținută la nodul părinte într-un mod asemănător felului în care în RBFS se reține cea mai bună alternativă la nodul curent. Și în acest caz vor exista reexplorări de noduri, dar acest lucru se va întâmpla doar când toate căile mai promițătoare vor eșua. Deși există probleme în care aceste regenerări de noduri au o frecvență care face ca algoritmul să devină intractabil, MA* și SMA* asigură un compromis bun între timpul de execuție și limitările de memorie.

XI.3 Concluzii și observații

Deseori singura modalitate de rezolvare a problemelor dificile este de a explora spațiul stărilor acelei probleme. Algoritmii clasici pe grafuri nu sunt întotdeauna potriviți pentru căutarea în aceste grafuri corespunzătoare spațiului stărilor, fie pentru că nu garantează un rezultat, fie pentru că sunt ineficienți.

Algoritmii euristici sunt algoritmi care explorează astfel de grafuri folosindu-se de o informație suplimentară despre modul în care se poate ajunge la o stare scop mai rapid. A* este, teoretic, cel mai eficient algoritm de explorare euristică. În practică, însă, pentru probleme foarte dificile, A* implică un consum prea mare de memorie. În acest caz se folosesc variante de algoritmi care încearcă să minimizeze consumul de memorie în defavoarea timpului de execuție.

XI.4 Probleme propuse

Problema comis voiajorului (Ciclu *hamiltonian* de cost minim)

Se dau n orașe și rutele rutiere de interconectare dintre acestea. Se cere să se găsească un tur complet de lungime minimă (orașul de plecare și cel de întorcere sunt unul și același, iar toate celelalte orașe sunt străbătute o singură dată).

Pentru fiecare dintre variantele propuse de rezolvare se va evalua *numărul de stări explorate* și se va interpreta evoluția acestora raportat la soluția și euristica propusă.

- Să se rezolve problema folosind o abordare de tip Best-First.
- Să se rezolve problema folosind o euristică simplă A^* , admisibilă, definită în raport cu o singură localitate neexplorată în cadrul stării curente.
- Rafinați soluția de la punctul b), propunând o euristică mai informată prin evaluarea întregului orizont neexplorat (variabilele = localitățile care nu au fost deja parcurse în starea curentă).

XI.5 Indicații de rezolvare

În prima etapă se consideră minimul distanței deja acoperite de localitățile selectate. Ulterior, pe lângă funcția $g(x)$ definită anterior, se va considera $h(x)$ = distanța către cel mai apropiat oraș neparcurs. Euristica cea mai bună presupune utilizarea, în locul funcției $h(x)$ definită anterior, a costul arborelui minim de acoperire (AMA) aferent orașelor neparcurse.

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). Introduction to Algorithms (3rd ed.). Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) Introducere în Analiza Algoritmilor. Bucuresti: Polirom, Capitolul VII
- [3] Russel, S., Norvig, P. (2009). Artificial Intelligence: A Modern Approach. Prentice Hall, 2nd Edition
- [4] Sherwani, N. (1995). Algorithms for VLSI Physical Design Automation. Kluwer.
- [5] Applegate, D. L., Bixby, R. M., Chvátal, V., Cook, W. J. (2006), The Traveling Salesman Problem, ISBN 0-691-12993-2.

- [6] Korf, R. (1985). Depth-first Iterative-Deepening: An Optimal Admissible Tree Search. *Artificial Intelligence* 27: 97–109.
doi:10.1016/0004-3702(85)90084-0
- [7] Hatem, M., Kiesel, S., Ruml, W., (2015), Recursive Best-First Search with Bounded Overhead, In *Proceedings of the Twenty-ninth AAAI Conference on Artificial Intelligence*,
- [8] Chakrabarti, P. P., Ghose, S., Acharya, A., de Sarkar, S. C. (1989). Heuristic search in restricted memory (research note). *Artif. Intell.* 41, 2, 197-222. DOI=[http://dx.doi.org/10.1016/0004-3702\(89\)90010-6](http://dx.doi.org/10.1016/0004-3702(89)90010-6)
- [9] Russell, S. (1992). Efficient memory-bounded search methods. In *Proceedings of the 10th European conference on Artificial intelligence (ECAI '92)*, Bernd Neumann (Ed.). John Wiley & Sons, Inc., New York, NY, USA, 1-5.

XII Algoritmi aleatori

XII.1 Aplicații practice

Algoritmii aleatori se împart, în principal, în 2 clase:

- Algoritmi care rezolvă probleme de optimizare: soluția calculată de algoritm este garantat corectă, dar este aproximativă (nu este optimală). În acest caz, soluția suboptimală este considerată acceptabilă având o marjă de aproximare controlată probabilistic – *algoritmi de aproximare, algoritmi genetici și algoritmi aleatori de tip Las Vegas*;
- Algoritmi care rezolvă o problema ce acceptă o singură soluție: se renunță la exactitatea rezolvării preferându-se o soluție rapidă care se apropie cu o probabilitate suficient de mare de soluția exactă – corectitudinea **nu** este garantată – *algoritmi aleatori de tip Monte Carlo și stocastici (Markov)*.

Printre implicațiile practice ale algoritmilor aleatori se numără: optimizarea diversilor algoritmi (de exemplu, quicksort cu pivot aleatoriu), în general în vederea asigurării dispersiei corespunzătoare a valorilor; diverse inițializări (de exemplu, algoritmi genetici pentru inițializarea indivizilor) sau selecții de date după o distribuție prestabilită, în general Gaussiană); reducerea complexității unor probleme specifice.

XII.2 Descrierea problemei și a rezolvărilor

Algoritmii aleatori sunt folosiți pentru rezolvarea problemelor în următoarele situații:

- Este necesară micșorarea timpului de rezolvare a problemei prin relaxarea restricțiilor impuse soluțiilor;
- Este suficientă o singură soluție care se apropie cu o probabilitate măsurabilă de soluția exactă;
- În final, se poate obține o soluție suboptimală cu o marjă de eroare garantată prin calcul probabilistic.

Generatorul de numere aleatorii se află la baza construcției și funcționării algoritmilor aleatori. Astfel, pentru rulări diferite există șansa (chiar foarte

mare) ca algoritmul să se comporte diferit, chiar dacă datele de intrare, respectiv rezultatele sunt aceleași. Astfel, pentru același set de date de intrare, algoritmi de tip Las Vegas, respectiv Monte Carlo descriși ulterior se comportă diferit, chiar dacă rezultatele sunt aceleași.

XII.2.1 ALGORITMI LAS VEGAS

Caracteristici:

- Determină soluția corectă a problemei, însă timpul de rezolvare nu poate fi determinat cu exactitate;
- Creșterea timpului de rezolvare implică creșterea probabilității de terminare a algoritmului;
- După un timp infinit se ajunge la soluția corectă și algoritmul se termină sigur;
- Probabilitatea de găsire a soluției crește extrem de repede încât să se determine soluția corectă într-un timp suficient de scurt.

Complexitate teoretică:

$f(n) = \tilde{O}(g(n))$ dacă există $c > 0$, $n_0 > 0$ astfel încât:

- $\forall n > n_0, 0 < f(n) < c \alpha g(n)$ cu o probabilitate de cel puțin $1-n^{-\alpha}$, α fixat și suficient de mare;

Implicații:

Procentul algoritmilor Las Vegas care consumă cel mult $c \alpha g(n)$ resurse de calcul din totalul unei familii de algoritmi de complexitate $\tilde{O}(g(n))$ este $1-n^{-\alpha}$. Pentru α suficient de mare, pe măsură ce dimensiunea datelor de intrare – n – crește, există șanse foarte mici să se folosească un algoritm al familiei care nu respectă limita de complexitate.

Problemă: Identificarea duplicatelor

Identificarea liniilor duplicate dintr-o secvență [2] este o problemă ce poate fi rezolvată folosind un algoritm Monte Carlo. Capitolele unei cărți sunt stocate într-un fișier text sub forma unei secvențe nevide de linii. Fiecare secvență este precedată de o linie contor ce indică numărul de linii din secvență, iar specificul indică că fiecare astfel de secvență este lungă. Fiecare linie din fișier este terminată prin CR/LF, iar toate liniile din secvență au aceeași lungime. Fiecare secvență de linii conține o linie (titlul capitoului) ce se repetă și care apare în cel puțin $q = 10\%$ din numărul de linii al secvenței.

Pentru fiecare secvență de linii, se cere să se tipărească titlul capitolului (linia care se repetă).

Complexitate variantă iterativă: $O(n^2)$ în cazul cel mai defavorabil, când linia care se repetă apare numai spre finalul secvenței.

Rezolvare aleatoare:

```
selectie_linii(n, Secv) // Pp n = dim secv > 100
while (true)
    i = random(0,n-1) // selectez prima linie
    j = random(0,n-1) // selectez a doua linie
    if (i != j && linie(i,Secv) == linie(j,Secv))
        return linie(i,Secv) // s-a identificat linia
```

Complexitate variantă aleatoare: $\tilde{O}(\log^{-1}(1/a)\log(n)) = \tilde{O}(\log(n))$, unde $a = (1 - q*(q-1)/10000)$, $q=10$ – probabilitatea de regăsire a titlului capitolului (fără a împărți la 100 – a se vedea numitorul).

Observații:

De exemplu pentru $n = 100$ și $q = 10\%$, după 3500 de iterații, probabilitatea ca soluția să fie corectă poate fi considerată 1; dacă $q = 30\%$, atunci numărul de iterații scade la 500. Aproprierea probabilității de 1 este atât de mare încât precizia de calcul cu 12 zecimale nu mai asigură obținerea valorii exacte și, practic, terminarea algoritmului devine certă.

XII.2.2 ALGORITMI MONTE CARLO

Caracteristici:

- Determină o soluție a problemei care e garantat corectă doar după un timp infinit de rezolvare – soluție aproximativă;
- Presupun un număr finit de iterații după care răspunsul nu este garantat corect;
- Creșterea timpului de rezolvare implică creșterea probabilității ca soluția găsită să fie corectă;
- Soluția găsită într-un timp acceptabil este aproape sigur corectă (există o probabilitate foarte mică ca soluția să nu fie corectă).

Complexitate teoretică:

$f(n) = \tilde{O}(g(n))$ dacă există $c > 0$, $n_0 > 0$ astfel încât:

- $\forall n > n_0, 0 < f(n) < c \alpha g(n)$ cu o probabilitate de cel puțin $1-n^{-\alpha}$, α fixat și suficient de mare;
- Probabilitatea ca soluția determinată de algoritm să fie corectă este de cel puțin $1-n^{-\alpha}$.

Implicații:

Procentul algoritmilor Monte Carlo care consumă cel mult $c \alpha g(n)$ resurse de calcul din totalul unei familii de algoritmi de complexitate $\tilde{O}(g(n))$ pentru a găsi o soluție corectă cu o probabilitate de cel puțin $1-n^{-\alpha}$ este $1-n^{-\alpha}$.

Pentru α suficient de mare, mai ales cu cât n este mai mare, există șanse foarte mici să se folosească un algoritm al familiei care nu respectă limita de complexitate și *nu se termină cu o soluție corectă*.

Problemă: Testarea dacă un număr dat n este prim.

Complexitate variantă clasică: $O(\sqrt{n}) = O(2^{k/2})$ unde k = nr. de biți alocați pentru a reprezenta n . Această soluție este ineficientă pentru numere mari cărorora am dori să le testăm primalitatea și vom introduce un algoritm aleatoriu de tip Monte Carlo care are o complexitate mult scăzută, însă nu asigură corectitudinea soluției [2].

Rezolvare aleatoare folosind teorema lui Fermat (dacă n este prim, atunci pentru $\forall 0 < x < n, x^{n-1} \bmod n = 1$):

```
prim1(n, α) // detectează dacă n e număr prim
    if (n <= 1 || n mod 2 == 0)
        return false
    limit = limita_calcul(n, α) //nr min pași pt sol corectă cu
                                //P=1-n^-α
    for (i = 0 ; i < limit ; i++)
        x = random(1, n-1) // alegem un număr aleator
        if (pow_mod(x, n) != 1)
            return false // T. Fermat
    return true

pow_mod(x, n) // calculează logaritmic x^{n-1} mod n
    r = 1
    for (m = n - 1 ; m > 0 ; m = m / 2)
        if (m mod 2 != 0) // testez dacă m e par sau nu
            r = x*r mod n
        x = (x*x) mod n
    return r
```

Problema acestei abordări constă în faptul că *nu putem stabili* cu exactitate care este *limita de calcul*.

Pornind de la următoarea teoremă: Pentru orice număr prim ecuația $x^2 \bmod n = 1$ are exact 2 soluții: $x_1 = 1$ și $x_2 = n - 1$, obținem următoarea definiție pentru $x = \text{martor al divizibilității lui } n$: Fie $n > 1$ și $0 < x < n$ două numere astfel încât $x^{n-1} \bmod n \neq 1$ sau $x^2 \bmod n \neq 1$, $x \neq 1$ și $x \neq n - 1$.

```

prim2(n, α)
    if (n <= 1 || n mod 2 == 0)
        return false
    limit = limita_calcul(n, α)
    for (i = 0 ; i < limit ; i++)
        x = random(1, n-1)
        if (martor_div(x, n)) return false
    return true

martor_div(x, n) // determina dacă X=martor al divizibilitatii
    r = 1;
    y = x;
    for (m = n - 1 ; m > 0 ; m = m / 2)
        if (m mod 2 != 0) // putere impara
            r = y*r mod n
            z = y // salvam valoarea lui y
            y = y*y mod n // calculam y^2 mod n
        if (y==1 && z!=1 && z!=n-1)//verificam T. anterioară
            return true
    return r != 1 // T. Fermat

```

Complexitate: $\tilde{O}(\log^2(n)) = \tilde{O}(k^2)$

XII.3 Concluzii și observații

Metodele descrise adresează o plajă largă de probleme, iar abordările prezentate pot duce la scăderi drastice a timpilor de execuție, chiar dacă nu există în anumite circumstanțe certitudinea unei soluții optime/corecte.

XII.4 Probleme propuse

k-Means

Fie n puncte într-un spațiu bi-dimensional. Se dorește o grupare a acestora în k cluster – un cluster este un grup de puncte situate într-o vecinătate spațială care să maximizeze distanța internă (coeziune intra-cluster) și să asigure o cuplare slabă inter-cluster.

De exemplu, pentru setul de intrare:

X	2	3	2	3	3	3	4	5	8	8	9	10	10	10
Y	9	10	5	4	3	2	1	1	2	3	4	5	6	7

Se poate observa o grupare naturală în 3 cluster:

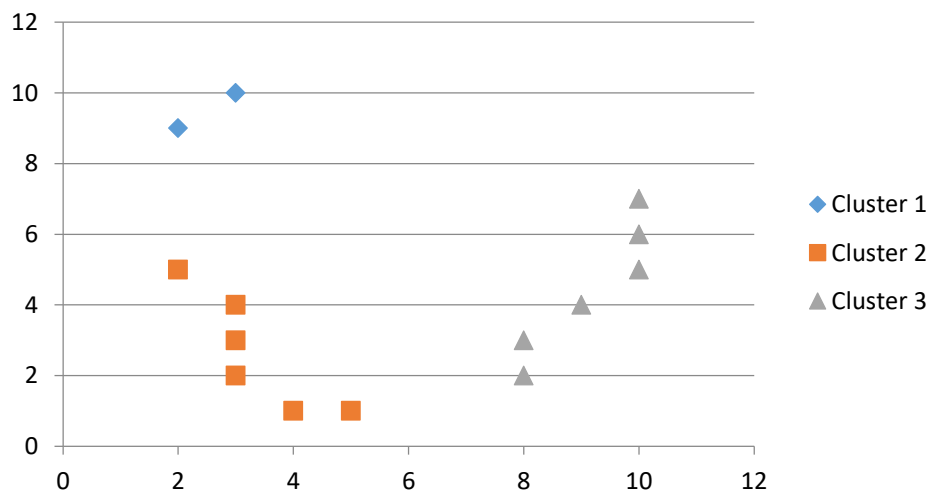


Figura 12.1 – Exemplu de clustering pentru setul de intrare format din 14 puncte

Pașii algoritmului sunt următorii (pentru un n și un k predefinite):

- Se selectează k puncte random din spațiu care vor fi centroizii inițiali ai clusterelor.
- Se efectuează iterativ următorii pași, până când atribuirile fiecărui punct la un cluster rămân neschimbate între doi pași consecutivi:
 - *Asignarea* fiecărui punct unui cluster în mod greedy pentru fiecare punct în parte (acel centroid care minimizează distanța euclidiană către centroizii din pasul curent este minimă);

- *Recalcularea centroidului* drept media aritmetică a coordonatelor punctelor asignate (se face acest lucru pentru fiecare coordonată).

Pentru un set de date și un k stabilit se vor determina clusterelor aferente. Se va implementa și o optimizare a selecției inițiale de puncte pornind de la principiul că acestea ar trebui să fie geometric cât mai dispersate (se dorește maximizarea distanței între centroizii inițiali).

XII.5 Indicații de rezolvare

***k*-Means**

Se implementează cele 2 etape specifice algoritmilor de tip Expectation Maximization:

- Asignarea fiecărui nod la unul dintre cele k cluster (distanța minimă euclidiană către centrozii din pasul curent este minimă);
- Recalcularea centroidului drept media aritmetică a coordonatelor punctelor asignate fiecărui cluster.

Pentru o asignare inițială mai eficientă, se pornește cu un punct random n_0 . Ulterior, fiecare punct n_i , cu $1 \leq i \leq k$, se alege pe considerentul că suma distanțelor până la toate punctele anterioare este maximă. Astfel, pentru n_1 se urmărește maximizarea distanței la n_0 , pentru n_2 suma distanțelor la n_1 și n_0 , etc. În final, punctele $n_{1...k}$ (n_0 este exclus) reprezintă alocările inițiale care asigură o dispersie cât mai mare a centroizilor.

Referințe

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (Eds.). (2009). Introduction to Algorithms (3rd ed.). Cap. 8.3. Cambridge, MA: MIT Press.
- [2] Giumale, C. (2004) Introducere în Analiza Algoritmilor. Bucuresti: Polirom, Capitolul VI.1

Anexa 1 – Convenții de programare (Coding style)

Principii

Pe lângă rezolvarea eficientă a problemelor și înțelegerea algoritmilor prezentați în această carte, este important pentru programatorul începător să înțeleagă că există aspecte suplimentare foarte importante pentru scrierea unor programe eficiente. În această anexă vom introduce câteva elemente de bază despre convențiile de programare (codare; redactare a codului).

Stilul de „redactare” („*coding standard*”, „*code convention*”, „*coding style*”) al programelor are un impact major asupra celor care parcurg ulterior codul, inclusiv asupra capacității de reamintire chiar a persoanei care a scris secvența respectivă. O **indentare** corespunzătoare ajută dezvoltatorii software oferind:

- Claritate;
- Structurare;
- Viteză mult mai mare de regăsire și reamintire a anumitor aspecte.

Totodată, stilul de codare ține și de gradul de profesionalism care se dorește a fi exprimat în cod, **comentariile** marcând:

- Coeziunea ideilor;
- Structurarea acestora;
- Claritatea prezentării;
- Ordonarea acestora din prisma organizării personale.

De asemenea, **comentariile ar trebui să exprime pe scurt detaliile de implementare într-un limbaj tehnic**, ghidând astfel programatorii la parcurgerea codului. Mai mult, comentariile sunt foarte utile chiar și pentru persoana care a scris codul respectiv, dacă dorește să îl înțeleagă rapid după o anumită perioadă de timp.

Desigur, există particularități de stil specifice fiecărui limbaj de programare, precum și caracteristici individuale care țin de preferințele personale ale dezvoltatorului. Ulterior pot apărea constrângeri generate de specificul echipei sau companiei (de exemplu, companiile mari au propriile standarde

de codare recomandate). Însă, există un set general de reguli care merită urmat.

Prezentare generală

o Formatare (indentare) generală

```
if (scor >=0 && scor <=100) {
    return 1;
} else {
    return 0;
}
```

o Spații în cadrul structurilor și între operatori

```
for (int i = 0; i <= 100; i++) {
    printf("%d", i);
}
```

o Tab-uri pentru aliniere

```
int          ix;          // Index
double       sum;         // Accumulator pentru suma
```

Nume variabile/funcții, logică precum și alte tehnici

o Nume adecvate și sugestive pentru variabile/funcții

```
int scorValid (int scor) {
    if (scor >=0 && scor <=100)
        return 1;
    else
        return 0;
}
```

o Valori boolene în structuri de decizie, doar pentru efect stilistic întrucât codul generat alternativ este oricum optimizat de compilator:

```
return (scor >=0 && scor <=100);
```

o Comparații de tip Left-hand:

```
if ( a = 42 ) { ... }    // Eroare logică
if ( 42 = a ) { ... }    // Eroare de compilare
```

o Cicliuri și structuri de control – utilizarea acoladelor și indentarea corespunzătoare nivelului de imbricare (evitarea problemelor care pot apărea de exemplu în Python datorate indentării greșite):

```
for (int i = 0; i <= 100; i++) {
    printf("%d", i * 2);
}
```



```
printf("Finalul buclei");
```

- o **Liste** – elementele corespunzătoare sunt plasate pe linii diferite:

```
const char *zile[] = {  
    "luni",  
    "marti",  
    "miercuri",  
    "joi",  
    "vineri",  
};
```

Caracteristici specifice fiecărui limbaj se regăsesc în referințele [1-7].

Alte aspecte care trebuie luate în vedere la scrierea elegantă a codului:

- **Modularitatea programelor** (dimensiune recomandată este de 20 de linii de cod per funcție sau procedură), totodată asigurând creșterea lizibilității codului, a posibilităților de reutilizare; se dorește, de asemenea, obținerea unei cuplări strânse în cadrul aceluiași modul, precum o cuplare cât mai slabă între module diferite din perspectiva proiectării arhitecturale;
- **Folosirea de constante/variabile** în loc de valori hard-coded;
- **Reutilizarea codului**, evitând astfel situații de repetare de tip „copy-paste” a unor secțiuni de cod, care în urma unor modificări ulterioare pot duce la anumite situații dificile de tip debugging;
- **Documentarea codului**, comentariile reprezentând un aspect important al scrierii elegante de cod.

Referințe

[1] Java Code Conventions, disponibil la

<http://www.oracle.com/technetwork/java/codeconventions-150003.pdf>
(original coding standard)

<http://www.oracle.com/technetwork/java/codeconvtoc-136057.html>
(updated version)

[2] Google C++ Style Guide, disponibil la

<https://google.github.io/styleguide/cppguide.html>

[3] C Coding Standard, disponibil la

https://www.gnu.org/prep/standards/html_node/Writing-C.html

[4] C++ Layout and Comments, disponibil la

<http://geosoft.no/development/cppstyle.html#Layout>

[5] Brad Abrams, Design Guidelines, Managed code and the .NET Framework, disponibil la

<http://blogs.msdn.com/b/brada/archive/2005/01/26/361363.aspx>

[6] Mozilla Coding Style Guide, disponibil la

https://developer.mozilla.org/en-US/docs/Mozilla/Developer_guide/Coding_Style

[7] PHP::PEAR Coding Standards, disponibil la

<http://pear.php.net/manual/en/standards.php>